NOAA Technical Memorandum OAR GSD-78

DOI: 10.25923/dhza-n042

TROPoe User Guide: v1.0

David D. Turner, Joshua Gebauer, Tyler Bell, and Bianca Adler

February 2026

National Oceanic and Atmospheric Administration
Oceanic and Atmospheric Research
Global Systems Laboratory
Boulder, Colorado, USA

noaa NATIONAL OCEANIC AND ATMOSPHERIC ADMINISTRATION / Office of Oceanic and Atmospheric Research

NOAA Technical Memorandum OAR GSD-78

TROPoe User Guide: v1.0

David D. Turner[1]
Joshua Gebauer[2,3]
Tyler Bell[4]
Bianca Adler[5,6]

[1] NOAA Global Systems Laboratory, Boulder, CO
[2] Cooperative Institute for Severe and High Impact Weather (CIWRO), Norman, OK
[3] NOAA National Severe Storms Laboratory, Norman, OK
[4] School of Meteorology, University of Oklahoma, Norman, OK
[5] Cooperative Institute for Research in Environmental Sciences (CIRES), University of Colorado, Boulder, CO
[6] NOAA Physical Sciences Laboratory, Boulder, CO

Acknowledgements

This research has been primarily supported by funding from NOAA via the Global Systems Laboratory, National Severe Storms Laboratory, and the NOAA ASRE and the VORTEX-Southeast / USA programs. There have been a large number of users of TROPoe output, and many have offered comments and suggestions which we have attempted to capture in development of the algorithm. Mention of any particular technology from a private vendor does not imply endorsement of the company. The scientific results and conclusions, as well as any views or opinions expressed herein, are those of the author(s) and do not necessarily reflect those of NOAA or the Department of Commerce. This effort was supported in part by the NOAA Cooperative Agreements with CIWRO and CIRES.

Abstract

This document serves as a user's guide for the TROPoe (tropospheric remotely observed profiling via optimal estimation) software package. TROPoe was designed to retrieve thermodynamic profiles and cloud properties from ground-based microwave radiometers and infrared spectrometers. It uses a physical-iterative approach and provides a full error characterization of each retrieval including information content. Additional observations that provide further constraints on the retrieved profiles are easily included. A large number of fields are included in the output file, including derived fields (e.g., planetary boundary layer height, convective available potential energy, etc. and their uncertainties) and other fields that can be used to diagnose the quality of the retrieval itself. TROPoe has been packaged in a Docker software container allowing it to be easily ported to other computer systems, ranging from laptops to high-performance computers.

Table of Contents

1) Introduction

Thermodynamic profiles are considered critical for a vast range of both operational and research objectives (e.g., Wulfmeyer et al. 2015). In particular, there is a huge need for accurate high temporal resolution temperature and humidity profiles in the planetary boundary layer (PBL), where synoptic boundaries, mesoscale processes and circulations, the diurnal cycle, and much more can modulate these profiles. These profiles are used for process-study research over a range of topics spanning mountain meteorology to turbulent mixing to moist cloud processes, and are critically needed to evaluate numerical weather forecasts. They also serve as input via data assimilation to these models, and for a large number of downstream stakeholders such as the dispersion modelers and the renewable energy community. This massive need served as the motivating force behind the development of this algorithm.

The Tropospheric Remotely Observed Profiling via Optimal Estimation (TROPoe) software product is designed to retrieve thermodynamic profiles and cloud properties from ground-based remote sensors. It is a robust retrieval system that is able to use a wide number of remote sensing and in-situ observations as input to provide high temporal resolution temperature and humidity profiles. The primary input data streams are made by an infrared spectrometer (IRS) or a multi-channel microwave radiometer (MWR), but the algorithm can include water vapor and temperature observations from lidars, radio acoustic sounding systems, radiosondes, surface met stations, numerical weather prediction model output, and more. The observations are constrained by an a priori climatology to provide realistic solutions. A complete characterization of the uncertainties of the retrieved profiles, as well as the information content in those profiles that comes from the observations, are included in the output files created by the algorithm.

The research and development activities that led to TROPoe were inspired by the Department of Energy Atmospheric Radiation Measurement (ARM) program (Turner and Ellingson, 2016). In particular, ARM-funded scientists developed and matured the Atmospheric Emitted Radiance Interferometer (AERI; Knuteson et al 2004a, 2004b). The primary goals of the AERI were multiple (Turner et al. 2016a), but one of them was to provide high temporal resolution water vapor and temperature profiles in the atmospheric boundary layer. In the mid-1990s, W.L. Smith and his colleagues at the University of Wisconsin – Madison developed the AERIprof algorithm to retrieve these profiles (Smith et al. 1999), and used it to perform a range of research studies (e.g., Feltz et al. 1998; Feltz et al. 2003; Wagner et al. 2008). There were multiple shortcomings with AERIprof: (1) it needed a very good first guess in order to converge to a solution; (2) it was primarily limited to clear-sky scenes, as it is difficult to get good estimates of cloud properties to use in the first guess; (3) it did not provide an uncertainty estimates for its retrieved profiles; (4) it was a physical-iterative retrieval that used a statistical fast radiative transfer model that was trained using ARM Southern Great Plains (SGP; Sisterson et al. 2016) observations and could not be run at other locations; and (5) it used a fixed value for the carbon dioxide concentration in this statistical radiative transfer model, and thus was not easily used in the changing climate.

D.D. Turner has been actively developing TROPoe since 2012. At that time, the original motivation was to address the shortcomings in AERIprof by using a Bayesian physical-iterative approach to get accurate retrieved profiles with their uncertainties; indeed, the original algorithm

was called AERIoe. However, there is a large community using ground-based MWRs for thermodynamic profiling, which led to the desire to use the same approach for both IRS and MWR (Löhnert et al. 2009; Blumberg et al. 2015). Thus, TROPoe evolved to use either dataset, or both simultaneously, as input.

Retrieving thermodynamic profiles from passive remote sensors like IRS and MWR is non-trivial. It is an ill-posed mathematical problem, and could be addressed either with a purely statistical retrieval or a physical-iterative method (Maahn et al. 2020). TROPoe uses the latter approach, which requires that a forward model (***F***) be used to convert the "state space" (i.e., profiles of temperature and humidity, cloud information, etc.; call this ***X***) into the "observation space" (i.e., the radiance observations made by the IRS and MWR; call this ***Y***). So ***F***(***X***) = ***Y***. We use radiative transfer models, which require additional inputs, as part of this process. We have encapsulated the retrieval software, radiative transfer models, the ancillary information needed by the radiative transfer models, and more into a software container that eases the distribution and use of this retrieval system.

TROPoe was designed to be both an operational algorithm and a research tool. As such, it is both easy to use and yet extremely flexible. To handle the flexibility and thus configure the retrieval, we use a "Variable Input Parameter" or VIP file. The VIP file is configured as a key-value system, and there are over 200 keys that are predefined that can be overridden by the user using the VIP file. The output file contains a wealth of information for each retrieved profile, including uncertainty estimates, measures of the correlated error, the information content, the observations and forward calculation for the converged retrieval, and a range of fields that can be used for quality control. It incorporates years of experience developing and using retrieval algorithms, and decades of improvements in the radiative transfer models that are the underpinnings of the retrieval. All of these components are discussed in subsequent chapters. We have attempted to make this Users Guide as complete as possible to help you configure the retrieval to use the observations you desire, to help you interpret the output, to provide the guidance needed to understand the role of the a priori dataset, and more. Additional information can be found in the references that describe TROPoe (formerly AERIoe); these are Turner and Löhnert (2014); Turner and Blumberg (2019), and Turner and Löhnert (2021). Output from this algorithm has been used in a wide range of studies, resulting in over 50 published papers; several of the more noteworthy of these are listed in Appendix 2.

2) Algorithm Overview

Physical retrievals exploit the idea, introduced earlier, that observations, forward models, and the assumptions necessary to drive them have inherent uncertainty and can be represented by probability distributions.

Suppose the current atmospheric state is given by the vector ***X***; in this application, this is the thermodynamic profile in the troposphere, cloud properties, and trace gases arranged like this:

$\boldsymbol{X}$ = [T(z_1), T(z_2), …, T(z_n), q(z_1), q(z_2), …, q(z_n), LWP, lreff, iTau, ireff, x_{co2}, x_{ch4}, x_{n2o}]^T

where T(z) and q(z) are the ambient temperature and water vapor mixing ratio profiles for heights z_1 to z_n, respectively, with z_1=0 m above the instrument level (which is usually assumed to be above ground level), and z_n being above the top of the tropopause (usually between 17 to 20 km above ground level, AGL). The four cloud properties that are in the state vector are the liquid water path (LWP), ice cloud optical depth (iTau), and the effective radius of the liquid (lreff) and ice (ireff) components. It is important to realize that TROPoe only assumes a single cloud layer, but that cloud layer could be either single phase (liquid or ice) or mixed-phase (both liquid and ice exist in it). The algorithm can also retrieve (in an experimental mode) trace gas information for carbon dioxide (CO_2), methane (CH_4), and nitrous oxide (N_2O); these are the x_{co2}, x_{ch4}, and x_{n2o} parameters. Each of these "x_{tg}" parameters is actually three variables: the free tropospheric concentration, the difference between the free troposphere concentration and the planetary boundary layer (PBL) concentration, and a profile shape parameter (no longer used).

We have some observation system that will make an observation that provides some information on the current atmospheric state. Almost always, we use the atmospheric state in a forward model ***F*** to mimic the behavior of the instrument, and thereby compute a pseudo-observation $\widehat{\boldsymbol{Y}}$. This can be expressed as:

$$\boldsymbol{F}(\boldsymbol{X}) = \widehat{\boldsymbol{Y}} \qquad \text{Eq 1}$$

Suppose we have an observation ***Y***, and desire the atmospheric state ***X*** that existed when the observation was taken. Note that ***Y*** does not equal $\widehat{\boldsymbol{Y}}$ because there are both random and (hopefully small to negligible) systematic errors in a measurement system. Ultimately, we would like to invert ***F*** so that we derive ***X***. However, this is frequently an ill-posed problem mathematically.

To solve these problems, we can use Bayes' theorem:

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)} \qquad \text{Eq 2}$$

which states how probable a certain ***X*** is given ***Y*** [i.e., $P(X|Y)$]. This probability density function is already the solution for the inverse problem, which means that the solution includes an uncertainty estimate. Bayes' theorem tells us that it can be obtained by multiplying the probability of ***Y*** given ***X*** [$P(Y|X)$; the observation] with the a priori probability density of the state $P(X)$ (e.g., based on a climatology, a model forecast, or a measurement at some distance away in space or time). The probability $P(Y|X)$ represents the uncertainty in the measurement process

and also in the model that converts the state ***X*** into observations ***Y***, while the division by $P(Y)$ is effectively a normalization factor.

Optimal estimation (OE) is a widely used physical retrieval method and the equations, given by the excellent textbook by Rodgers (2000), can be derived from Eq. 2 if one assumes all of the probabilities have a Gaussian distribution. OE also implicitly assumes that there is no bias in the measurements and prior dataset; in other words, the average of their uncertainties is zero. A natural advantage of OE is that the uncertainties in the retrieved solution (i.e., the uncertainties in ***X***) are automatically provided as a full error covariance matrix, which includes the uncertainties of the observations ***Y***, the uncertainties in the a priori information used to constrain the retrieval, and the sensitivity of the forward model ***F***. A nice tutorial of the OE method is given by Maahn et al. (2020).

A detailed description of TROPoe is given by Turner and Löhnert (2014), which includes equations that specify how the next iteration is derived in this physical-iterative technique, uncertainty estimates, and information content. Turner and Blumberg (2019) indicate how the observation vector ***Y*** can be expanded to include observations from multiple instruments. Turner and Löhnert (2021) continue this exploration of using multiple different observation types in ***Y***, and quantify the information content when different observations are added to ***Y***. All of the results in these papers were generated directly from output in the TROPoe data files; thus, these papers demonstrate how the information content and uncertainties change for different retrieval configurations.

Passive retrievals are, by their nature, ill-posed mathematical problems. In other words, there often exists more than one state space solution that maps to a given observation. Stated mathematically, this implies $F(X_1) = Y$ and $F(X_2) = Y$, where $X_1 \neq X_2$. Furthermore, every observation has some amount of random uncertainty associated with it; i.e., $Y_{true} = Y + \epsilon$, where Y_{true} would be the observation from a perfect instrument. Thus, we need to constrain the solution (i.e., the retrieved ***X***) so that the solution is "reasonable". This is done using a priori information, which is given by $P(X)$ in Eq 2. Additional information on the a priori will be discussed in a later chapter.

3) Using TROPoe in a Container

There are many components needed to run TROPoe. These include the retrieval code itself (found in the GitHub repository https://github.com/OAR-atmospheric-observaIons/TROPoe), the radiative transfer models, the input for the radiative transfer models (e.g., the spectroscopic databases and cloud property scattering databases), standard atmospheres, and more. All of these components must be installed in a particular configuration to allow the retrieval to work. This was done on individual computer systems in the past, but would often take a full day to compile the radiative transfer models and get the configuration set up properly. This was unsustainable. We have moved to using Docker containers to create a packaged software system that can easily be downloaded off the internet.

a) What is a container?

A software container is a relatively lightweight executable package of code that contains everything it needs to run. Thus, software in a container can be run in a standalone manner without any significant dependencies that may be difficult to configure. In this example, we use a Docker container, as it is an open-source platform that makes it easy to build, deploy, run, and update. We, the TROPoe developers, build updated container images when we make improvements to the retrieval code, the radiative transfer models have been updated, or similar improvements. We then push the new container image to DockerHub, which is a web-based platform very similar to GitHub. Users, such as yourselves, pull the new container image to your computer and then run it. This makes updating the codebase extremely easy and robust.

Containers can be run on Linux, Windows, and MacOS environments. One advantage of using containers is that the environment and software is the same for all users and thus two people running the same data with the same version of the container on different computing systems will get the same answer.

i) Running the code on linux-based (and MacOS) machines

While the container has the required software environment, the code needs access to the data it will process as well as the a priori data needed to constrain the retrieval (more on the a priori dataset in Sec 6). This requires that we "bind" a directory that has our observational data to the container; this process is similar to a network file system (NFS) mount. Additionally, we need to pass in other information to the container such as the date to process, some control parameters, etc. We can do this via the command line on a linux / MacOS operating system.

For a unix-based operating system like a linux server or a MacOS computer, we can use either "docker" or "podman" to get and execute the TROPoe software container image. [Your system administrator may have preferences about which they would like to install / support, but the execution is virtually identical for each. For this discussion, we will assume we are using docker.] To get the latest version of the TROPoe software from DockerHub, issue this command:

```
docker pull davidturner53/tropoe:latest
```

If you desire a particular version of TROPoe, say version 0.20 (differences among the versions can be found in Appendix 3), then you download it like this:

```
docker pull davidturner53/tropoe:v0.20
```

This will pull the software down to your computer. You will see it updating in layers, as that is how it was built.

Another important command is to see what software images you have on your computer. This can be done using:

```
docker image ls
```

In the statements above, replace "docker" with "podman", if your system only supports the latter. Note that there are a multitude of other docker (podman) commands available; a quick online search may be informative for you.

Now you are ready to run the TROPoe software package. We have created a bash shell script to make a simple command line interface, and the script takes care of binding the external directories and passing in the arguments as environmental variables. This script, run_tropoe_ops.sh (available via the TROPoe GitHub location), is executed like this:

```
run_tropoe_ops.sh date VIPfile priorFile shour ehour verbose dataDir tmpDir imageName
```

Where:

- date is an 8-digit date of the format yyyymmdd
- VIPfile is the path/name of the VIP file (details on this later).
- priorFile is the name of the a priori file (details on this later).
- shour is the starting hour for the retrievals (usually should be 0; units are UTC)
- ehour is the ending hour for the retrievals (maximum is 24; again, units are UTC). Note that if ehour <= shour then the code will perform a single retrieval at shour
- verbose dictates how "noisy" the output to the screen is. The values range from 1 (pretty quiet) to 3 (very noisy)
- dataDir is the "head" of the data tree where your input observational data; the VIPfile must be either here or in a subdirectory.
    - Note that the container will map this directory (suppose dataDir is `/home/user/my_input_data`) to the directory `/data` inside the container. In other words, the file` /home/user/my_input_data/thisfile` will look like `/data/thisfile` from inside the container.
    - Note that because of the way the directory binding works, the container can only see directories and files that are in dataDir. Thus, all of your input data must be in that directory
    - An example of how we typically arrange our directories is given in Section 3b
- tmpDir is only used for two purposes: (1) if you have a very fast hard drive that can be used for the input/output files that are created regularly as part of the retrieval process

(the radiative transfer models used by TROPoe use temporary files heavily), or (2) you would like to see these temporary files perhaps for debugging purposes. Note that, by default, TROPoe won't use this tmpDir; instead, it uses a temporary directory inside the container that it can more easily clean up. Thus, we recommend that you set tmpDir to be the same as dataDir, or perhaps dataDir/tmp.

- imageName is the name of the container image you want to run. When a new TROPoe container is built, we tag it three ways: (1) with the date it was built (as an 8-digit date), (2) with the version number like v0.20, and (3) as "latest". Thus, general users can always get the latest code by pulling `davidturner53/tropoe:latest` (and this would be the imageName you use on that command line). However, if you desire to run an older version of the code for some reason and you have pulled it down to your computer using the date it was built (e.g., `davidturner53/tropoe:v0.18`), then you can execute that version by using that as the imageName instead.

ii) Running the code on Windows machines

This still needs to be developed and tested. We will use an ASCII file to pass in the input arguments. This will be further developed and documented in a future version of the code and this User Guide.

However, we have successfully run TROPoe using the on computers running Windows Subsystem for Linux (WSL). In this case, the same commands as described above in 3a-i are used.

iii) Running the code on HPC machines using Singularity (Apptainer)

Many high-performance computing (HPC) systems have multiple processing nodes, and it is impractical to run a Docker container on these as that container would need to be installed on each processing node. However, the application Singularity helps this issue, as it embeds the Docker container within a Singularity container where the latter is stored on a local disk (vs in some hidden directory within the compute system). To build a singularity container named "TROPoe.singularity.sif" in the home directory, execute this on the command line:

```
singularity build --fix-perms ~/TROPoe.singularity.sif docker://davidturner53/tropoe:latest
```

After this new Singularity container is built, then you can execute the code using this command:

```
singularity run -B dataDir:/data --env app=TROPoe,yyyymmdd=20250813,
vfile=/data/tropoe/vip.txt,pfile=prior.MIDLAT.nc,shour=00,ehour=24,verbose=1
~/TROPoe.singularity.sif
```

which assumes that `dataDir` is the location that you want to bind (mount) to /data, and that you want to use the mid-latitude prior. Note that the input parameters are passed into the singularity container via environment variables using the "--env" option; these environment variables are app, yyyymmdd, vfile, pfile, shour, ehour, and verbose, with each of them assigned a value. There should be no spaces in this string of environment variables.

b) Recommended directory structure

As indicated above, the data to be processed exists outside the container, and the execution script will essentially mount the external directory to a directory that is accessible by the code in the container. We like to put the different datasets in different directories, such as this:

```
…./dataDir/
                    irs_ch1/
                    irs_ch2/
                    irs_sum/
                    irs_eng/
                    mwr/
                    met/
                    ceilometer/
                    tropoe/
                    tmp/
```

where the VIP file is usually placed in the 'tropoe' subdirectory with the name 'vip.txt'. With this structure, this is the command to execute the script for 13 August 2025 for the entire day using the standard mid-latitude prior (see next section) and v0.20 of TROPoe:

```
cd …/dataDir
run_tropoe_ops.sh 20250813 /data/tropoe/vip.txt prior.MIDLAT.nc 0 24 1 ${PWD} ${PWD}/tmp davidturner53/tropoe:v0.20
```

Information on the prior dataset will be given in Section 6 and the VIP file in Section 4.

4) Configuring the retrieval via the VIP

The philosophy behind TROPoe is that most users will configure the retrieval for a given site once, and then process a large amount of data with it. Thus, we wanted to make this processing mode as easy as possible, which is why both the input date and the prior file are passed in with the command line. The other parameters which are used to configure the retrieval are set in the VIP file, which ultimately is the configuration file for the retrieval.

a) Command line arguments

The command line arguments are the date, VIP file, prior file, the start / end hours (which are usually 0 and 24, respectively, for most users), verbose (which is usually set to 1 for minimal noise; the other values are used to help debug things), and the name of the container image to run. All times used here are in UTC. Details are in section 3a-i.

Note that if the end hour is smaller than or equal to the start hour, the retrieval will perform a single retrieval (i.e., for one observation time) at the start hour. This can be very handy when the user would like to perform a retrieval to compare against another observation (e.g., a collocated radiosonde profile).

b) VIP file – high level overview

The Variable Input Parameter (VIP) file sets the configuration of the retrieval. We use a series of key-value pairs within the code, each of which is set to a default value. The VIP file allows the user to overwrite the default values of these keys, and thus change the default behavior of the retrieval. There are a certain number of entries that must be in every VIP file; for example, the path to the data used in the retrieval is absolutely required otherwise the code will not be able to find the input data! We have grouped these key-value pairs into two groups: the standard group and the experimental group. For most users, only a subset of the standard group is needed. For advanced users, keywords in the experimental group might be changed (although we highly recommend you communicate with the TROPoe developers before you use these experimental keywords).

The VIP keywords are organized in “blocks”. For example, there are 6 VIP entries associated with the “station” (general information about the location of the site), over 30 entries for infrared spectrometer (IRS) data, and 13 VIP entries for a zenith-pointed microwave radiometer (MWR). Many MWRs also collect data at off-zenith angles (i.e., elevation scans), and these data can be included in the retrieval using another VIP keyword block. Other standard blocks include the surface met data, cloud base height, and prior information, amongst others. A brief example VIP file is given in the next section.

To see all of the standard VIP entries (i.e., all of the blocks), run the code with the date set to zero. Each key-value pair will be written to standard output, with a small amount of descriptive text to understand the values that each key can take. Most of the VIP keys have default values that provide baseline capability. If the date is set to 1, then all of the VIP key-value pairs will be written out (i.e., both the standard and experimental). Appendix 1 lists the standard and

experimental entries in the VIP file, their default values, and what other values can be assigned for each.

c) Example VIP file

To run the retrieval, relatively few of the key-value pairs need to be set in the VIP file. In the code, there are hundreds of key-value pairs used to set the configuration of TROPoe (see appendix 1); the VIP file allows the user to overwrite a subset of these to modify the retrieval as they desire. Thus, the VIP file can actually be quite short with only a few lines. For the sake of this discussion, we will assume that the user desires to process observations from an infrared spectrometer through the code. Here are the key-values pairs that need to be set in the VIP file, and what they are doing.

```
tres = 10    # Temporal resolution [min], 0 implies maximum (native) temporal resolution

station_lat = 36.6              # Station latitude [degN]
station_lon = -97.5             # Station longitude [degE]
station_alt = 316.0             # Station altitude [m MSL]
station_pres = 980.0            # Station pressure [mb]; will be only be used if there is no other Psfc input

irs_type = 1                    # Specifies the type of IRS data to read, in this case it is the ARM AERI data
irsch1_path = /data/aerich1         # Path to the IRS ch1 radiance files
irssum_path = /data/aerisum         # Path to the IRS summary files
irseng_path = /data/aerieng         # Path to the IRS engineering files

cbh_type = 1                    # Specifies the type of CBH data to read, in this case it is ARM ceilometer
cbh_path = /data/ceil           # Path to the CBH data
cbh_default_ht = 2.0            # Default CBH height [km AGL], if no CBH data found

output_rootname = tropoeOutput      # String with the rootname of the output file
output_path = /data/tropoe          # Path where the output file will be placed

globatt_Site = Some metadata about the site provided by the user (you!)
globatt_Dataset_Contact = Some metadata about who created these data (like your email)
```

This configuration will result in retrievals from the ARM AERI using ARM ceilometer data to specify the cloud base height, and the retrievals will be performed at 10-minute resolution. The retrieval always needs to be able to specify a cloud (should the input radiance observations suggest a cloud is needed to get radiative closure), and thus a default cloud base height needs to be specified in case the ceilometer doesn't observe a cloud. [More on this later in Section 5d]. Note anything after the `#` is considered to be a comment.

Keywords that are always required by the retrieval are these:

- `tres` – defines the temporal resolution of the retrieval. Use 0 if you desire the maximum temporal resolution of the master dataset. This is an integer value.
- The three `station_{lat,lon,alt}` keywords listed above, so that the output file can properly contain this information

- `station_pres` - defines the default surface pressure for the station, should the atmospheric pressure not be found in the other datastreams. Note that the code will first try to get the pressure from the co-located surface met station (if that keyword block is enabled – more on that in Section 5c) and the values are valid. If the surface met data is missing then this default pressure be used.
- The three `cbh_` keywords. The retrieval must know where to place the cloud vertically, should the radiance and other observations suggest there is a cloud. More on this keyword block in section 5d
- The two `output_` keywords. The code needs to know how to name the output and were to write it. In this example, the output file will have the name `tropoeOutput.yyyymmdd.hhmmss.nc`, where hhmmss is the time of the first sample in the file.
- There can be any number of `globatt_XX` key-value pairs as long as the keys are unique. These allow additional metadata to added to the global attributes of the output file. We strongly recommend you use these to provide additional metadata for this dataset you are creating. Note that the entire list of VIP entries will be added to the global attributes of the output file, so that any file created by TROPoe contains the configuration used. The global attributes will also include the version number and package identification number of TROPoe (again, to ensure reproducibility).

TROPoe is designed to have a passive remote sensor (either an IRS or MWR) serve as the "master instrument", and other observations (if any) are interpolated to the master's sampling time. If the temporal resolution (`tres`) is set to zero, then retrievals will be performed for each sample in the master dataset. If the retrieval is set to 10-min resolution, then the code will look for samples in the 00:00-00:10 period, 00:10-00:20, … 23:50-24:00 and process them. When `tres` > 0, the time associated with the retrieval is the center of the averaging period.

As indicated above, the key-value pairs in the VIP file are organized into blocks. In the above example, there are 6 blocks. We will go into additional details about the various standard keywords that are in each block in the instrument sections below (section 5). A full list of the standard and experimental keywords, organized by block, are described in Appendix 1. Additional example VIP files for a few selected use cases are presented in later chapters (section 9).

5) Overview of typical input observations

As one of the motivations for TROPoe was to provide an improved retrieval for the ARM AERIs (Knuteson et al. 2004a,b), and due to the fact that D.D. Turner has been part of the DOE ARM program since the mid-1990s, the format of the output file is guided by the ARM data philosophy. Similarly, for each observation type, there will be readers (options) to allow the ARM data to be read in easily. The primary feature of the ARM data format are the time fields: "base_time" is the number of seconds since 1 Jan 1970 at 00:00:00 UTC (as a singleton), and "time_offset" is an array of times that are added to base_time to specify the actual times for each sample in the output file. To make it easier for the user, we only include a maximum of a single day worth of data in a netCDF file, and the 8-digit date is given in the filename. We also include the field "hour" in the output file, which is the hour from 00 UTC on the given date.

The original name of this retrieval algorithm was AERIoe, which had the AERI serving as the master instrument and all other observations were interpolated to the sample times of the AERI. However, we can also run the retrieval in MWR mode (i.e., without an IRS instrument), but this requires an additional key-value pair in the VIP file to indicate that the MWR is the master instrument as it is possible (and often done) to include both the IRS and MWR in a single retrieval. If you desire to use the MWR as the master instrument (i.e., there won't be an IRS dataset included), then this key-value pair must be added to the VIP: `irs_type` = -1. Section 9a provides several examples illustrating this option.

a) Infrared spectrometer

i) Basic instrument / measurements

At the moment, TROPoe is configured to use downwelling spectral infrared radiance data from two instruments: the AERI (manufactured by ABB, described by Knuteson et al. 2004a,b) and the ASSIST (manufactured by Telops, described by Michaud-Belleau et al. 2025). (The mention of any particular vendor does not imply endorsement in any way). The ASSIST and AERI are essentially identical instruments from both a philosophical and dataset perspective. Both instruments measure downwelling spectral radiance using an interferometer with a 1 cm maximum optical path delay, regularly view two high-emissivity blackbody targets operating at a warm (usually 60 degC) and ambient temperatures, account for instrument issues like detector nonlinearity, finite field-of-view self-apodization, and spectral calibration using a metrology laser. Due to the use of complex arithmetic in the calibration process (Revercomb et al. 1988), the uncertainty for each observed infrared spectrum is provided and is an important input (along with the radiance spectrum itself) to TROPoe. The key point here is that we can model the AERI and ASSIST using an identical approach in the forward model within TROPoe.

As part of the on-board processing by the instrument, the AERI and ASSIST each produce three primary files per day. These are:

- The "channel 1/A" radiance file, which is the spectral radiance from the MCT detector and typically spans from 530 to 1800 cm^{-1}. For some "extended range" systems, the lower wavenumber observed may start at 400 cm-1. Some important calibration data is stored in this file.

- The “channel 2/B” radiance file, which is the spectral radiance from the InSb detector and typically spans from 1800 to 3000 cm^{-1}. Note that the channel 2 data are not currently being used by TROPoe.
- The “summary” file, which contains spectral uncertainty for each radiance observation and a lot of the instrument housekeeping data.

The ARM program decided in the mid-1990s to split the summary file into two data streams, which they call “summary” and “engineering”. Ultimately, TROPoe needs input from both of those data streams. However, the raw AERI data has a of the needed housekeeping in the one summary file. Some groups, such as the CLAMPS team (Wagner et al. 2019), process the raw AERI summary file into a single netCDF file, and thus we will list the same path in the VIP file for both. Details on this below.

ii) The IRS block in the VIP file

The IRS data may come in slightly different file formats depending on the instrument used (e.g., AERI or ASSIST), and if ARM ingested the data to netCDF or one of the various ingest scripts used by the community did. Furthermore, the ASSIST data format is slightly different still. Thus, the first keyword needed in this block is `irs_type`, which will specify which type of data that is being read. This keyword currently can assume these values:

- 1 – the data are from the AERI in the ARM data format, and we will need to read in the irsch1, irssummary, and irsengineer files
- 2 – the raw AERI data were converted to netCDF using the University of Wisconsin - Madison / Space Science Engineering Center (SSEC) “dmv2cdf” application, which is part of their “AERI Armory”. This expects the channel 1 files to be named yyyymmddC1_rnc.cdf and the summary files to be named yyyymmdd_sum.cdf. The engineering path must point to the directory with the summary file
- 3 – the raw AERI data were also converted by dmv2cdf, but the files are named differently: channel 1 files are yyyymmddC1.RNC.cdf and summary files are named yyyymmdd.SUM.cdf. Again, the engineering path must point to the summary directory
- 4 – the raw AERI data were converted by dmv2cdf, but the channel 1 and summary files were renamed to conform to the ARM naming convention. Again, the engineering path must point to the summary files
- 5 – the native ASSIST netCDF files produced by the instrument. The engineering path must point to the summary files.
- 6 – the REFIR-PAD interferometer, as deployed to Cerro Toco, Chile during RHUBC-II (Turner and Mlawer 2010)
- 7 – the REFIR-PAD interferometer, as deployed Dome C in Antarctica (Di Natale et al. 2017)
- -1 – this is a special entry that is used to specify that (a) there will be no IRS data input and (b) that the MWR is consider the “master” instrument. It is likely this particular option will be removed and a new more intuitive keyword added in the future.

Assuming the `irs_type` > 0, then the code will read in IRS data. (We recently updated the codebase to read in and process the REFIR-PAD interferometer for experimental purposes; in the future it is possible that additional irs_type values will be added). The next few keywords are required to find the needed data:

- `irsch1_path` – the path to the channel 1 data (e.g., `/data/sgpaerich1C1.b1`)
- `irssum_path` – the path to the summary data
- `irseng_path` – the path to the engineering data, but this should be identical to the irssum_path for `irs_type = {2, 3, 4, 5, 6, 7}`.

The AERI and ASSIST both use a gold-plated scene mirror to direct radiance from different directions into the interferometer. There are versions of both instruments that look in multiple elevations. For example, during the RHUBC-2 field campaign (Turner and Mlawer 2010), the AERI was configured to collect data from both zenith and 60 degrees off zenith. The ASSIST is designed to provide not only zenith radiance observations, but also provides radiance observations of both black bodies in its channel 1 output file. Both instruments include a field in the channel 1 file that is named `sceneMirrorAngle` that captures the elevation angle of this mirror when the sample was collected. TROPoe needs to know what angle is associated with the zenith angle. Typically, this is 0 deg for the ASSIST and 180 degrees for the AERI. Thus, we need to specify the value of this keyword in the VIP file to identify the proper zenith angle for TROPoe:

- `irs_zenith_scene_mirror_angle` –the unit of this is degrees.

However, there are times when the AERI's sceneMirrorAngle is set to -9999 (this is a bug in the ARM configuration of some AERI systems); fortunately, in this case, all of the radiance data are zenith views so setting the value of this keyword to -9999 will allow the code to process the data correctly. Note that the code looks for data where the sceneMirrorAngle is +/- 3 degrees of the specified irs_zenith_scene_mirror_angle, and considers them zenith observations.

There is a very simple quality control check applied that identify obviously bad spectra before the processing occurs. This test looks at the mean brightness temperature in the 675-680 cm^{-1} window, which provides an estimate of the near surface air temperature. There are two keywords that specify the minimum and maximum temperature; observations outside these bounds are assumed to be bad and are skipped. If the environment is significantly warmer or colder than the default values for these two keywords, then add the keywords to the VIP file and redefine their limits to something that is more appropriate for the environment.

- irs_min_675_bt = 233 K
- irs_max_675_bt = 313 K

Another keyword that may be useful is `irs_min_noise_flag`. As indicated above, both the AERI and ASSIST provide estimates of the random uncertainty in the observation, and this is used for the retrieval. However, if the user expects that the code is trying to overfit the spectral radiance (i.e., that the noise estimate may be too small), then this keyword can be used to increase the observed noise in order to approximate the forward model error also. The use of this minimum noise spectrum was explored in Adler et al. (2024).

There are many other experimental keywords in the IRS block. These are primarily associated with performing sensitivity tests associated with various calibration factors used in the IRS calibration. These are discussed in the Appendix 1.

iii) Noise filtering

When the AERI was first developed in the 1990s (Turner et al. 2016a), the standard sampling strategy was to view the sky continuously for 3 minutes, and then each calibration blackbody for 2 minutes, resulting in a single spectrum every 7 minutes. However, while this had an excellent signal-to-noise ratio in the observed infrared spectrum, this approach was found to convolve clear and cloudy radiances into the average spectrum, which makes it much harder for retrieval algorithms to separate out the signals. In the early 2000's, the sampling strategy changed to "rapid scan" mode, wherein the sky averaging period was reduced to ~15 seconds; typically, 6 to 8 sky samples are collected before the blackbodies are viewed. This results in a markedly larger random noise level in the AERI data. The ASSIST has chosen to use a similar rapid scan sampling strategy. To reduce the random error, a principal component noise filter for IRS spectra was developed (Turner et al. 2006). We highly recommend that this noise filter be applied, especially if the data you are analyzing were collected in broken sky conditions. (The noise filter can be found in the GitHub repository https://github.com/OAR-atmospheric-observations/IRS-Noise-Filter.) This noise filtering is applied to the IRS radiance data before TROPoe is executed. Eventually, this capability will be made available via another container; in the meantime, contact one of the authors of this document for implementation details if you desire to use it.

The sampling strategy of the IRS instrument, whether or not it has had the noise filter applied, and desired the temporal resolution of the retrieval all should be considered when setting an important keyword "avg_instant", which can take on 3 values:

- 1 – take the instantaneous sample closest to the middle of the averaging period
- 0 – average a of the spectral observations found in the tres averaging period, and decrease the average noise value by the square root of the number of samples in the retrieval
- -1 – average a of the spectral observations found in the tres averaging period, but use the average noise in the retrieval (i.e., do not decrease it by sqrt(N)).

A few thoughts here. If `tres = 0`, then all three of these options for `avg_instant` yield the same results. However, if `tres > 0` then we recommend you either use `avg_instant = 1` or `avg_instant = -1` in your processing, as we have found that using `avg_instant = 0` results in TROPoe overfitting the data and the quality of the resulting retrievals is poor. Using `avg_instant = 1` is recommended if the IRS data have been noise filtered, as using `avg_instant = -1` will convolve clear and cloudy scenes in the averaging process (if the sampling period has broken clouds overhead).

b) Microwave radiometer

i) Basic instrument / measurements

There are a multitude of ground-based microwave radiometers available to the operational and research community. For thermodynamic profiling, the most common models are the MP-3000 (manufactured by Radiometrics Inc.; Solheim et al. 1998) and the HATPRO (manufactured by Radiometer Physics GmbH; Rose et al. 2005). (Again, mention of any particular vendor does not imply endorsement in any way). Both of these radiometers make multiple measurements along the side of the 22.2 GHz water vapor absorption line (i.e., the K-band) and the side of the 60 GHz oxygen complex (i.e., the V-band). However, radiometers have also been built to take advantage of the much stronger 183.3 GHz water vapor band, especially for profiling in the Arctic (e.g., Cimini et al. 2009). Additionally, some MWRs have only a few channels (e.g., ARM's two and three channel systems; Cadeddu et al. 2013), and including observations from any of these MWRs provides additional constraints to the retrieved profiles and cloud properties. Thus, TROPoe was designed to use any set of microwave frequencies.

In all of these cases, the basic measurement made by the MWR is the downwelling sky brightness temperature at several frequencies. Often, the MWR is collecting data at multiple elevation angles, as this increases the optical depth along the path and thus can provide additional information to the retrieval, especially in the boundary layer (e.g., Crewell and Löhnert 2007). TROPoe uses two different VIP blocks to handle the zenith and off-zenith (i.e., elevation scan) observations.

The sky brightness temperature observations (henceforth called Tbsky) can be arranged in the input file a couple different ways. One way is to store the data as a set of 1-dimensional time-series, with each frequency having a different field name; e.g., the ARM 2-channel microwave radiometer data files have the fields tbsky23 and tbsky31. A second way is to have a single 2-dimensional field, perhaps called tbsky, where the two dimensions are time and frequency. In this case, we need to provide in the VIP file the name of the field that contains the frequency information. In both of the previous cases, elevation scans may be interspersed within the zenith observations and thus a separate field is used to identify the elevation angle for each sample (the name of this field must also be specified in the VIP file). A third case, which is used heavily in Europe, is where the MWR data have been quality controlled using software developed at the University of Cologne; a trademark of this format is that they use a 3-dimensional matrix to encode the elevation scans as a function of time, frequency, and elevation.

The MWR community has typically used an elevation angle of 90 degrees as the zenith angle. We adopt that strategy here, and only data that point in this direction will be included using the configuration in this `mwr_type` block.

Unlike the IRS observations, most MWRs do not provide uncertainty estimates for the observed brightness temperatures. Thus, the user needs to indicate, via the VIP, the uncertainty level that should be assumed for each frequency. TROPoe will assume that these uncertainties are uncorrelated.

The bandpass typically used in MWRs is between 50 to 1,000 MHz, depending on vendor and frequency channel. TROPoe models each channel using a central wavelength; i.e., it does not perform a high-spectral resolution run which is then convolved with the filter bandpass. As some radiometers use a double-side passband (i.e., measuring two specific bandpasses on either side of a set frequency point), this does possess challenges in properly simulating the radiometer's observation. TROPoe has adopted the perspective that each MWR frequency channel can be represented with a single "center wavelength", wherein that frequency is specified in the VIP file for each brightness temperature frequency.

Lastly, many MWRs have exhibited what appears to be radiometric biases in their brightness temperature observations. TROPoe allows the user to specify, again via the VIP, a brightness temperature offset that should be added to the observation at each frequency before the retrieval begins. This is discussed in a later chapter (section 10).

ii) The MWR block in the VIP file

- `mwr_type` – the type of zenith pointing data
  - 1 – Tbsky data arranged as a series of 1-dimensional time-series.
  - 2 – Tbsky data arranged as a single 2-dimensional field (frequency by time)
  - 3 – the data arranged following the Univ Cologne data format
- `mwr_path` – the path to the MWR data (again, starting from `/data`)
- `mwr_rootname` – the rootname of the MWR data file
- `mwr_elev_field` – the name of the elevation field. If all data are zenith, then omit this
- `mwr_freq_field` – the name of the frequency field, if `mwr_type = 2`
- `mwr_n_tb_fields` – the number of Tbsky fields to use in the retrieval
- `mwr_tb_field_names` – the name of the Tbsky field(s). If `mwr_type = 2`, then this is a single name. If `mwr_type = 1`, then a comma-delimited list with the name of the `mwr_n_tb_fields` in the netCDF file to read (note that the order of the names should match the order of the frequencies listed below)
- `mwr_tb_field1_tbmax` – a very simple QC test, wherein the observed brightness temperature of the first field listed above must be below this value. [D.D. Turner uses a simple rule-of-thumb wherein if the observed Tbsky at 31 GHz is above 100 K, then it is raining; thus, he usually lists this frequency first and sets this threshold to 100 K]
- `mwr_tb_freqs` – a comma-delimited list of central frequencies, in GHz, to use for each of the `mwr_n_tb_fields` channels
- `mwr_tb_noise` – the random noise level, in K, to assume for each of the `mwr_n_tb_fields` channels
- `mwr_tb_bias` – the bias error, in K, to assume for each of the `mwr_n_tb_fields` channels. This will be added to the observed `Tbsky` values. If you don't know what the biases are, you still need to specify this entry in the VIP file with `mwr_n_tb_fields` zeros

iii) MWRscan block

If the MWR is collecting data at elevation angles other than zenith, then the user can include these observations using this block. As indicated above, the MWR community has traditionally

used 90 degrees as the zenith elevation. Some MWRs only scan in a single half-plane (i.e., from near 0 degrees to 90 degrees), whereas others scan in the entire plane (i.e., from 0 degrees through 90 degrees to near 180 degrees). Typically, these full-plane scans are done with equal elevation angles on both sides of zenith; i.e., if there is a 10-degree observation there will also be one at 170 degrees. TROPoe assumes the atmosphere is plane-parallel when treating these elevation scans, and thus these equal elevation scans are modeled exactly the same way. Azimuthal scans are ignored, for the same plane-parallel assumption.

The MWRscan block has a very similar set of keywords as the mwr block above; just use "mwrscan_" in place for each "mwr_" in the above list. Note that the number of Tbsky fields used in the mwrscan_ block does not have to be the same as in the mwr_ block; for example, Crewell and Löhnert (2004) recommend only using the more-opaque channels in the V-band in elevation scans (and we concur with that recommendation). However, we do recommend that if you use some of the same channels in both the mwr_ and mwrscan_ blocks that they have the same center frequencies, noise levels, and Tb bias offsets.

There are two new keywords that are unique to the mwrscan_ block that are not in the mwr_ block. These are associated with the elevation scans:

- `mwrscan_n_elevations` – the number of elevation angles to use
- `mwrscan_elevations` – the elevation angles to use in the retrievals (comma-delimited)

As an illustration, suppose that a MWR is regularly scanning and the elevation scans are 10, 25, 40, 90, 140, 155, and 170 degrees (which are then repeated ad nauseum). The 90-degree scan should be handled using the mwr_ block. Suppose you would like to include only the 10, 25, 40, and 140 degree scans in your retrieval (e.g., let's pretend that the 155 and 170 degree data are obscured by trees or something). Then the mwrscan_n_elevations should be equal to 4, and the mwrscan_elevations = 10, 25, 40, 140.

iv) Bias determination / correction

Most MWRs use an internal noise diode to provide a second reference point (the primary reference being an internal ambient temperature blackbody target) for calibration. There are multiple ways to calibrate the noise diode; e.g., so-called TIP curves and external liquid nitrogen (LN2) cooled blackbodies. Both methods have strengths and weaknesses. However, many radiometers demonstrate drift in the radiometric calibration of one or more channels with time. This can result in spectral Tbsky observations that are non-physical; i.e., there does not exist an atmospheric state that will satisfy the spectral observations. This is one example where a bias correction scheme can be useful. The second is when the sky is clear but the retrieval is providing LWP >> 0 g/m^2; this is usually attributed to a bias in the more transparent K-band channels. The mwr_tb_bias and mwrscan_tb_bias keywords enable the user to apply a brightness temperature offset to one or more channels before the retrieval is performed. Details on how to determine the spectral bias for a MWR is shown in a later chapter (section 10).

c) Surface met

Many sites where either IRS or MWRs are located have surface meteorological stations that measure atmospheric pressure, temperature, and relative humidity with in-situ probes. Indeed, most IRSs and MWRs have sensors like this integrated into the instrument. These met observations provide additional information to the retrieval, and are especially useful for MWR-only retrievals (e.g., Guy et al. 2022). Here are the keywords needed to include surface met observations into the retrieval. [Note that the code that reads in the surface met was refactored in v0.18, and thus earlier versions need to use these keywords differently – see appendix 3.]

The first several key/value pairs are used to specify basic characteristics about the in-situ surface meteorology data stream:

- ext_sfc_path – Path to the external surface met data
- ext_sfc_rootname – Rootname of the external surface met data stream
- ext_sfc_time_format – Time format for external surface temperature: 0-base_time/{time_offset,time} [epoch seconds], 1-same as option 0 but base_time is in milliseconds (ASSIST), 2-time [epoch seconds]
- ext_sfc_time_delta – Maximum amount of time from endpoints of external surface met dataset to extrapolate [hours]
- ext_sfc_relative_height – Relative height of the met station to the IRS zenith port [m]; note if met station is below IRS port then the value should be negative. Importantly, even measurements from a single height on a tall collocated tower can be used as input, as long as this field is properly set!

The next several key/value pairs are used to provide information on the ambient temperature observation:

- ext_sfc_temp_type – External surface temperature met data type: 0-none, 1-external met data
- ext_sfc_temp_fieldname – Name of surface temperature met field
- ext_sfc_temp_units – Units of surface temperature met field: 1-degC, 2-degK
- ext_sfc_temp_random_error - Random error for the surface temperature measurement [degC]
- ext_sfc_temp_rep_error – Representativeness error for the surface temperature measurement [degC], which is added to the random error

The in-situ humidity observation is assumed to be relative humidity (RH). TROPoe will convert this surface RH observation to water vapor mixing ratio, and will utilize that as the constraint in the retrieval:

- ext_sfc_wv_type – External surface water vapor met data type: 0-none, 1-external met data
- ext_sfc_wv_fieldname – Name of surface water vapor met field
- ext_sfc_wv_units – Units of surface temperature met field: 1-percent_rh, 2-fraction_rh

- ext_sfc_rh_random_error – Random error for the surface relative humidity measurement [%], which is used with the ext_sfc_random_temp_error to get the uncertainty in WVMR
- ext_sfc_wv_mult_error – Multiplier for the error in the surface water vapor mixing ratio measurement. This is applied BEFORE the 'rep_error' value
- ext_sfc_wv_rep_error – Representativeness error for the surface water vapor measurement [g/kg], which is added to the assumed random uncertainty

The surface pressure is required for the retrieval. If a met station does not exist (or more accurately, the pressure from the met station), then the code will use the pressure field in the IRS or MWR dataset, and then the value of the keyword "station_pres" as the surface pressure. The retrieval uses this surface pressure together with the temperature profile (at each iteration) to derive the pressure profile using the hypsometric equation.

- ext_sfc_pres_type – 0-Use the internal {IRS,MWR} pressure sensor for psfc; 1-external surface met data
- ext_sfc_pres_fieldname – Name of surface water vapor met field
- ext_sfc_pres_units – Units of surface temperature pressure field: 1-kPa, 2-hPa, 3-Pa

In-situ meteorological measurements are very common, and typically measure atmospheric pressure, ambient temperature, and relative humidity. The default surface met parameters are selected to read in the ARM "met" datastream. The user has the ability, via the above VIP entries, to specify the random error in the in-situ temperature and relative humidity observations. The default values for these two VIP values are 0.5 degC and 3.0 %RH, respectively. The uncertainties are propagated in the standard way to provide the uncertainty in the surface-level water vapor mixing ratio value. Additional "representativeness errors" can be added, if desired; for example, if the met station is located several kilometers away from the IRS/MWR. Note that the ambient pressure is assumed to have zero uncertainty.

d) Ceilometer

The retrieval will always assume there is a cloud somewhere in the atmosphere (unless the user disables the cloud retrieval by setting both retrieve_lcloud and retrieve_icloud to zero in the VIP file – this is NOT recommended). Thus, the algorithm needs to have some information on how high to place this cloud, should it exist. (The cloud can be considered to not exist if the retrieved liquid water path and ice cloud optical depth are zero within their uncertainties). Note that currently the measured cloud base height from the ceilometer (or the default value, if that is used) is assumed to have no uncertainty.

i) Basic instrument / measurement

The ceilometer is a simple attenuated backscatter lidar that is designed to identify the lowest height that has a markedly larger backscatter return above the background value. Ceilometers are commercially available from many different vendors, and are typically deployed around airports. Many experimental research sites will include ceilometers as part of the instrument complement due to their rugged automated capability, simplicity of use, and the importance of having

observations of the cloud base height. The desired unit for cloud base height within TROPoe is km AGL, where the 0 km AGL is the height of the master instrument (either the IRS or MWR).

ii) Possible formats

Multiple different ceilometer readers have been implemented within TROPoe, selected using ceil_type:

1) ARM ceilometer data: base_time (single value, time since 1970-01-01 00:00:00 UTC), time_offset (array, seconds since base_time), and first_cbh (in m AGL). Files should be named *ceil*yyyymmdd*.
2) ASOS/AWOS ceilometer data (also known as the Blumberg ingest): base_time (single value, time since 1970-01-01 00:00:00 UTC), time_offset (array, seconds since base_time), and cloudBaseHeight (in km AGL). Files should be named either *ceil*yyyymmdd* or *cbh*yyyymmdd*
3) NOAA CLAMPS Doppler lidar fixed point (zenith) data: base_time (single value, time since 1970-01-01 00:00:00 UTC), time_offset (array, seconds since base_time), and cbh (in km AGL). Files should be named either *dlfp*yyyymmdd*
4) ARM Doppler lidar vertical motion moment datastream: base_time (single value, time since 1970-01-01 00:00:00 UTC), time_offset (array, seconds since base_time), and dl_cbh (in m AGL). Files should be named *dlprofwstats*yyyymmdd*.
5) JOYCE Vaisala ceilometer data: base_time (single value, time since 1970-01-01 00:00:00 UTC), time_offset (array, seconds since base_time), and first_cbh (in m AGL). Files should be named *ct25k*yyyymmdd*.
6) JOYCE Jenoptic ceilometer data: time (array, unix time + 2.0828448e+09 in seconds), cbh (in m AGL). Files should be named *CHM*yyyymmdd*
7) E-PROFILE ceilometer data: time (array, unix time in days), cloud_base_height (a 2-d matrix, will use the first level, units of m AGL, and vertical_visibility (array, units m AGL). Files should be named L2*yyyymmdd*. If the CBH <= 0 and visibility > 0, then the latter will replace the former.

If you have ceilometer cloud base height estimates in a different format, we recommend reformatting your data to meet one of the above types. We anticipate that the ceilometer reader will be refactored in a future version of the code to simplify it.

iii) Importance of cloud base height in TROPoe

Getting a cloud, if one exists, at the correct height is important for TROPoe, and especially if IRS data are being used in the retrieval. TROPoe is attempting to separate out the emission of the cloud from the emission from the atmosphere below the cloud, and the algorithm will struggle to do this if the cloud is at the wrong height (and thus its temperature is incorrect). Errors in cloud base height are more important when the cloud is near the surface because both the weighting functions from both the IRS and MWR observations tend to peak in the lower part of the atmosphere. This was demonstrated in Guy et al. (2022), who was retrieving thermodynamic profiles and cloud properties in near-surface fog layers. As the cloud height increases, the sensitivity to the errors in the cloud base height decreases.

iv) Diagnosed cloud base height

Ground-based IRS observations have sensitivity to cloud base height (Turner 2003). We are experimenting with diagnosing cloud base height from the radiance observations using two approaches: (1) assuming the cloud is radiating as a blackbody and determining the height using a window channel (e.g., 11 μm) and the previously retrieved temperature profile, and (2) using the minimum local emissivity variance (MLEV) technique (Huang et al. 2004). If either of these methods, or perhaps a combination of the two, provide an accurate estimate of cloud base height, then TROPoe will be modified to allow them to be used in place of the ceilometer observation.

Many MWRs are deployed with broadband infrared thermometers (BIRT) mounted on them (e.g., Solheim et al. 1998; Marke et al. 2016); these can also provide some sensitivity to cloud base height. Cloud base height could be derived using the same "assuming the cloud is a blackbody" approach described above. However, TROPoe does not yet emulate these BIRTs, but future versions will allow these observations to be used in the retrieval.

6) Prior mean and covariance

a) Thermodynamic prior covariance matrix

i) Structure of the prior data

The retrieval of thermodynamic profiles from passive spectral radiance data is a mathematically ill-posed problem; namely, there are many different solutions (i.e., profiles) that if input into a radiative transfer model would agree with the observation within its uncertainty. However, many of these solutions are not physically realistic. For example: suppose the true temperature profile follows a standard lapse rate; a profile that oscillates back and forth with height around this true profile could be possible solution mathematically, but that type of profile isn't observed in nature. Stated a different way, there is correlation between any two levels in the atmosphere, and incorporating information about this correlation helps to constrain the derived solution to a more physically realistic profile. This is the job of the prior covariance matrix.

TROPoe requires the use of an a priori dataset to help constrain the retrieved thermodynamic profiles. These prior datasets are typically built using thousands of high-resolution radiosondes. Given this dataset, the ambient temperature and water vapor mixing ratio profiles from each radiosonde are interpolated to a fixed height grid. The thousands of interpolated sondes are used to compute both a mean profile X_a and a covariance matrix S_a that are structured like this:

$$X_{a,Tq} = (\overline{T(z)} \quad \overline{q(z)})^T$$

where $\overline{T(z)}$ is the mean ambient temperature profile [degC] from the surface to the uppermost height (which should be above the tropopause), and $\overline{q(z)}$ is the mean water vapor mixing ratio profile [g kg^{-1}].

$$S_{a,TQ} = \begin{bmatrix} S_{T,T} & S_{T,q} \\ S_{q,T} & S_{q,q} \end{bmatrix}$$

where $S_{T,T}$ is the covariance matrix of temperature and temperature, $S_{q,q}$ is the covariance matrix of water vapor mixing ratio and mixing ratio, and $S_{T,q}$ is the covariance of temperature and water vapor mixing ratio.

TROPoe has three priors that are distributed with (i.e., inside) the container:

- prior.MIDLAT.nc – created using 5868 profiles from the ARM SGP site
- prior.POLAR.nc – created using 1493 profiles from the ARM NSA site
- prior.TROPIC.nc – created using 631 profiles from the ARM MAO deployment

When using any of these three priors, you specify these names directly as the priorFile (i.e., with no path). However, if you desire to use your own created prior, then the path should be specified to your prior (e.g., /data/specialPrior.nc).

The three embedded prior covariance matrices are computed on a vertical grid that has finer resolution at the surface that coarsens with height; this is to capture finer detail near the surface where there is higher information content in the IRS and MWR observations. If the user wants to

use their own prior dataset, we encourage them to use a similar geometrically expanding-with-height vertical grid.

ii) Vertical grid

TROPoe allows the user to specify the vertical grid that they desire to use in the retrieval. This is done via the VIP keyword `zgrid`. This comma delimited array, which has unit [km AGL], must have its first value at 0, its last value below the maximum height in the prior covariance file (which is 20 km for the three standard priors), and be monotonically ascending. TROPoe will interpolate the prior covariance matrix to this input zgrid. It is important to realize that the true vertical resolution is not given by zgrid, but that the choice of zgrid can be useful when there is a desire to compare against another observation. For example, if the IRS/MWR is located next to a tall tower that has in-situ observations at various heights, then including these heights in zgrid can facilitate the intercomparison. Also note that the overall retrieval speed is inversely proportional to the number of levels in zgrid.

We caution the user against creating their own prior datasets with a very large number of vertical levels, especially if the spacing between the levels is very fine, as numerical issues could result. Please consult with TROPoe developers if you are considering retrievals on vertical grids such as these.

iii) Recentering of the thermodynamic prior

The prior dataset consists of a mean profile of temperature and water vapor mixing ratio, and a matrix that describes the covariance around this mean. However, if the current conditions are markedly different than the prior mean profile, then the biases in the retrieved profile (especially in regions where the signal in the observations is low) can result. To help mitigate this, we borrow a technique from data assimilation used within numerical weather prediction centers. In particular, TROPoe has the ability to recenter the prior data. This is done by adjusting the mean profile, and then modifying the covariance matrix accordingly. The goal is to reduce the bias in the mean profile enough that the retrieved profile is not biased by the difference between the mean prior and the true conditions.

The VIP keyword `recenter_prior` is used to turn on / off the recentering. Its default is on.

Of the two retrieved variables, the signal-to-noise for water vapor is weaker than for temperature in both the IRS and MWR. Thus, our approach is to recenter the mean water vapor first. This is done by using the input surface met data and comparing the daily mean in-situ value with the lowest level in the prior. The ratio of these two values (SF) is used as a height-independent multiplier to the entire prior mean water vapor profile, which brings the adjusted mean water vapor profile in closer agreement with the current conditions. Note that the magnitude of the QQ, TQ, and QT portions of the covariance matrix are scaled also. The new mean temperature profile is computed as the temperature required to yield the same relative humidity profile as was in the original covariance matrix. Since the relative change in the temperature is small (on the Kelvin scale), no adjustment is applied to the TT, TQ, or QT parts of the covariance matrix for this change in temperature.

Suppose that the user does not have any independent surface met data, yet would still like to recenter the prior. There are two options. The default fallback option is to first estimate the near-surface temperature value from the brightness temperature derived from the IRS (using the 675-680 cm^{-1} region), or if the IRS is not available from the highest frequency channel in the 56-60 GHz region from the MWR. The daily mean of this near-surface temperature estimate is computed. Again, assuming that the RH profile from the original prior is to be conserved, a new surface water vapor mixing ratio value is computed that would be in agreement with this derived radiometric near-surface temperature value. This new water vapor value is used to compute SF, and the logic from the above paragraph is used. One drawback of this method is that RH at the surface can be quite variable, and thus this approach can occasionally lead to poorer retrievals. The second option is to let the user specify the value of the surface water vapor mixing ratio, and thus control the SF value. This is done via the VIP keyword `recenter_input`.

iv) Specifying the thermodynamic prior

The prior is specified by the user on the command line when executing the retrieval. If the user specifies "prior.TROPICS.nc", "prior.MIDLAT.nc", or "prior.POLAR.nc" then the corresponding embedded prior will be used. If the command line option for priorFile is some other value, the retrieval will use prior file specified for the retrieval. This provides the user the ability to evaluate the impact of the prior on the accuracy of the retrieval.

b) Other prior information that needs to be specified

i) Cloud prior covariance information

The prior covariance discussed above was in the context of temperature and water vapor, which is why we denoted the mean and its covariance as $\boldsymbol{X_{a,TQ}}$ and $\boldsymbol{S_{a,TQ}}$. However, TROPoe also retrieves cloud properties (and indeed, we encourage users to always have at least one of `retrieve_lcloud` or `retrieve_icloud`, or both, turned on). This means we need to specify the prior conditions (mean and standard deviation) for the cloud properties that will be retrieved. These values are set in the VIP file. The VIP keywords, and their default values [units], are

```
prior_lwp_mean = 0      # mean liquid water path [g/m^2]
prior_lwp_sdev = 200    # standard deviation of liquid water path [g/m^2]
prior_lreff_mean = 8    # mean liquid droplets effective radius [µm]
prior_lreff_sdev = 4    # standard deviation of liquid droplets effective radius [µm]
prior_itau_mean = 0     # mean ice optical depth [unitless]
prior_itau_sdev = 50    # standard deviation of ice optical depth [unitless]
prior_ireff_mean = 25   # mean ice crystal effective radius [µm]
prior_ireff_sdev = 8    # standard deviation of ice crystal effective radius [µm]
```

It is important that, if you are retrieving cloud properties, that the prior_lwp_sdev and prior_itau_sdev values are large enough as to not over constrain the retrieval. Note that the signal-to-noise for cloud properties is extremely large in the IRS, and pretty strong in the MWR, so having mean values for `prior_lwp_mean` and `prior_itau_mean` of zero is not very restrictive (given the very large uncertainty for those two parameters as specified by the sdev values).

Naturally, the user can override any of these with their own values. These are added to the prior mean vector and covariance matrix like this:

$$X_{a,TQ,c} = (\overline{T(z)} \quad \overline{q(z)} \quad c)^T$$

$$S_{a,TQ,c} = \begin{bmatrix} S_{T,T} & S_{T,q} & 0 \\ S_{q,T} & S_{q,q} & 0 \\ 0 & 0 & S_{c,c} \end{bmatrix}$$

where *c* denotes the cloud properties arranged as [lwp, lreff, itau, ireff]. Note that we assume there is no correlation between T and C or Q and C, hence the zeros in the above covariance matrix.

ii) Trace gas prior covariance

TROPoe has some limited ability to retrieve trace gases; namely, carbon dioxide (CO_2), methane (CH_4), and nitrous oxide (N_2O). This capability is still very experimental, and thus we recommend that these retrievals be turned off (using VIP keywords `retrieve_co2`, `retrieve_ch4`, and `retrieve_n2o`, respectively) if the objective is to get thermodynamic profiles and cloud properties. The default is for these components to be turned off; to turn one or more of these on, then the retrieve flag should be set to 2 (not 1).

We have very few vertical profiles of these trace gases from which to build a statistically we-defined level-to-level covariance matrix. Thus, if a trace gas (TG) retrieval is enabled, we assume that the shape of the TG profile is a constant in the free troposphere, and a different constant in the PBL (i.e., a "chair" profile). The code actually retrieves a three-element vector for each trace gas: [free troposphere concentration, difference in PBL vs free troposphere concentration, and "shape"]. (Note: the "shape" parameter goes with retrieve_xxx = 1; this option was experimental and is no longer supported).

Like the cloud prior information, trace gas prior information needs to be appended to the covariance matrix. These are specified via the VIP file with these keywords, each of which is expecting a 3-element vector:

```
prior_co2_mean = [-1, 5, arbitrary]              # units ppm, ppm, and unused
prior_co2_sdev = [2, 15, arbitrary]              # units ppm, ppm, and unused
prior_ch4_mean = [1.793, 0, arbitrary]           # units ppm, ppm, and unused
prior_ch4_sdev = [0.0538, 0.0015, arbitrary]     # units ppm, ppm, and unused
prior_n2o_mean = [0.310, 0 ,arbitrary]           # units ppm, ppm, and unused
prior_n2o_sdev = [0.0093, 0, arbitrary]          # units ppm, ppm, and unused
```

These TG = are added to the mean prior and its covariance like this, giving the actual mean prior and prior covariance that is used in the retrieval:

$$X_a = (\overline{T(z)} \quad \overline{q(z)} \quad c \quad TG)^T$$

$$S_a = \begin{bmatrix} S_{T,T} & S_{T,q} & 0 & 0 \\ S_{q,T} & S_{q,q} & 0 & 0 \\ 0 & 0 & S_{c,c} & 0 \\ 0 & 0 & 0 & S_{TG} \end{bmatrix}$$

A few comments about these default values:

- The –1 in the first element of the CO_2 mean value has special meaning. CO_2 has a strong seasonal cycle, and the mean annual value has been increasing steadily for the last 6 decades. TROPoe has a very simple parameterization, built using data from the NOAA site at Mauna Loa, that uses the year and month of the day being processed to estimate the CO_2 concentration in the free troposphere. When this flag is set to –1, this parameterization is used to provide the estimated free-tropospheric value for the retrieval. Naturally, this is only an approximation and works best for the northern hemisphere.
- If a standard deviation is set to zero (whether in the trace gas, cloud, or any section), the assumed intent is that the user does not want the retrieval to actually retrieve that value. So, in this example here, the zero in the second element of the `prior_n2o_sdev` implies that the retrieval should not change the relative PBL concentration at all; i.e., that the N_2O profile should be treated as a constant with height profile by the retrieval. Note that TROPoe actually will replace this zero with an extremely small number in the processing, as a diagonal matrix (like this part of the a priori covariance matrix) cannot be inverted with zeros on the diagonal.
- The default mean and standard deviation values for free-tropospheric elements of the CH_4 and N_2O are derived from a multi-year analysis of TCCON data at the ARM SGP site.

The depth of the PBL (PBLH), which separates the PBL from the free troposphere and thus is important for these trace gas retrievals, is derived from the retrieved temperature profile using a lifted parcel approach and the potential temperature profile. There is a minimum (floor) value set for the PBLH; this is set via the VIP keyword `min_PBL_height` [km AGL], which has a default value of 0.050 km. The user can also set a maximum value (ceiling) for the PBLH also using the VIP keyword `max_PBL_height` [km AGL].

7) Interpreting TROPoe output

TROPoe output is in a netCDF file, and each file contains a wealth of data that can be used to understand the quality of the retrieval. The fields in the output file are arranged in this order: time fields, qc_flag, retrieved fields, uncertainties in the retrieved fields, fields associated with deeper understanding of the quality of the retrieval, fields for degrees of freedom (DFS) for signal and true vertical resolution, diagnosed and derived fields, fields capturing the observations used and the forward calculation for the final solution, and the full error posterior covariance matrix and the input prior covariance matrix. Each of these fields has many field-level attributes to describe the fields. The global attributes include all of the information needed to recreate the output. The header of an example of output file is given in Appendix 4.

a) Quality control flags

There are several fields that provide information on the quality of the TROPoe in the output file. The primary field is the 'qc_flag'. This field as four possible values:

- 0 – implies the quality appears to be good
- 1 – implies the IRS hatch was not open for full observation period (TROPoe no longer processes these samples, so you should not see this value of qc_flag)
- 2 – implies retrieval did not converge
- 3 – implies retrieval converged, but RMS between observed and computed spectrum is too large

A qc_flag of 2 suggests that the user should check the 'converged_flag' variable (which will be described below) for more information about the lack of convergence. This does not necessarily mean that the retrieval is "bad", but the users will need to do more investigations of the variables that describe the quality of the retrievals to ensure that the retrieval is acceptable. Finally, a qc_flag of 3 indicates that the RMS difference of the radiance observations from the forward calculation of the state vector is larger than the VIP entry `qc_rms_value`. (This threshold may be site specific, which is why we allow the user to update it via the VIP file). These retrievals are most likely bad and should not be used.

The output files contain two RMS values that describe how consistent the retrieval is with the observations. The first RMS, 'rmsa', is calculated for the entire observation vector, which has *n* elements in it:

$$rmsa = sqrt\left(\frac{\sum\left(\frac{Y - F(X)}{\sigma_Y}\right)^2}{n}\right)$$

Notice that the difference between the observation vector and the forward calculation is normalized by the uncertainty of the observation in this calculation to account for the varying levels of uncertainty in the observation vector. RMSa is the RMS value that is used to determine the qc_flag (using the qc_rms_value keyword mentioned above) and is also monitored during each iteration to check if the retrieval is trending away from the observations. The second RMS value in the output file is 'rmsr', which is the RMS for only the radiance observations in the

observation vector (IRS and MWR observations). This is calculated the same way as rmsa. The inclusion of rmsr in the output allows the user to differentiate whether a high rmsa value is caused by the radiance observations or other observation types in the observation vector.

TROPoe checks for convergence in the retrieval using the technique described in Rodgers (2000). The retrieval is determined to converge when the difference in the forward calculation between subsequent iterations scaled by the observational uncertainty is less than 30% of the length of the observation vector. If the retrieval does not converge before the maximum number of iterations is reached (which is set by the VIP entry `max_iterations`), then iteration with the best rmsa value is used as the final retrieval. If at any time while the code is iterating the rmsa drastically increases over the iteration with best rmsa, then the code will stop iterating as the retrieval is likely trending away from the proper solution and the iteration with the best rmsa will be used as the final retrieval. Although unlikely to occur, the code will also stop iterating if a NaN is found in the updated state vector. In this scenario, the previous iteration is used as the final retrieval. These different states of convergence (or lack thereof) are stored in the output variable converged_flag, which has these values:

- 1 – indicates convergence in Rodgers sense (i.e., di2m << dimY) (this is ideal)
- 2 – indicates best rms after rms increased drastically
- 3 – indicates best rms after maximum iterations
- 9 – indicates NaN in updated state vector

In order to account for the first guess state vector potentially being far away from the final solution, a scalar parameter, 'gamma', is applied to increase the weight of the prior in the retrieval during the early iterations. This helps to keep the retrieval stable while the observations with the most information content slowly modify the retrieval towards the correct solution; i.e., this is key to overcoming a bad first guess. The value of gamma decreases with each iteration until it becomes 1. The rate of decrease in gamma as a function of iteration number is different depending on whether the master instrument is an IRS or MWR, and is as follows:

IRS: gamma = [1000, 300, 100, 30, 10, 3, 1, 1, 1, ...]
MWR: gamma = [100, 30, 10, 3, 1, 1, 1, ...]

The code will not check for convergence (in the Rodgers sense) until gamma = 1. However, the iterations can still be stopped if rmsa drastically increases. Therefore, the value of gamma for the final retrieval is stored in the output variable gamma. The VIP keyword qc_gamma_value (default value of 5) is used to flag samples with high gamma values as poor quality in the qc_flag.

Finally, with the different ways for the retrieval to finish, it is important to know the iteration number of the final solution. This information is stored in the 'n_iter' variable. For a retrieval that converged in the traditional way this will be the last iteration, but when the retrieval does not converge this will be the iteration with the lowest rmsa.

b) Actual retrieved fields

i) Temperature and humidity

Temperature and humidity are two of the retrieved fields that most users probably desire the most. The temperature field is output in degrees Celsius and stored in the output variable 'temperature'. The humidity variable for the retrieval is water vapor mixing ratio, is output with units of g/kg, and stored in the 'waterVapor' output variable. Both temperature and waterVapor are 2-dimensional fields with dimensions of (time, height).

It is also possible to run TROPoe without retrieving temperature or humidity. In these cases, the VIP entries `retrieve_temp` and `retrieve_wvmr` are set to 0. When these fields are not retrieved, the output variables will be equal to the first guess profiles for a retrieval times. This is experimental, and not recommended. If cloud properties only are desired, the user is encouraged to use the MIXCRA (mixed-phase cloud retrieval algorithm; Turner 2005), which is also in the TROPoe Docker container. Contact the TROPoe developers for more information.

ii) Cloud properties

TROPoe also has the ability to retrieve cloud properties. The retrieved cloud property variables are:

- lwp – Liquid water path in g/m^2
- lReff – Liquid water effective radius in µm
- iTau – Ice cloud optical depth (geometric limit)
- iReff – Ice effective radius in µm

All of the cloud property retrieved fields have a single dimension which is time. Like temperature and humidity, the retrieval can be run without retrieving these cloud properties. This is controlled by the VIP entries `retrieve_lcloud` and `retrieve_icloud`. When these fields are not retrieved the output variables will contain the prior mean value for that field. Note: the default VIP entries are `retrieve_lcloud`=1 and `retrieve_icloud`=0. Also, for the frequencies used by typical MWRs (i.e., frequencies between 20 and 60 GHz) there is negligible sensitivity to ice, and thus we recommend that retrieve_icloud = 0 when an IRS is not used in the retrieval.

iii) Trace gases

TROPoe currently is able to retrieve limited information on three trace gases: carbon dioxide, methane, and nitrous oxide. The output variables "co2", "ch4", and "n2o", along with the 1-σ uncertainties, are each 3-element vectors. The first element is the gas concentration in the free troposphere, the second element is the difference of the PBL concentration minus the concentration in the free troposphere, and the third element is no longer used (it used to specify the profile shape, but TROPoe only uses the "chair profile"). These 3-element vectors are translated into actual profiles named "co2_profile", "ch4_profile", and "n2o_profile" for ease of use.

c) Uncertainties in the actual retrieved fields

One of the biggest strengths of TROPoe is that the retrieval propagates the uncertainties from the prior and the observations and the sensitivity to the forward model through the calculation of the posterior covariance matrix. The square-root of the diagonal of the posterior covariance matrix provides the 1-σ uncertainties for the final state vector. These 1-σ uncertainties for the retrieved fields in the following output file

- sigma_temperature – 1-σ uncertainty in temperature
- sigma_waterVapor – 1-σ uncertainty in water vapor mixing ratio
- sigma_lwp – 1-σ uncertainty in liquid water path
- sigma_lReff – 1-σ uncertainty in liquid water effective radius
- sigma_iTau –1-σ uncertainty in ice cloud optical depth (geometric limit)
- sigma_iReff – 1-σ uncertainty in ice cloud effective radius
- sigma_co2, sigma_ch4, sigma_n2o – see section 6b-ii

The off-diagonal elements of the posterior covariance matrix describe the covariance between different elements of the state vector. The posterior covariance matrix's variable name in the output file is 'Sop' and it has three-dimensions which are (time, arb, arb) where arb is the length of the state vector ***X***. There is also an output variable named 'arb' that is encoded to specify what each element in the state vector and posterior covariance matrix is:

- 1 – temperature
- 2 – water vapor
- 3 – liquid water path
- 4 – liquid water effective radius
- 5 – ice cloud optical depth
- 6 – ice cloud effective radius
- 7 – carbon dioxide
- 8 – methane
- 9 – nitrous oxide

d) Information content in actual retrieved fields

Observations seldom measure what is desired: in this application, both the IRS and MWR are measuring the downwelling radiation emitted by the atmosphere when we really desire thermodynamic profiles. These are ill-defined mathematical problems, which are constrained by the use of a prior dataset that ideally is based upon climatology. Thus, a natural question to ask is how much information on the retrieved profiles is actually in the observation? Said another way: how many independent levels are in the retrieved profiles? We use the concept of degrees of freedom for signal (DFS), which is defined as the diagonal of the averaging kernel (***A***), which can be written as:

$$A = I - \frac{S_{op}}{S_a} \qquad \text{Eq 3}$$

where $\boldsymbol{I}$ is the identity matrix (Turner et al. 2025). The prior covariance matrix $\boldsymbol{S_a}$ illustrates the climatological "volume" of state vector, and posterior covariance matrix $\boldsymbol{S_{op}}$ is the "volume" of that space that results after the retrieval is performed and the information from the observations are included. If there is no information about the state vector from the observations, then $\boldsymbol{S_{op}}$ will be approximately $\boldsymbol{S_a}$, and thus $\boldsymbol{A}$ is approximately 0. If there is a lot of information in the observations, then $\boldsymbol{S_{op}}$ will be markedly smaller than $\boldsymbol{S_a}$ and thus $\boldsymbol{A}$ will be approximately 1.

The field "DFS" is a 16-element vector that indicates the DFS in each of the variables, where the elements are the DFS in each of these retrieved variables:

DFS = [overall_total, total_T, total_q, LWP, lreff, iTau, ireff, CO_2(3), CH_4(3), N_2O(3)]

where overall_total is the total DFS for all variables, and total_T and total_q are the total DFS in the profiles of ambient temperature and water vapor mixing ratio, respectively. The next four elements are the DFS in the cloud variables. The final 9 elements provide the DFS for the trace gases, where there are three elements for each gas: the DFS in the free troposphere, the DFS in the PBL, and an unused value (historical).

It is important to know where the information content is vertically in the retrieved temperature and humidity profiles. This is given by the fields 'cdfs_temperature' and 'cdfs_waterVapor', which are the cumulative DFS profiles. These profiles illustrate how much information is in the retrieved profile below the selected height.

There are two other fields that are very useful to understand the information content in the retrieved thermodynamic profiles: 'vres_temperature' and 'vres_waterVapor' which provide the true estimate of the vertical resolution of these retrieved profiles. These two profiles are derived from the averaging kernal. Many users confuse the vertical grid used for the retrieval as the vertical resolution of the retrieval; however, that is false, and the true vertical resolution is given by these two fields.

The averaging kernal can be used to smooth higher resolution data (e.g., collocated radiosondes that you wish to compare against TROPoe retrievals) to the same vertical resolution of the retrieval. Turner and Löhnert (2014) illustrate how this done.

e) Derived fields

After the final state vector has been retrieved, the ambient temperature and water vapor mixing ratio profiles are used to obtain derived profiles and indices which are included in the output files.

i) Profiles

The derived profiles are:

- theta – potential temperature [K]
- thetae – equivalent potential temperature [K]
- rh – relative humidity [%]
- dewpt – dew point temperature [C]
- co2_profile – carbon dioxide profile [ppm]

- ch4_profile – methane profile [ppm]
- n2o_profile – nitrous oxide profile [ppm]

These variables a have dimensions of (time, height). When calculating the profiles, the pressure that is used is obtained from the surface pressure data and the hypsometric equation is used to calculate the change in pressure with height. The trace gas profiles are derived from the actual 3-element retrieved variables, which are in the variables 'co2', 'ch4', and 'n2o'.

ii) Indices

A large number of useful variables can be derived from profiles of temperature and humidity. The derived indices are a stored in separate variables in the output files. Each variable has dimensions of (time) and correspond to the following variables:

- pwv – precipitable water vapor [cm]
- pblh – planetary boundary layer height [km AGL]
- sbih – stable boundary layer inversion height [km AGL]
- sbim – stable boundary layer inversion magnitude [degC]
- sbLCL – surface-based lifted condensation level [km AGL]
- sbCAPE – surface-based CAPE [J/kg]
- sbCIN – surface-based CIN [J/kg]
- mlLCL – mixed-layer lifted condensation level [km AGL]
- mlCAPE – mixed-layer CAPE [J/kg]
- mlCIN – mixed-layer CIN [J/kg]

Note that most-unstable CAPE, CIN, and LCL are not computed, as Blumberg et al (2017) demonstrated that that the poorer vertical resolution of these passive retrievals impacts the accuracy of most-unstable indices.

The output file also provides 1-sigma uncertainties for these derived indices in the corresponding 'sigma_xxx' variable that has the same dimensions as derived indices. These uncertainties are determined through Monte Carlo sampling of the posterior covariance matrix to obtain perturbed thermodynamic profiles from which the samples of the derived indices are calculated to obtain the spread of each index.

f) Evaluating how well the observations are fit

The TROPoe output files include information on the observations that are used for each retrieval. All of the observations are stored in the 'obs_vector' variable (this is $\boldsymbol{Y}$) which has dimension of (time, Ydim) where Ydim is the length of the observation vector. The units for obs_vector are mixed as it can contain many different types of observations (e.g. IRS radiances, MWR brightness temperatures, surface met station, etc.). The 'obs_flag' variable indicates the type of observation for each element in obs_vector, and the field-level attributes of obs_flag provides the units of the various observations in the obs_vector. The 1-σ uncertainties of the elements in obs_vector are stored in 'obs_vector_uncertainty'. The field 'obs_dim' provides the

wavenumber, frequency, height, etc. of the elements in obs_vector. The possible values for obs_flag and the corresponding meaning of the values in obs_dim are:

- 1 – radiance in mW/(m^2 sr cm^{-1}) from IRS; obs_dim – wavenumber [cm^{-1}]
- 2 – brightness temperature in K from zenith-pointing MWR; obs_dim – frequency [GHz]
- 3 – temperature from an external profile (units depend on type); obs_dim – height [km]
- 4 – water vapor from external profile (units depend on type); obs_dim – height [km]
- 5 – surface met station temperature (units depend on type); obs_dim – height [km]
- 6 – surface met station water vapor (units depend on type); obs_dim – height [km]
- 7 – temperature from a model profile (units depend on type); obs_dim – height [km]
- 8 – water vapor from a model profile (units depend on type); obs_dim – height [km]
- 9 – carbon dioxide in-situ obs (units depend on type); obs_dim – height [km]
- 10 – brightness temperature in K from scanning MWR; obs_dim – frequency*1000 + elevation/1000 [arbitrary]
- 11 – temperature from previous TROPoe retrieval (add_tropoe); obs_dim – height [km]
- 12 – water vapor from previous TROPoe retrieval (add_tropoe); obs_dim – height [km]

The "add_tropoe" option is described in later section. Thus, an easy way to see what observation types are used in a retrieval, just make a plot of "obs_type" and note which values are set using the above list.

Also included in the output file is the forward calculation for the final state vector in the variable 'forward_calc'. The forward calculation is the expected value of the observation based on the retrieved state vector. The forward calculation is performed using the forward models discussed in Section 9. The values in obs_flag and obs_dim also apply to forward_calc. Having both the obs_vector and forward_calc in the output allows the user to examine how we the observations are being fitted by the retrieval. For a "good" retrieval the difference between the elements in the obs_vector and forward_calc should be within the obs_vector_uncertainty. This type of analysis of the observation vector and forward calculation, especially when evaluating by the observation type denoted in obs_flag, can help the user to pick out which observation might be causing problems in the retrieval (e.g., inconsistency between different observation types). Note that the if either the observation or the forward calculation has a value less than -700, then the value is treated as a missing value and that particular observation will not impact the retrieval.

g) Prior and posterior covariance matrices

The output file contains the prior mean ($\boldsymbol{X_a}$) and its covariance matrix ($\boldsymbol{S_a}$), which are used for each retrieval. It also has, for each sample, the full posterior covariance matrix ($\boldsymbol{S_{op}}$) and averaging kernal ($\boldsymbol{A_{kernal}}$) also. A lot of information is already extracted from $\boldsymbol{S_{op}}$, such as the 1-σ uncertainties for the temperature and humidity profiles. Monte Carlo sampling, following the example in Turner and Löhnert (2014), is used to generate 20 representative profiles from each retrieval, from which each derived index is computed and the standard deviation of these samples is reported as the uncertainty in the derived index. Also, the off-diagonal elements of $\boldsymbol{S_{op}}$ allow the user to evaluate the correlated error between any two levels (e.g., T(z_i) vs T(z_j)), or between different variables (e.g., T(z_i) vs q(z_j) or q(z_i) vs LWP, etc).

The averaging kernal allows the user to investigate the DFS (by analyzing the diagonal), the vertical resolution, or the smoothing. See Rodgers (2000) for more information on how to evaluate and utilize information in $\boldsymbol{A_{kernal}}$.

h) Understanding the global attributes

The global attributes of the output file contain the version of TROPoe used for the retrieval, date the retrieval was performed, and references for TROPoe, LBLRTM, and MonoRTM. The global attributes also have information on the prior that was used. Lastly, the global attributes specified in the VIP file by the user and all of the entries for the VIP parameters when the retrieval was performed are also included.

8) Miscellaneous topics

a) Forward models

The IRS forward model is the LBLRTM (Clough et al. 1995; Clough et al. 2005). There are two versions included in the container: v12.1 and v12.17. The water vapor continuum absorption was modified in the latter version (Mlawer et al. 2024). TROPoe versions prior to v0.20 used LBLRTM v12.1, whereas TROPoe versions starting from v0.20 will have LBLRTM v12.17 set as the default.

The MWR forward model is the MonoRTM (Clough et al. 2005; Payne et al. 2011), version 5.0. This absorption model uses the improved liquid water absorption model (Turner et al. 2016b). Work is underway to implement Rosenkranz (2017), which would be selectable via the VIP file in a future TROPoe version.

Clouds are included in the radiative transfer calculations. The assumption is that only liquid clouds will influence the MWR forward calculation; i.e., that the MWR forward calculation is insensitive to ice. This is clearly not true if the MWR frequency is above 200 GHz, and if observations at these high frequencies will be used then the radiative transfer codebase will need to be updated.

In the infrared, we use precomputed cloud properties by treating liquid and ice cloud particles as spheres, computing their radiative properties for a range of sizes, and integrated these properties over a gamma size distribution for different effective radii. These "single scattering property databases" (SSP_DBs) were used the MIXCRA cloud retrieval algorithm (Turner 2005), and were convenient to use here. The VIP file allows the user to select other SSP_DBs also; contact the authors for details.

b) The "add_tropoe" option

The baseline configuration of TROPoe treats each new radiance observation (i.e., each sample time in the retrieval) independently from the previous retrievals, and thus there can be uncorrelated noise in adjacent profiles that is not realistic. However, it is well-known that the evolution of temperature and humidity at any given altitude has some temporal correlation that depends on the location (in the PBL or free troposphere), stability, and other factors. Adler et al. (2024) demonstrated that an earlier retrieval, say at time (t-1), could be used as part of the input observation vector $\boldsymbol{Y}$ at time (t), which helps to reduce the uncorrelated noise without masking genuine mesoscale atmospheric changes. The key to the approach is adjusting the uncertainties of the retrieved thermodynamic profile at time (t-1) appropriately as to not (a) overfit the lowest 1 km where the MWR and IRS have the largest information content, (b) that only "good" clear sky retrievals from a previous time are used as input to the current retrieval, and (c) inflating the uncertainties of the previous good retrieval as the time separation between it and the current time being processed grows. Using this approach, Adler et al. was able to demonstrate that water vapor time-series from the TROPoe retrieval at different heights from IRS observations at the ARM SGP site better matched the autocorrelation with the co-located Raman lidar. They also demonstrated that the approach improves the information content in the retrievals, especially above 1 km, for both IRS and MWR-based retrievals.

The VIP "add_tropoe_T_input_flag" and "add_tropoe_q_input_flag" options enable the capability to include previous temperature and/or water vapor retrievals as input into the current retrieval. The VIP entries "add_tropoe_input_lwp_thres", "add_tropoe_input_cbh_thres", and "add_tropoe_gamma_threshold" are used to identify the retrievals that are considered "good" and can be used as input into a future retrieval. How these are used are described in the Appendix 1, section a. Only the closest good retrieval to the one currently being processed will be included into ***Y***. The random error in the previous good retrieval will be inflated using a linear function between the surface and the lower of either the PBL height or 1 km, using other VIP keywords in the experimental VIP list (Appendix 1, section b). Note that if the "add_tropoe" option is turned on, then the output files "obs_flag" fields will be marked with 11 (for TROPoe temperature input) or 12 (for TROPoe water vapor input), allowing these observations to be easily identified in the observation vector ***Y***.

Using the add_tropoe option, and its impacts, are demonstrated in section 9 below.

The trace gas retrievals would benefit from also using the add_tropoe approach, as the evolution of trace gases is relatively slow with time. This feature is currently under development, and will be available in a future version of TROPoe.

c) Append mode

TROPoe is designed to process data in both regular mode (i.e., when the entire day of data is already collected) and in 'real-time' mode (i.e., regularly performing retrievals as the data is being collected). The latter is done using the "append mode", which is selected when the VIP keyword output_clobber is set to 2. In this mode, TROPoe reads the input files and the preexisting output file, and only processes observations that were collected after the last sample in the preexisting output file. These new sample(s) are then appended to the preexisting output file. The TROPoe developers have used this mode extensively to process the retrievals in real-time. Note that the user should use not change the VIP file or prior when using the append mode. If a preexisting output file does not exist, then TROPoe will create a new output file as it does in normal processing mode.

d) Interpretation of the retrieval in and above cloud base

Compared to a layer of cloud free atmosphere, clouds have markedly higher optical depth, especially at infrared wavelengths. Thus, the presence of a cloud can greatly impact the ability of the retrieval to get accurate answers both just below the cloud base (due to the vertical extent of weighting functions), as well as above cloud base (because of the attenuation of the downwelling radiance from above by the cloud). Thus, care needs to be taken when interpreting results above cloud base. If IRS data are used in the retrieval, we urge users to cautiously use data above cloud base if the LWP is greater than roughly 10 g m^{-2}, as the optical depth of the cloud increases very quickly in the infrared as LWP increases (Turner 2007). Liquid clouds are less attenuating at microwave wavelengths, and thus including MWR data can improve the information content above cloud base. The fields "cdfs_temperature" and "cdfs_waterVapor" provide profiles of information content, and examining the differences in these fields for cloudy cases vs clear sky cases will help the user appreciate how sensitive the retrievals are to clouds.

9) Example VIP files: Different use cases

This section provides several different use cases that demonstrate how to run TROPoe for different configurations. These examples can be found in a tar file on the Zenodo archive (doi:10.5281/zenodo.18774680). That tar file has an input directory, where the input files needed for the examples are kept, and a multitude of other directories for each case study. In each example, there is a README and one or more VIP files that illustrate how the retrieval was configured. To rerun these examples yourself, untar this tar file in a working directory, and in that working directory, execute the command that is in the corresponding README file. We have provided the output you should expect as netCDF files that have been renamed to have the suffix ".orig".

In those example directories, you will also find many quicklook plots that illustrate aspects of the data that were created by TROPoe, as well as the IDL scripts that were used to create the plots. We realize most readers of this guide are not IDL experts, but hopefully the in-line documentation is adequate enough that the IDL scripts can be used as pseudo code for your own analyses.

a) Microwave only, zenith-only, retrievals

For this retrieval, the only observations used in the retrieval are zenith-pointing MWR observations and surface met data. There are actually three examples in this directory (*usecase01*). We will provide some details for these here, but a more complete picture can be achieved by looking at all of the TROPoe output, quicklooks, and analysis script in the directory.

The input data used here is the HATPRO microwave radiometer that is deployed at Jülich, Germany, on 16 June 2022 as part of the JOYCE cloud observatory (Löhnert et al. 2015). This radiometer has a built-in surface meteorology station, so we will also read in those data into the retrieval. For this example, we did not download the collocated ceilometer, so the code will use the default CBH. (Due to the relatively low attenuation of microwave radiation by liquid water clouds, the impact of the thermodynamic profile retrieval is relatively insensitive to uncertainties in the CBH, as long as the CBH used in the retrieval is within a kilometer or so).

i) Example 1: no Tb bias correction

This is the VIP file used for this example (vip.no_Tb_bias_correction):

```
# Basic information
tres       = 10
station_lat = 50.9090
station_lon =  6.4140
station_alt = 84

# IRS block
irs_type = -1   # This indicates that the MWR will be the master instrument

# MWR block
mwr_type = 3    # The data are in the Uni Cologne format
mwr_path = /data/input_data/mwr.hatpro_juelich
```

```
mwr_rootname      = sups_joy_mwr00_l1_tb_zo_p00
mwr_elev_field    = ele
mwr_freq_field    = freq_sb
mwr_n_tb_fields   = 14
mwr_tb_field_names = tb
mwr_tb_freqs = 22.24, 23.04, 23.84, 25.44, 26.24, 27.84, 31.4, 51.26, 52.28, 53.86, 54.94, 56.66, 57.3, 58.0
mwr_tb_noise =  0.3,  0.3,  0.3,  0.3,  0.3,  0.3, 0.3,  0.7,  0.7,  0.5,  0.3,  0.2, 0.2, 0.2
mwr_tb_bias =  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, 0.0,  0.0,  0.0,  0.0,  0.0,  0.0, 0.0, 0.0

# Surface met block
ext_sfc_path = /data/input_data/mwr.hatpro_juelich
ext_sfc_rootname = sups_joy_mwr00_l1_tb_zo_p00
ext_sfc_time_format   = 2
ext_sfc_pres_type     = 1
ext_sfc_pres_fieldname = pa
ext_sfc_pres_units    = 3
ext_sfc_temp_type     = 1
ext_sfc_temp_fieldname = ta
ext_sfc_temp_units    = 2
ext_sfc_wv_type       = 1
ext_sfc_wv_fieldname  = hur
ext_sfc_wv_units      = 2

# Ceilometer block
cbh_type = 2
cbh_path = None
cbh_default_ht = 1.0

# Output block
output_path = /data/usecase01.mwr_only.zenith_only
output_rootname = tropoe.zenith_only.no_Tb_bias_correction

# Global attributes
globatt_site = JOYCE Site at Juelich, Germany
globatt_instrument = HATPRO version5
globatt_mwr_format = Uni Cologne formatted HATPRO data
globatt_cloud_info = No ceilometer used; default cloud base height used instead
```

Note that we've used "typical" noise levels for each frequency (the "mwr_tb_noise" VIP entry) above; it is not the function of any particular analysis.

We start our analysis by looking at the 'rmsr' and 'gamma' fields to make sure that the retrieval performed reasonably; if it struggles mightily, then there will be a lot of jumps in the rmsr field and gamma will have values much larger than 1. For this example, gamma was 1 for all samples (very good) and the rmsr was less than 1.5 (also good) – this is not shown here but can be found as figure "results1.before_Tb_corr.01.png". Next, we like to look at the LWP and PWV time-series to get a sense of the cloudiness and overall water vapor burden; this day exhibited clouds for the first few hours and then was cloud-free with the PWV ranging from approximately 1.6 to 2.3 cm (figure "…02.png"). Because this was the first time we've processed this particular MWR, we selected the clear-sky periods from 03-24 UTC and made a spectral bias plot (figure

"…03.png", shown here). Please look at the IDL analysis code in the directory to see how this was computed.

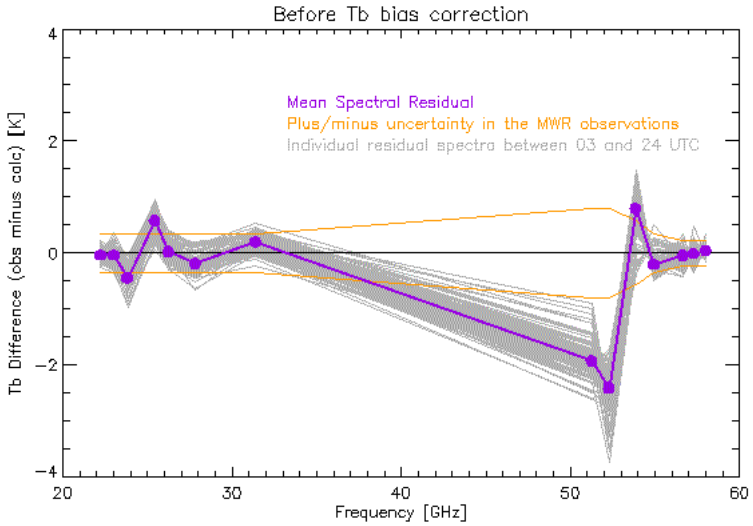

Figure 1: Spectral bias during clear sky periods

This figure demonstrates that the Tb difference in some channels is significantly outside the assumed Tb uncertainties (denoted as orange lines, and specified in the VIP above). This will result in the retrieval struggling against itself, as it is trying to find a solution (i.e., an atmospheric state ***X***) that satisfies all of the observations.

ii) Example 2: applying at Tb bias correction

This example builds upon the previous one, using the same day of data. As the above figure shows the mean spectral bias as observation minus calculation, we took the negative of this spectrum and added it to the VIP file for the next example (i.e., the VIP entry mwr_tb_bias will be ADDED to the observed Tb data, and thus should be computed at "truth" minus observation). The VIP file used here is exactly the same as above, with these two changes:

```
mwr_tb_bias = 0.04, 0.04, 0.45, -0.57, -0.02, 0.20, -0.20, 1.94, 2.42, -0.79, 0.21, 0.05, 0.02, -0.04
output_rootname = tropoe.zenith_only.yes_Tb_bias_correction
```

Again, we first investigated the rmsr and gamma fields. Gamma was again 1 for the entire day (same as before), but the rmsr decreased from a mean value of approximately 1.2 to 0.4 (i.e., 3x smaller); thus, applying the Tb bias correction allowed the retrieval to fit the observed MWR radiance much better.

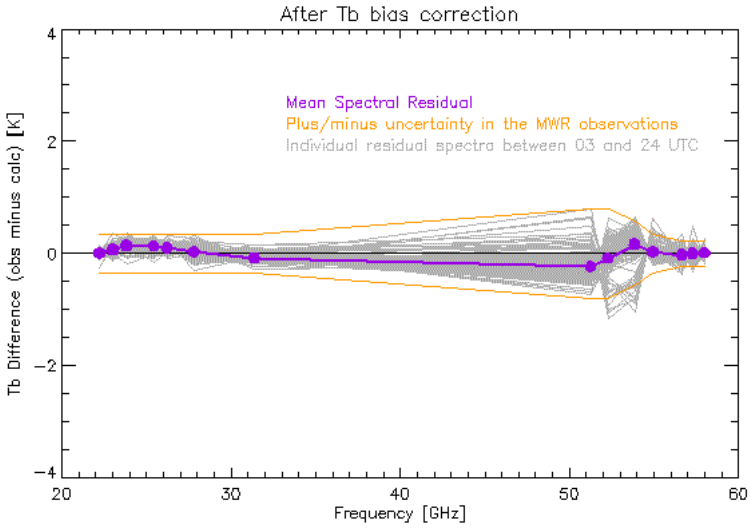

Figure 2: Spectral bias the results for the results that used the Tb corrected observations

We then recomputed the spectral bias, and noted that the differences for all spectral elements were within the assumed radiometric uncertainty. But is this a better retrieval? One subjective method to evaluate it is the temporal consistency of the results. We made time-height cross-sections of the retrieved temperature and water vapor mixing ratio.

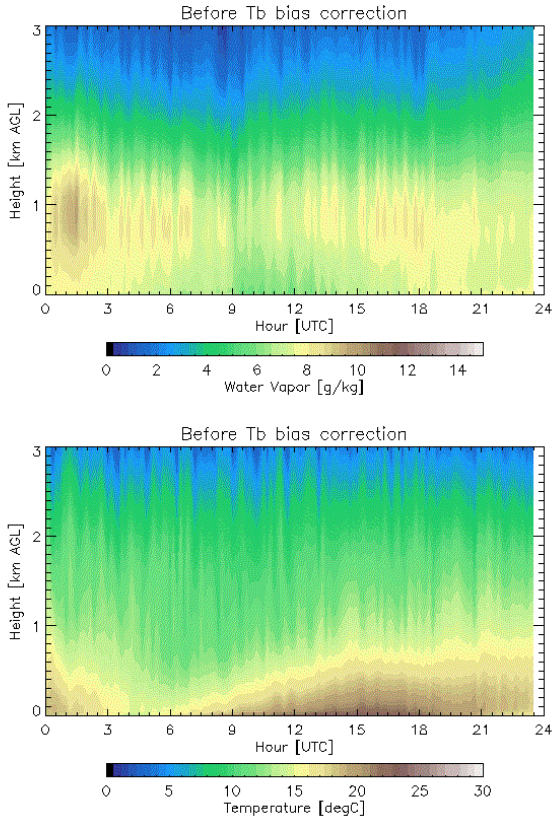

Figure 3: Water vapor (top) and temperature (bottom) retrievals before the Tb correction

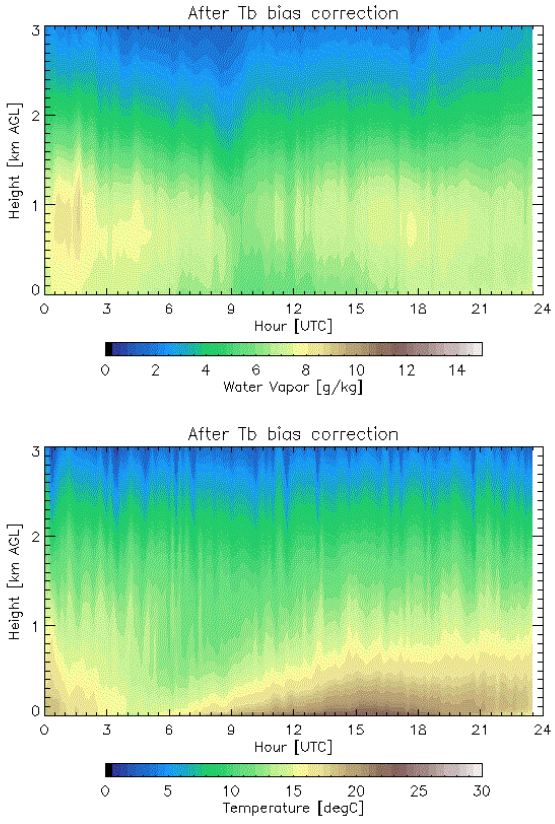

Figure 4: Water vapor (top) and temperature (bottom) retrievals after the Tb correction

The time-height retrievals at any given height after the Tb correction were applied are “smoother” with time than the results using the original uncorrected MWR observations. This can also be seen in the near-surface time-series of temperature and water vapor from the retrievals (height=0 m AGL) and the in-situ observations.

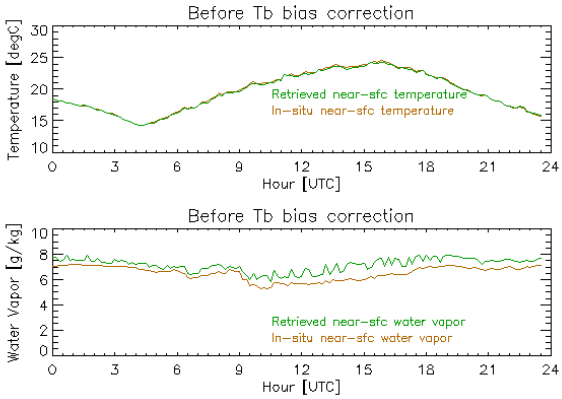

Figure 5: Near-surface retrievals before the Tb correction was applied

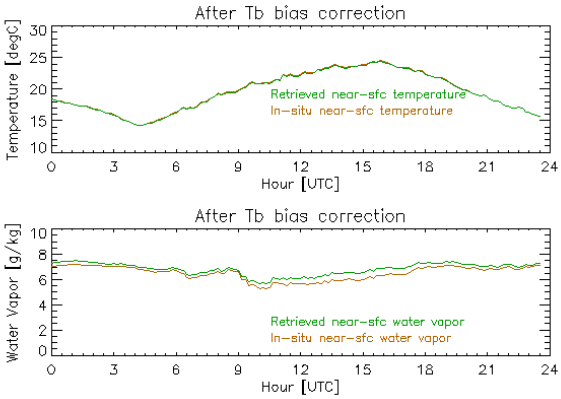

Figure 6: Near-surface retrievals after the Tb correction was applied

Again, the results using the Tb-corrected data are temporally smoother and in better agreement with the in-situ observations. The in-situ observations cannot be considered as an independent data source, as they were used in the observation vector for the retrieval. However, because the in-situ data have uncertainties, the retrieval does not have to agree perfectly with these observations, as the retrieval is trying to find the solution that best agrees with all of the observations within the constraint of the prior.

The next figures show example profiles retrieved on this day in the daytime at 15 UTC and in the early evening at 23 UTC; note the formation of a near-surface temperature inversion in the latter time. These figures show the retrieved profiles with their uncertainties, together with the prior and its uncertainty.

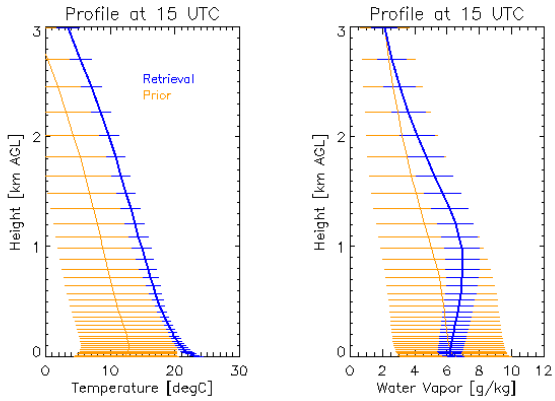

Figure 7: Retrieved profile at 15 UTC, the prior profile, and their uncertainties

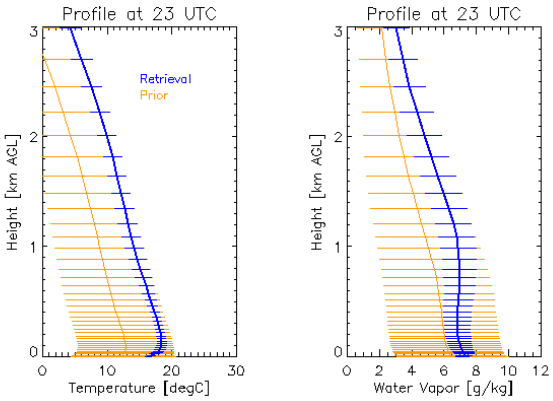

Figure 8: Retrieved profile at 23 UTC, the prior profile, and their uncertainties

Lastly, it is useful to look at the information content of the retrieval, expressed as degrees of freedom for signal (DFS). These figures show the total DFS for the temperature profile (red), water vapor profile (blue), and LWP (green) as well as how the DFS is distributed vertically for both temperature and water vapor for the 15 and 23 UTC profiles.

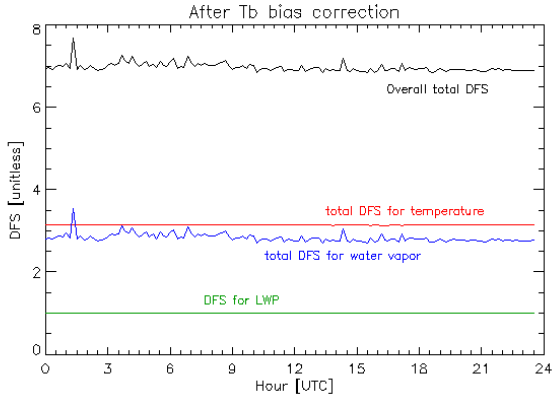

Figure 9: Time-series of the DFS for the temperature profile, water vapor profile, and LWP. The sum of these 3 components is the overall DFS.

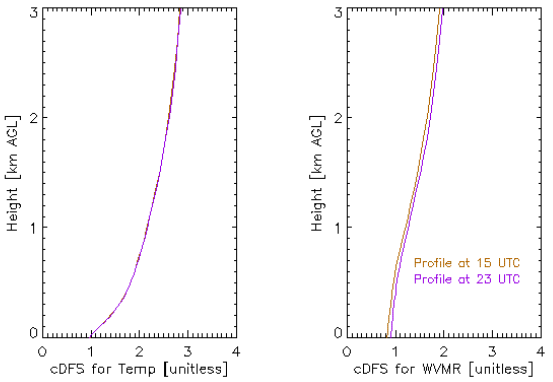

Figure 10: Profiles of the cumulative DFS for temperature (left) and water vapor (right) for profiles at 15 and 23 UTC.

As a continued illustration of the utility of the retrieval, this last figure shows several fields that were derived from the retrieved water vapor and temperature profiles. Note that the fixed PBL height for the first 5 hours and last 4 hours of the day are due to the derived PBL height being less than the threshold set in the default VIP file at 300 m. The mean uncertainty, computed over all of the samples from this day, of these fields are: 244 m for PBL height, 130 m for the surface-based lifted condensation level (LCL), and 186 m for the 100-mb mixed layer LCL.

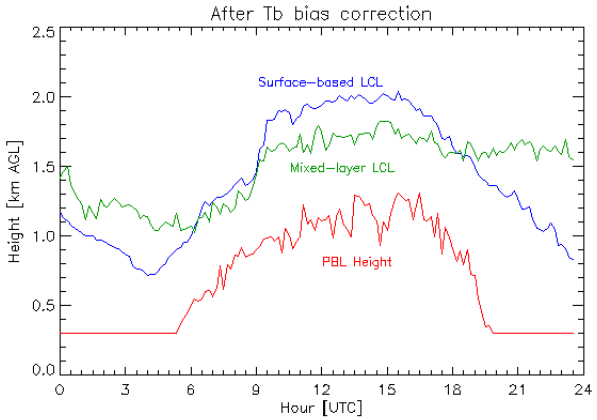

Figure 11: The derived PBL height, surface-based LCL, and mixed-layer LCL.

iii) Example 3: using the add_tropoe option

This next example also builds upon the previous one, but illustrates how the "add_tropoe" option described in Adler et al. (2024) can add information content to the retrieved profiles and impact the retrievals. This option is still considered somewhat experimental, as the results can be "tuned" by adjusting the magnitudes of the additional noise added to the previous TROPoe temperature and water vapor profiles. The VIP file used here is exactly the same as above, with these changes:

```
add_tropoe_T_input_flag = 1
add_tropoe_q_input_flag = 1
add_tropoe_T_noise_adder_val = 10,3,3
add_tropoe_q_noise_mult_val  = 10,5,5
output_rootname = tropoe.zenith_only.yes_Tb_bias_correction.add_tropoe
```

Note that we are still applying the spectral Tb bias that we determined in the previous example here.

The keywords add_tropoe_T_noise_adder_val and add_tropoe_q_noise_mult_val must be 3-element vectors of floating point numbers, with values above 1.0. The associated height fields for these two keywords are given by add_tropoe_T_noise_adder_hts and add_tropoe_q_mult_hts; these are also 3-element vectors with default values of 0.0, 1.0, and 20 km for both. These keywords are used to inflate the random error in the previous TROPoe retrieved profiles by adding (for temperature) or multiplying (for water vapor) the uncertainty in the previous retrieval by these values after they are interpolated to the vertical grid of the retrieval. The uncertainties in the previous retrieval, which is being used as input in this add_tropoe option, must be inflated in the lowest 1 km otherwise the retrieval will be too constrained and unable to capture rapid evolution of the boundary layer.

Again, we start by evaluating the rmsr and gamma fields; they are essentially unchanged from the previous run that did not use the add_tropoe option and thus not shown. However, we can

see that the temporal smoothness in the time-height cross-sections of temperature and humidity is much smoother when the add_tropoe option is turned on.

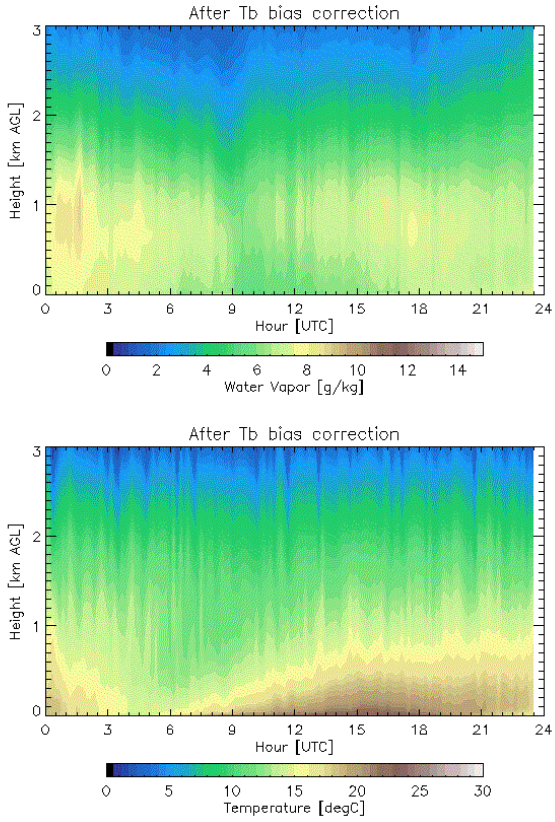

Figure 12: Time-height cross-sections of water vapor (top) and temperature (bottom) where add_tropoe is not used

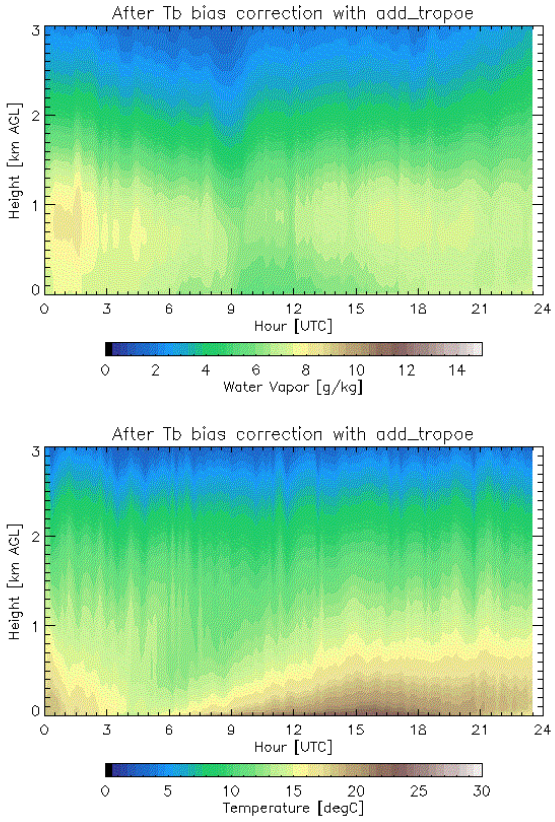

Figure 13: Time-height cross-sections of water vapor (top) and temperature (bottom) where add_tropoe is used

The impact of using the add_tropoe option is really seen in the derived uncertainty in the retrieved fields; here we only illustrate the impacts on the water vapor uncertainties.

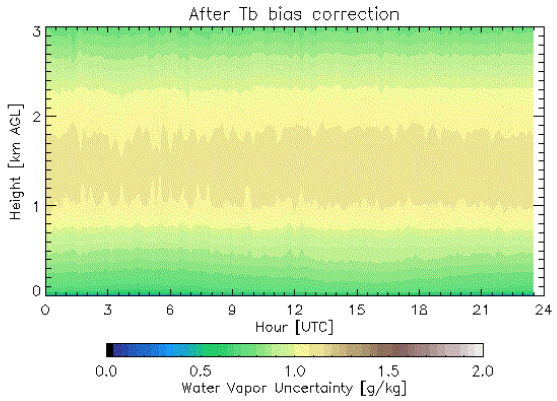

Figure 14: The time-height cross-section of the uncertainty in the retrieved water vapor when add_tropoe is not used

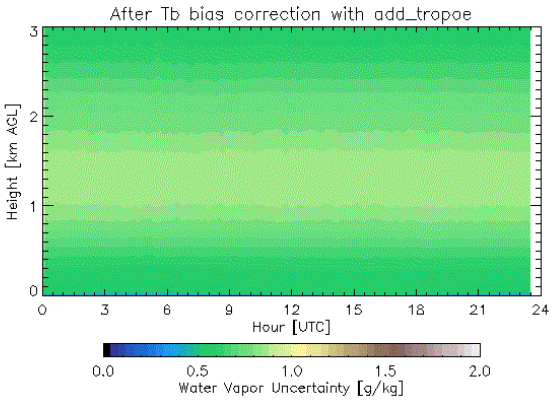

Figure 15: The time-height cross-section of the uncertainty in the retrieved water vapor when add_tropoe is used

There is a marked reduction in the uncertainties using the add_tropoe option, as seen above. The user must take care, as it is very easy to overfit the previous valid TROPoe retrieval with this option, which would prevent the true information in the observations from being utilized (e.g., when there is a rapid change in the thermodynamic structure of the PBL). We highly recommend that the add_tropoe tuning parameters "add_tropoe_T_noise_adder_val" and "add_tropoe_q_noise_mult_val" be adjusted so that the retrieval uncertainty in the thermodynamic profiles in the lowest 1 km is largely unchanged between cases where add_tropoe is not used and when it is. In this example, we see that the uncertainty in the retrieved water vapor profile is slightly decreased using add_tropoe in the lowest 500 m; as such, we should really adjust add_tropoe_q_noise_mult_val to be something like "13,7,7". (The selection of these tuning factors is somewhat subjective and may be site dependent, and thus should be used with care).

Using the add_tropoe option improves the information content of the retrieval by enforcing some temporal smoothing in the process, and we know that the atmosphere has temporal autocorrelation. This can be seen in both the temporal evolution of the DFS and in the time-series of the derived PBLH and LCL indices.

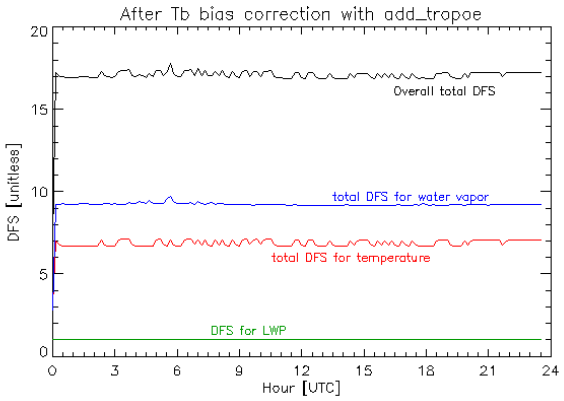

Figure 16: Time-series of the DFS for the temperature profile, water vapor profile, and LWP. This should be compared against Figure 9

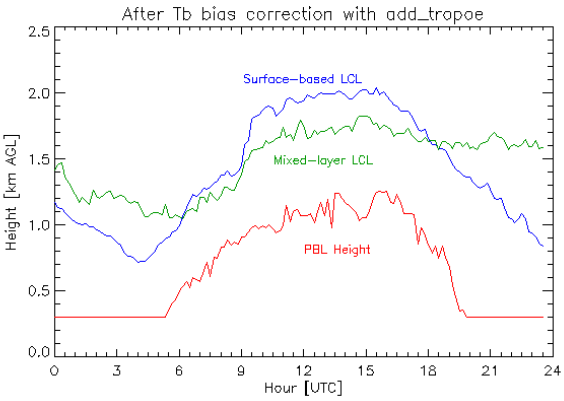

Figure 17: The time-series of the derived PBL height, surface-based LCL, and mixed-layer LCL. This should be compared against Figure 11

b) Microwave only, including elevation scan, retrievals

As discussed earlier, the two most common commercially available MWRs can easily collect data while the scene mirror scans in elevation. Adding these elevation scanning observations can greatly improve the retrieved profiles in the lowest several hundred meters near the surface (Crewell and Löhnert 2007). However, there is a big assumption: the atmospheric is horizontally homogeneous. Thus, only MWR channels that have high opacity should be used; for the MP3000 or HATPRO, we recommend only frequencies between 56 and 60 GHz be used in elevation scans. Indeed, Schween et al. (2011) have used the lower opacity water vapor observations from a scanning (elevation and azimuth) MWR to investigate water vapor gradients around their site, and Adler and Kalthoff (2014) used azimuth scans from a MWR to investigate transport processes in complex terrain.

This second use case is in the directory (*usecase02*), using the same HATPRO dataset that was used in section 9a. The VIP file is the same as what was used in section 9a-i with the bias correction from 9a-ii, with these additional modifications:

```
# MWRscan block
mwrscan_type = 3    # The TBsky data are arranged as a 2-d matrix, but in the Uni Cologne format
mwrscan_path = /data/input_data/mwr.hatpro_juelich
mwrscan_rootname     = sups_joy_mwrBL00_l1_tb_p00
mwrscan_elev_field     = ele
mwrscan_freq_field     = freq_sb
mwrscan_n_tb_fields  = 3
mwrscan_tb_field_names = tb
mwrscan_tb_freqs = 56.66, 57.3, 58.0
mwrscan_tb_noise =   0.2,  0.2,  0.2
mwrscan_tb_bias  =  0.05,  0.02, -0.04
mwrscan_n_elevations = 3
mwrscan_elevations   = 30.0, 19.2, 10.2
output_path = /data/usecase02.mwr_only.zenith_and_elevation
output_rootname = tropoe.zenith_and_elevation
```

Three things to note. First, only the 3 frequencies larger than 56 GHz are used in this retrieval. Second, the tb_noise and tb_bias values applied to those three frequencies are the same as in the vertically pointing configuration. Third, this particular MWR was only scanning along one side of the radiometer at elevation angles 5.4, 10.2, 19.2, 30.0, and 42.0 degrees. We have elected, for the sake of illustration, to only use three elevation angles in this calculation (see the keyword mwrscan_elevations above). If this radiometer was scanning on both sides of the radiometer, the elevations would be something like 5.4, 10.2, 19.2, 30.0, 42.0, 138.0, 150.0, 160.8, 169.8, and 174.6 degrees.

We will compare this run against the retrieval made in case 9a-ii, as the only difference between this run and that one is the addition of the elevation scan observations. As before, evaluate rmsr and gamma; they are essentially unchanged. We also visualize the obs_flag output here, as we've only discussed it in words before.

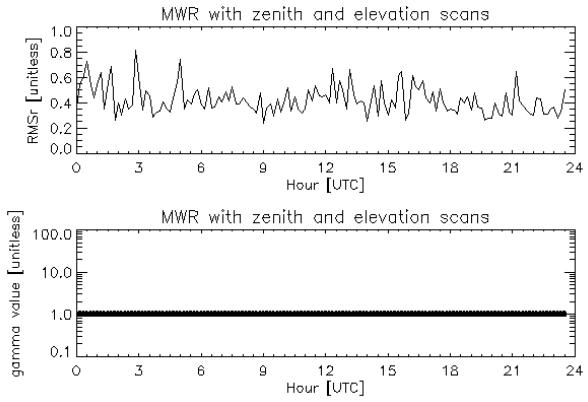

Figure 18: Time-series of the rmsr and gamma value for this retrieval

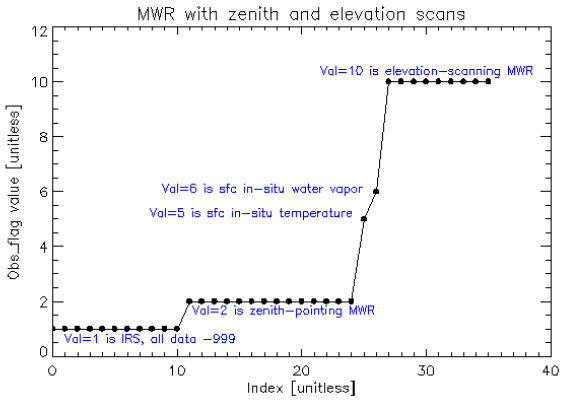

Figure 19: The obs_flag field for this example, annotated

Then we look at the spectral fit to the observations. The mean residuals for the zenith observations (obs_flag == 2) look very similar to the results for case 9a-ii. The "obs_dimension" field for the elevation scans (which are obs_flag == 10) has been encoded as each point has both a frequency and an elevation angle. The encoding is given by "frequency*1000 + elevation_angle/1000"; thus, the first three points are at 30-degree elevation angle for the frequencies 56.66, 57.30, and 58.00 GHz, the next three points are the same frequencies for the 19-degree elevation angle, and the final three points are for the 10-degree elevation angle. Note how the residuals are all within the assumed 1-σ uncertainty given by the orange lines. This demonstrates that the retrieval was able to find consistency between both the zenith and off-zenith MWR observations.

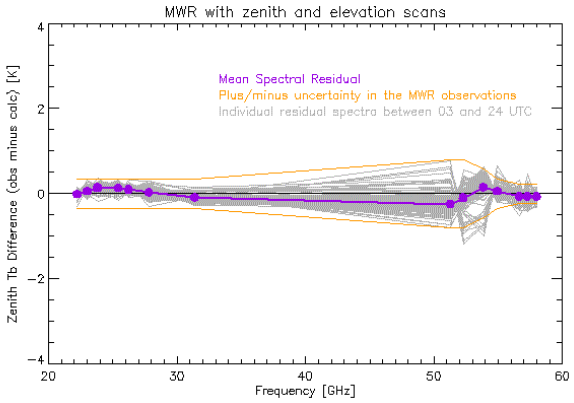

Figure 20: Spectral bias for zenith observations

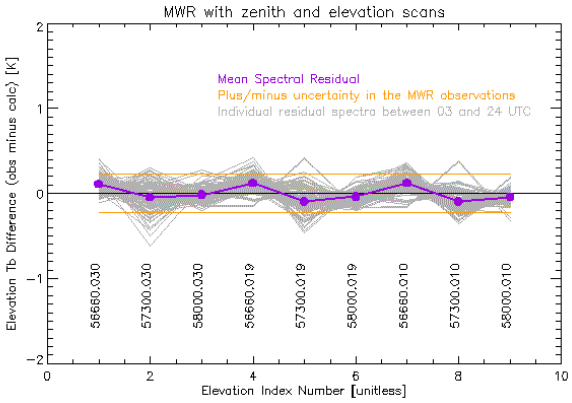

Figure 21: Bias for the elevation angle observations

The retrieved temperature profile here was slightly changed relative to the retrieval that used only the zenith radiance observations, primarily in the lowest several hundred meters of the PBL. This is most easily quantified by looking at how the uncertainty in the retrieved temperature profile changed between the zenith-only (case 9a-ii) and zenith+elevation (this case) results.

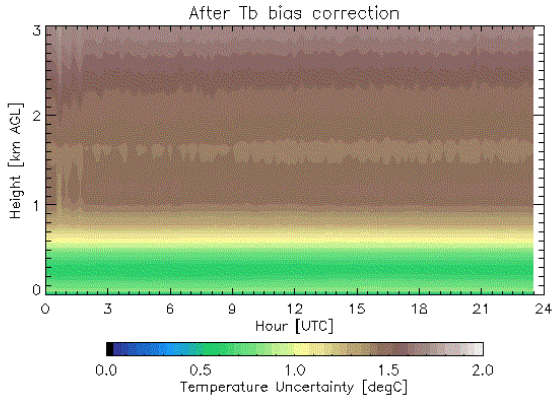

Figure 22: Time-height cross-section of the uncertainty in the retrieved temperature when using only zenith-observations (example 9a-ii)

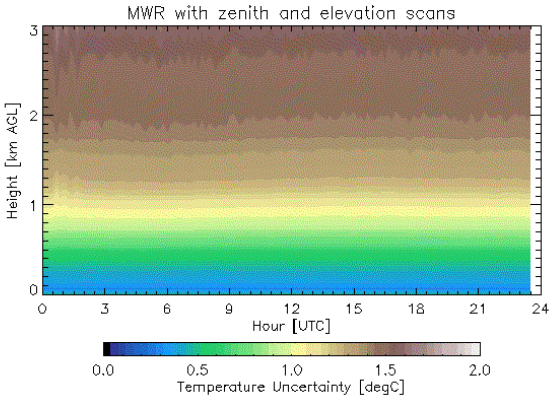

Figure 23: Time-height cross-section of the uncertainty in the retrieved temperature when elevation scanning data is added to the retrieval

This is because the addition of the elevation scans provided additional information, increasing the DFS for temperature markedly from a mean of about 3.2 for case 9a-ii (Fig 9) to a mean of approximately 4.2 here (not shown).

c) Infrared sounder retrievals

i) IRS-only retrieval example

For this retrieval (*usecase03*), the only observations used in the retrieval are the IRS observations and surface met data. As before, we will provide some details for these here, but a more complete picture can be achieved by looking at all of the TROPoe output, quicklooks, and analysis script in the directory.

The input data used here is the ASSIST infrared spectrometer that was deployed in Boulder, Colorado, USA, on 29 September 2023 as part of a multi-instrument intercomparison effort (Turner et al. 2026). This radiometer has a built-in surface meteorology station, but it is known that the observations from the in-situ temperature sensor can be biased due both to solar heating and poor air flow near the sensor. So, we will use the in-situ data as input into the retrieval, but will inflate the uncertainty in the temperature observation using the keyword "ext_sfc_temp_rep_error". For this example, we did not download the collocated ceilometer, so the code will use the default CBH. Here is the VIP file that was used:

```
# Basic information
tres              = 10
station_lat     =  39.99
station_lon     = -105.26
station_alt     = 1650.
station_psfc_min = 700

# IRS block
irs_type = 5   # This is an ASSIST
irsch1_path  = /data/input_data/irs.assist_boulder/ch1
irssum_path = /data/input_data/irs.assist_boulder/sum
irseng_path  = /data/input_data/irs.assist_boulder/sum
irs_zenith_scene_mirror_angle = 0

# Surface met block
ext_sfc_path = /data/input_data/irs.assist_boulder/sum
ext_sfc_rootname = assistsummary
ext_sfc_time_format    = 1
ext_sfc_pres_type       = 1
ext_sfc_pres_fieldname = atmosphericPressure
ext_sfc_pres_units     = 1
ext_sfc_temp_type      = 1
ext_sfc_temp_fieldname = externalTemperature
ext_sfc_temp_units     = 1
ext_sfc_temp_rep_error = 5      # Because the ASSIST in-situ sensor is known to suffer from solar heating
ext_sfc_wv_type       = 1
ext_sfc_wv_fieldname   = externalHumidity
ext_sfc_wv_units      = 1

# Ceilometer block
cbh_type = 2
cbh_path = None
cbh_default_ht = 1.0

# Output block
output_path = /data/usecase03.irs_only
output_rootname = tropoe.irs

# Global attributes
globatt_site = NOAA David Skaggs Research Center, Boulder, CO
globatt_instrument = ASSIST, unit #20
globatt_cloud_info = No ceilometer used; default cloud base height used instead
```

As usual, first check the rmsr and gamma values. While gamma has reached unity for all retrievals, there are spikes in the rmsr. However, these correspond to times when there are clouds overhead (see the LWP retrieval), suggesting that the issue is likely associated with errors in the cloud base height. (Indeed, we did have a collocated ceilometer for this event which observed the cloud base at approximately 4 km; we did not use it for this example on purpose to illustrate how a bad cloud base height estimate could be identified). Accurate cloud base height observations are very important for IRS retrievals, due to the sensitivity of the downwelling IR radiance to increasing LWP, especially when the cloud is close to the surface (e.g., Guy et al. 2022).

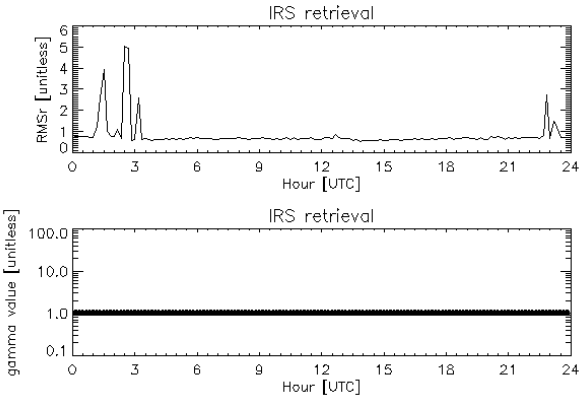

Figure 24: The rmsr and gamma time-series for this IRS retrieval

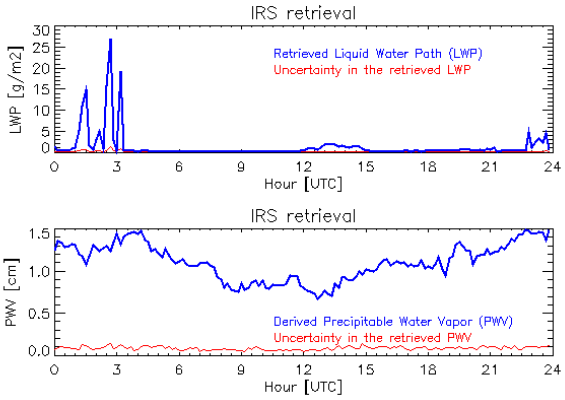

Figure 25: The retrieved LWP (top) and derived PWV (bottom), with their uncertainties in red, for this IRS retrieval

Additional information on the impact of the misplaced clouds can be seen in the uncertainties of the retrieved temperature and humidity profiles, and especially in the former. Note the temporal discontinuities at the CBH used in the retrieval (which is the default height of 1 km, as specified in the VIP, as there was no ceilometer data used).

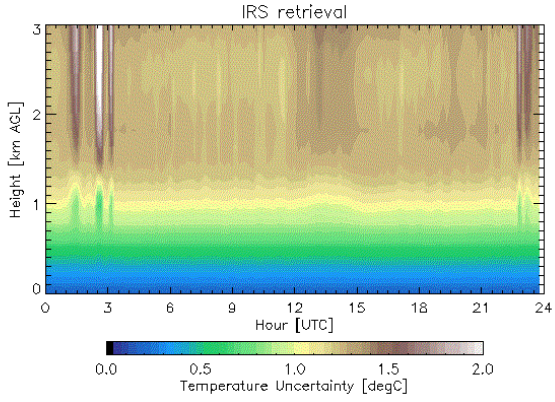

Figure 26: Time-height cross-section of the uncertainty in the retrieved temperature from the IRS instrument

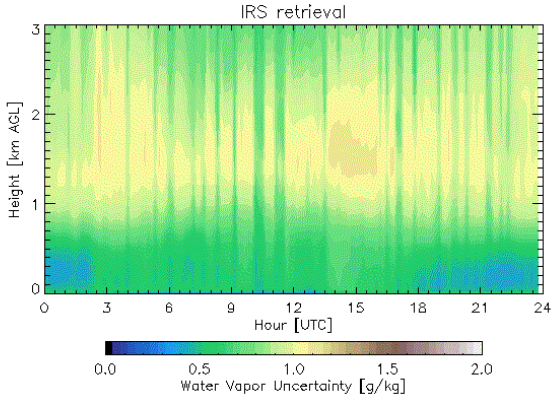

Figure 27: Time-height cross-section of the uncertainty in the retrieved water vapor from the IRS instrument

One thing to note in this IRS example is that the uncertainty in the retrieved water vapor exhibits some vertical striping. This is also seen in the retrieved water vapor field, especially above 2 km. This striping is because the retrieval at each time is treated entirely independent of each other, and thus using the add_tropoe option may help reduce this temporal artifact.

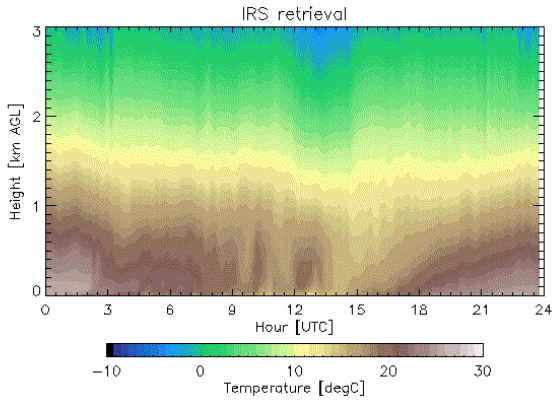

Figure 28: The retrieved temperature from the IRS data on this day

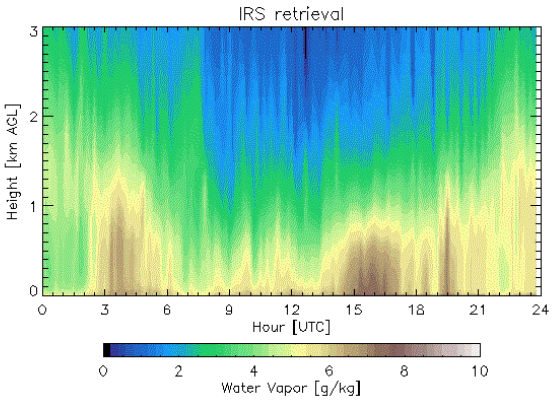

Figure 29: The retrieved water vapor from the IRS data on this day

ii) IRS with add_tropoe example

Vertical striping occurs because the retrieval at each time is treated entirely independent of each other, and thus there is no temporal correlation within the retrieved product. This is exactly why the add_tropoe option was developed. Adding these three lines to the above VIP enables this capability:

```
# Add_tropoe block
add_tropoe_T_input_flag = 1
add_tropoe_q_input_flag = 1
```

and essentially removes the striping in the temperature and water vapor cross-sections, as seen below.

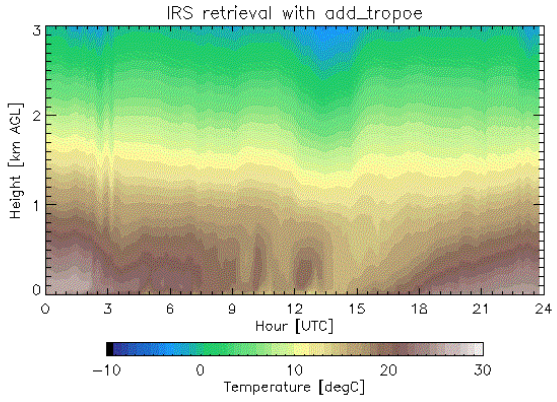

Figure 30: Temperature time-height cross-section, using IRS data with the add_tropoe option turned on. Compare this result with Fig 28, which does not use add_tropoe.

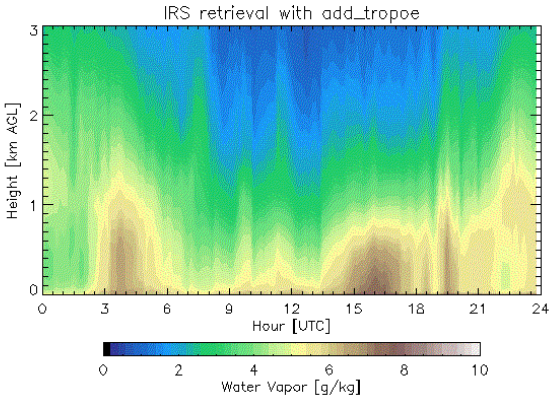

Figure 31: Water vapor time-height cross-section, using IRS data with the add_tropoe option turned on. Compare this result against Fig 29, which does not use add_tropoe.

Furthermore, there is a positive impact on the time-series of the DFS also, with the DFS increasing by approximately 2x and 4x for the temperature and water vapor profiles, respectively (compare the two figures below). One feature of the add_tropoe is that a previous profile is only used when

it passes certain quality control conditions (see section 8b above). In particular, if clouds advect over the instrument, then the retrievals in cloudy conditions are not used and a retrieval before the cloudy period is used instead. However, the add_tropoe option inflates the uncertainties of the prior retrieval as a function of the amount of time between that previous good profile and the one being processed, which decreases the impact that the previous profile has on the current retrieval. This is seen by the "scalloping" action seen in the DFS time-series between 01-04 UTC in Figure 33 below, which is when there are clouds above the IRS (see Fig 25).

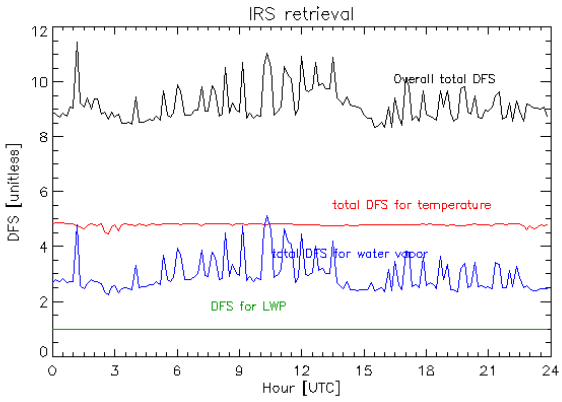

Figure 32: Time-series of the DFS for the temperature profile (red), water vapor profile (blue), LWP (green), and the sum of these (black) for the IRS case when the add_tropoe option is not enabled.

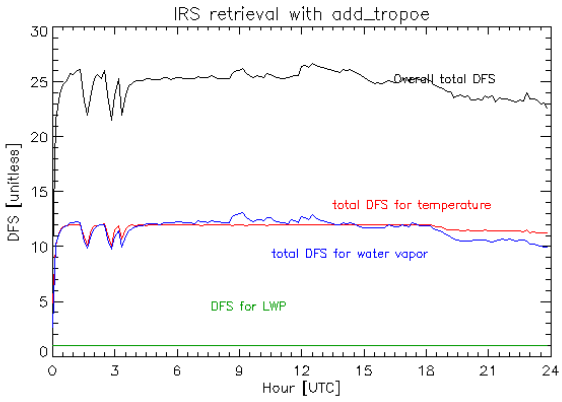

Figure 33: Time-series of the DFS for the temperature profile (red), water vapor profile (blue), LWP (green), and the sum of these (black) for the IRS case when the add_tropoe option is turned on.

d) Adding external thermodynamic profiles to the retrieval

For some applications, there may be other sources of information on the vertical profile of either temperature and/or water vapor, and you may desire to include this information into TROPoe's observation vector ***Y*** to improve the accuracy of the retrieval. Adding this information is done with the VIP keywords "ext_wv_prof_" and "ext_temp_prof_" options. These allow you to bring in data such as coincident radiosonde profiles, UAS observations, NWP model analyses or forecasts, lidar water vapor profiles, radio acoustic sounding system profiles of virtual temperature, and more.

Here, we will talk about two particular options: (i) adding ARM-like radiosondes to the retrieval and (ii) adding external profiles using a more generic option.

i) Adding radiosondes to observation vector

Each radiosonde launched by the ARM program is a single netCDF file, and there could be multiple launches (i.e., multiple netCDF files) at a given site per day. These radiosonde files are named "*sonde*yyyymmdd*.nc" and if your files are named this way, they will be found and read in. Each of these files have the following variables:

- "base_time", which is a singleton with the number of seconds since 1970-01-01 00:00:00 UTC to the first time in the file

- "time_offset", which is an array with the number of seconds since base_time for all samples in the file
- "alt", which is an array of heights [m MSL] for the ascending profile
- "pres", which is an array or pressures [mb] for the ascending profile
- "tdry", which is an array of ambient temperature [degC] for the ascending profile
- "rh", which is an array of relative humidity [%RH] for the ascending profile

where all profiles have the same dimension. To read in these data and add it to the water vapor retrieval, you need to set these keywords in the VIP file:

```
ext_wv_prof_type = 1
ext_wv_prof_path = /data/directory_to_sonde_data
```

and similarly, to add the radiosonde observation to the temperature retrieval, set these keywords:

```
ext_temp_prof_type = 1
ext_temp_prof_path = /data/directory_to_sonde_data
```

The obs_flag field in the TROPoe output file will mark these observations with a value of 3 (for temperature) and 4 (for water vapor). The other ext_wv_prof and ext_temp_prof keywords specified in Appendix 1 are used to adjust the assumed uncertainties, specify over what height ranges to use, and more.

An important point here is that the TROPoe code will take the input radiosonde profiles, assume that the sonde ascended instantaneously at the launch time, interpolate each sonde vertically to the retrieval's vertical grid, and then interpolate these data temporally to for all sample times. The temporal interpolation will not extrapolate; it will assume that the closest profile is what should be used between that launch time and the start (first launch) or end (last launch) of the day. However, the code is designed to look for radiosonde profiles on adjacent days (i.e., the day before and the day after), so that interpolation can be used to provide data for the entire day. Note that the code interpolates water vapor mixing ratio (not relative humidity), and it does a simple temporal interpolation (not a convective boundary layer aware interpolation method that was proposed by von Klitzing et al. 2026).

The uncertainty profiles associated with the radiosonde observations are estimated using a simple approach. The uncertainty in the radiosonde temperature profile is assumed constant, 0.5 degC, and independent of height. The uncertainty in the radiosonde water vapor mixing ratio is computed assuming 0.5 degC uncertainty in temperature and 3% uncertainty in the radiosonde's RH profile. These can be inflated using other VIP keywords, and inflation should be used especially in the boundary layer. However, it is important to understand that these uncertainty profiles are interpolated temporally in the same identical manner as the radiosonde data itself; i.e., that the uncertainties at a given height level only change linearly with time between two bounding launches. If you desire the uncertainty between radiosonde to change in a more dynamic way between launches (e.g., that the uncertainty is maximum mid-way between two launches), then use the generic input option below.

We demonstrate how including a collocated radiosonde profile into the retrieval with *usecase04*. In this example, we created a single ARM-like radiosonde from the RAP model analysis, as we did not have a real radiosonde collocated with the instrument on this day. We started with the VIP file used in section 9c (i.e., *usecase03*) that only included the IRS data, and made a single retrieval at 12 UTC by setting shour=12 and ehour=0. Then we updated the VIP to add these additional lines to it, thereby bringing in the simulated sonde data:

```
> # External profile block (to bring in the radiosonde)
> ext_wv_prof_type = 1
> ext_wv_prof_path = /data/input_data/irs.assist_boulder/sonde
> ext_wv_prof_min_error = 0.25
> ext_wv_noise_mult_hts = 0, 3, 20
> ext_wv_noise_mult_val = 6, 3, 3
> ext_temp_prof_type = 1
> ext_temp_prof_path = /data/input_data/irs.assist_boulder/sonde
> ext_temp_noise_adder_hts = 0, 3, 20
> ext_temp_noise_adder_val = 4, 2, 2
```

and ran the retrieval again for just the single sample time at 12 UTC.

The results demonstrate that including the simulated radiosonde into the IRS retrieval had only small impact on the retrieved temperature profile, with much of that impact being above 2 km, but with more substantial impact on the retrieved water vapor profile.

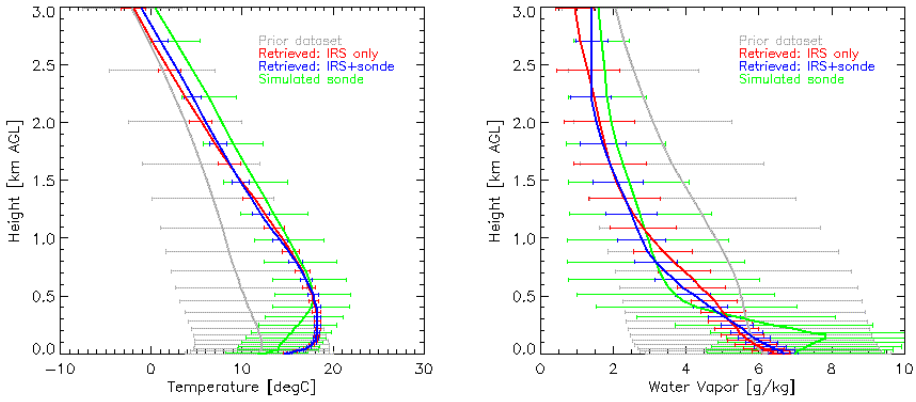

Figure 34: Demonstration of the retrieval both using (blue) and not using (red) the simulated radiosonde (green) profile.

Changing the uncertainties assumed for the radiosonde profile will change the impact that including the radiosonde has on the retrieval. Setting these uncertainties is subjective, and somewhat of an art. The user needs to consider how representative the radiosonde profile is for the conditions being viewed by the ground-based remote sensor. Also, this is just a single profile, and if the radiosonde was truly collocated then it would be natural to not use the keywords that inflate the radiosonde's uncertainties; thus, only the "prof_type" and "prof_path" keywords would be used in the VIP above. However, as the strength of the ground-based remote sensor is its ability to monitor the evolution of the atmosphere with time and their greatest signal-to-noise is in the PBL, and radiosondes are often sparse with time and drift with the wind, we recommend inflating radiosonde uncertainties in the PBL substantially (e.g., as was done here).

ii) Adding generic time-height profile data to the observation vector

This option is the most flexible, and allows the user to provide external water vapor and temperature data from any sensor in a very flexible manner. We assume that the external dataset is on a time-height grid, and that all of the height are the same for each time. The code would expect files (one or more) named "*grid*yyyymmdd*.nc", and that each of these files have the following variables:

- "base_time", which is a singleton with the number of seconds since 1970-01-01 00:00:00 UTC to the first time in the file
- "time_offset", which is an array with the number of seconds since base_time for all samples in the file
- "height", which is an array of heights [km AGL] for the ascending profile
- "temperature", which is a 2-d array of ambient temperature [degC] for the ascending profile
- "waterVapor", which is a 2-d array of water vapor mixing ratio [g/kg] for the ascending profile
- "sigma_temperature", which is a 2-d array of the uncertainty in the temperature data [degC]
- "sigma_waterVapor", which is a 2-d array of the uncertainty in the water vapor mixing ratio data [g/kg]

where all profiles have the same dimensions of time x height. We recommend that you add one or more global attributes to this gridded netCDF file when you create it that describes the dataset, as these global attributes will be copied into the TROPoe output.

To read in these gridded data and add it to the water vapor retrieval, you need to set these keywords in the VIP file:

```
ext_wv_prof_type = 7
ext_wv_prof_path = /data/directory_to_gridded_data
```

and similarly, to add the radiosonde observation to the temperature retrieval, set these keywords:

```
ext_temp_prof_type = 7
ext_temp_prof_path = /data/directory_to_gridded_data
```

The obs_flag field in the TROPoe output file will mark these observations with a value of 3 (for temperature) and 4 (for water vapor). Yes, these are the same flag values as used for radiosondes, but the field-level attributes in the output file will indicate that this is from the gridded data input and the global attributes from that file will be captured in the TROPoe output file. The other ext_wv_prof and ext_temp_prof keywords specified in Appendix 1 are used to adjust the assumed uncertainties, specify over what height ranges to use, and more.

iii) Adding NWP model output to the observation vector

You can use numerical weather prediction (NWP) model output as a constraint in your retrieval also. This can be a very reasonable addition, as many NWP modeling centers assimilate

numerous observations when initializing their models and generally these models have pretty accurate thermodynamic structure in the middle-to-upper free troposphere. If you want to use NWP model output, then it should be written into a file following the above gridded format. However, use these VIP key words to bring these data into the observation vector:

- mod_wv_prof_type = 4
- mod_wv_prof_path = /data/directory_to_gridded_data

and similarly, to add the radiosonde observation to the temperature retrieval, set these keywords:

- mod_temp_prof_type = 4
- mod_temp_prof_path = /data/directory_to_gridded_data

The obs_flag field in the TROPoe output file will mark these observations with a value of 7 (for temperature) and 8 (for water vapor).

An example of incorporating RAP model output (Benjamin et al. 2016) into the IRS retrieval is shown in *usecase04*. Here, we start with the IRS example shown in section 9c (without the add_tropoe option), and added these few lines to the VIP:

```
# Model input block
mod_temp_prof_type = 4
mod_temp_prof_path = /data/input_data/irs.assist_boulder/rap
mod_temp_noise_adder_hts = 0, 3, 20
mod_temp_noise_adder_val = 5, 1, 1
mod_wv_prof_type = 4
mod_wv_prof_path = /data/input_data/irs.assist_boulder/rap
mod_wv_noise_mult_hts = 0, 3, 20
mod_wv_noise_mult_val = 3, 1, 1
```

to read in the model datafile that was created from the actual RAP data. Note that is important that when this option is used that the input model data have the string “model” in the filename, otherwise the code will not find it. You will also notice that we are inflating the noise levels in both the model’s temperature profile (by adding noise that ramps from 5 degC at the surface to 1 degC at 3 km to 1 degC at 20 km) and water vapor profile (by multiplying the noise field by a factor of 5 at the surface to a factor of 1 at and above 3 km). This was done to prevent overfitting to the model output in the lowest several km of the PBL, where the IRS has the majority of its signal.

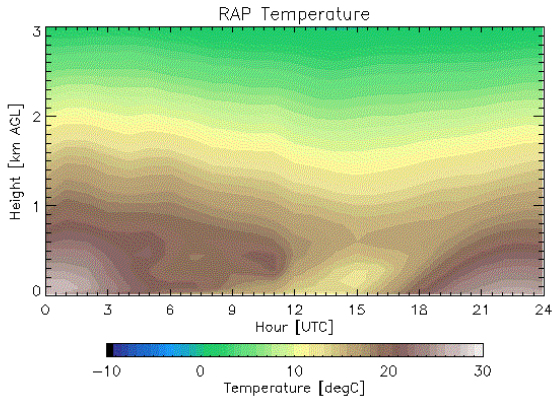

Figure 35: The time-height cross-section of temperature from the RAP at the IRS observing site that is used as input to the retrieval

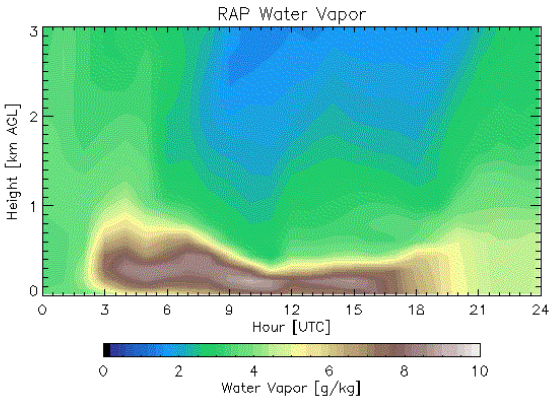

Figure 36: The time-height cross-section of water vapor from the RAP at the IRS observing site that is used as input to the retrieval

Comparing the RAP data (above) to the IRS-only retrievals (Figs 28 and 29, respectively) demonstrates that the model has similar temperature structure, but that there are some important differences in the water vapor structure in the lowest 1 km. However, when the RAP is used as input into the retrieval, using the VIP keywords listed above, these are the resulting thermodynamic cross-sections.

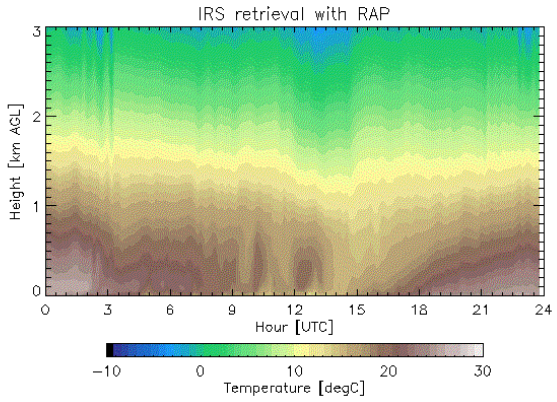

Figure 37: The temperature retrieval combining IRS and RAP data

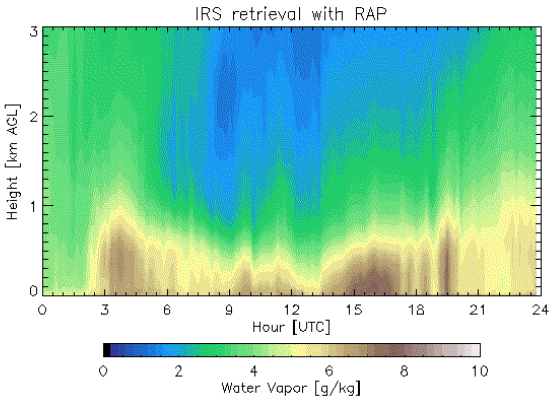

Figure 38: The water vapor retrieval combining the IRS and RAP data

The resulting retrieval combining the IRS and RAP data looks extremely similar to both the IRS-only retrievals (Figs 28 and 29) and the IRS-only that used the add_tropoe option (Figs 30 and 31).

How did the inclusion of the model data impact the DFS? The model data has the same input format as the "gridded" data (section 9d-ii). However, by using the mod_temp_* and mod_wv_* keywords (instead of the ext_temp_* and ext_wv_* keywords), we can easily see the impact on the DFS of the model data. The are two sets of DFS fields in the output file; the original ones already mentioned (e.g., "DFS", "cdfs_temperature", "cdfs_watervapor", "vres_temperature", and "vres_waterVapor") but also ones that have been computed where the model input was excluded ("DFS_no_model", "cdfs_temperature_no_model", etc). This allows the user to easily see how much information is being added, and where, to the retrieval. In the figure below, the total DFS (IRS plus model) are shown with the dotted lines, whereas the DFS without the model is shown

by the solid lines. The magnitudes of the solid lines are very similar to the DFS shown in the IRS-only retrieval (Fig 32), but with less oscillation in the water vapor DFS due to the better constraint in the middle-to-upper troposphere provided by the model input.

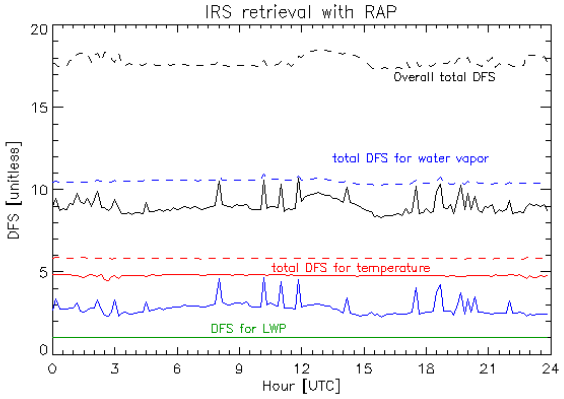

Figure 39: Time-series of the DFS for the temperature profile (red), water vapor profile (blue), and LWP (green) for both all of the observations including the model (dashed) and only the observations (solid)

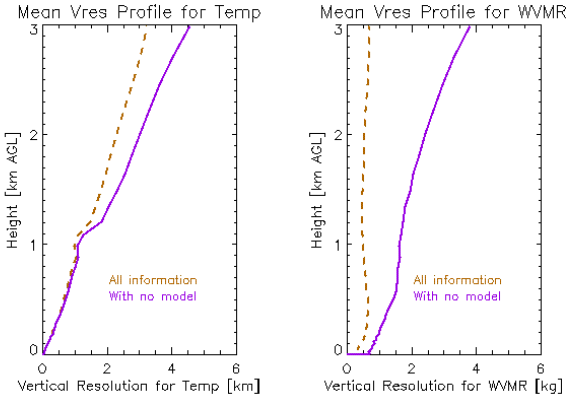

Figure 40: The mean true vertical resolution using all of the information (observations and model; dashed) and the observations only (solid)

The profiles of the vertical resolution, which are derived from the averaging kernal, provide another view of the impact by including the model output as an observation into the retrieval. These vertical resolution profiles are linked to the cumulative degrees of freedom for signal profiles (e.g., such as what was shown in Fig 10 for the MWR retrieval), with more information resulting in finer vertical resolution.

e) The observation-minus-background (OMB) option

In data assimilation studies, when a new observation type becomes available it can be quite useful to evaluate the observation against the model background to get a sense of the bias characteristics between the two; this is called an "observation minus background" (OMB) evaluation. This capability has been implemented into TROPoe, where specific thermodynamic profile can be input into TROPoe, which will only perform a single forward calculation for the MWR and/or IRS instrument and then output. This capability is extremely useful for exploring different spectral bands, evaluating new instruments, etc.

The VIP keywords that are used for this capability are

- omb_flag: 0 turns the OMB option off (i.e., normal retrieval behavior will result), 1 turns the OMB option on
- omb_path: the path to the input thermodynamic profile file
- omb_file: the name of the file with the input thermodynamic profile.
- omb_required_minht: the minimum height that the input thermodynamic profile must reach for a forward calculation to be performed

There are three types of files, all netCDF format, that can be used as input: (1) an ARM-like radiosonde file (see section 8bi for details); (2) a previously computed TROPoe output file, which

can have one or more retrieved profiles; and (3) a simple netCDF file with a single profile, with these fields and units: “height” [km AGL], “pressure” [mb], “rh” [%RH], and “temperature” [degC]. The OMB portion of the TROPoe code looks at the structure of these files to determine what sort of file to read.

Generally, the IRS or MWR observation you are “matching” has many samples in the data file. However, both the first and third option above provide only a single profile. Thus, you need to run TROPoe with the shour being exactly the time you desire, with ehour being the same time (or earlier) so that only a single forward calculation is performed. (For example, if you have a sonde that was launched at 04:45 UTC, then you should set both shour and ehour to 4.75). For option 2 (i.e., when a TROPoe file is used), you can process the entire day (i.e., shour=0 and ehour=24) but the code may produce weird results if the input IRS/MWR data is processed at a different temporal resolution than what was used in the original processing the produced the input TROPoe file.

Why might I be interested in using the OMB option? Here we provide two examples. The first is that I’ve just deployed my IRS or MWR to a new location, and launched a radiosonde in clear sky conditions that I would like to compare to my radiometer. This is trivially done using the OMB option by formatting the radiosonde data as either (1) or (3) above. The second example is illustrated for an IRS application, but is also possible for a MWR application. For IRS retrievals, we only use a portion of the infrared spectrum in the typical and recommended configuration. However, I may be interested to see how well the retrieved thermodynamic profile fits the entire IRS observed spectrum. In this case, I run the retrieval using the baseline configuration to get the TROPoe file that will be used as input, change the spectral_bands keyword in the VIP to cover the larger spectral range in an OMB model.

As an illustration to this second example, as set up *usecase06.omb*. Here is the input VIP:

```
# Basic information
tres          = 10
station_lat     =  39.99
station_lon     = -105.26
station_alt     = 1650.
station_psfc_min = 700

# IRS block
irs_type = 5   # This is an ASSIST
irsch1_path = /data/input_data/irs.assist_boulder/ch1
irssum_path = /data/input_data/irs.assist_boulder/sum
irseng_path = /data/input_data/irs.assist_boulder/sum
irs_zenith_scene_mirror_angle = 0

# Observation-minus-background (OMB) block
omb_flag = 1
omb_path = /data/usecase03.irs_only
omb_file = tropoe.irs.20230929.000005.nc.orig

# Spectral bands that I want computed
spectral_bands = 560-660,660-1500
```

```
# I need to use the other TAPE3 that has a more extensive spectral range
lbl_tape3 = TAPE3.10-3500cm-1.first_7_molecules

# Output block
output_path = /data/usecase06.omb
output_rootname = tropoe.irs_omb
```

Note the new "omb" block. In particular, we will read in the atmospheric state from a previous TROPoe retrieval that was performed for *usecase03*. But here we will compute the downwelling infrared radiance from 560-1500 cm^{-1}; one "feature" of the spectral_bands keyword is that it always needs at least two spectral bands specified.

Here, we show the difference in the observed and calculated spectra over this wide spectral band, and overlay the differences from the spectral regions used in the actual retrieval. An important point is that the information added to the prior via the VIP file, namely the cloud and trace gas information, will be used in the forward calculation. By default, the cloud liquid water path (LWP) and ice cloud optical depth (iTau) are set to zero in the VIP. The first OMB calculation, in the top panel, shows the comparison for the forward calculation using these default values which essentially indicate that the sky is cloud free. The agreement between the OMB full-spectrum residuals and the residuals in the retrieval is very poor. This is because there was actually an optically thin cloud detected above the IRS, and TROPoe retrieved a LWP of 0.393 g/m^2 with an effective radius of 8.554 μm. We then added this information to the VIP by including these lines:

```
# These are the cloud values for the 15 UTC retrieval
prior_lwp_mn   = 0.393
prior_lReff_mn = 8.554
```

and reran the OMB calculation, yielding the bottom panel which has much better agreement between the OMB residuals and the residuals from the actual retrieval.

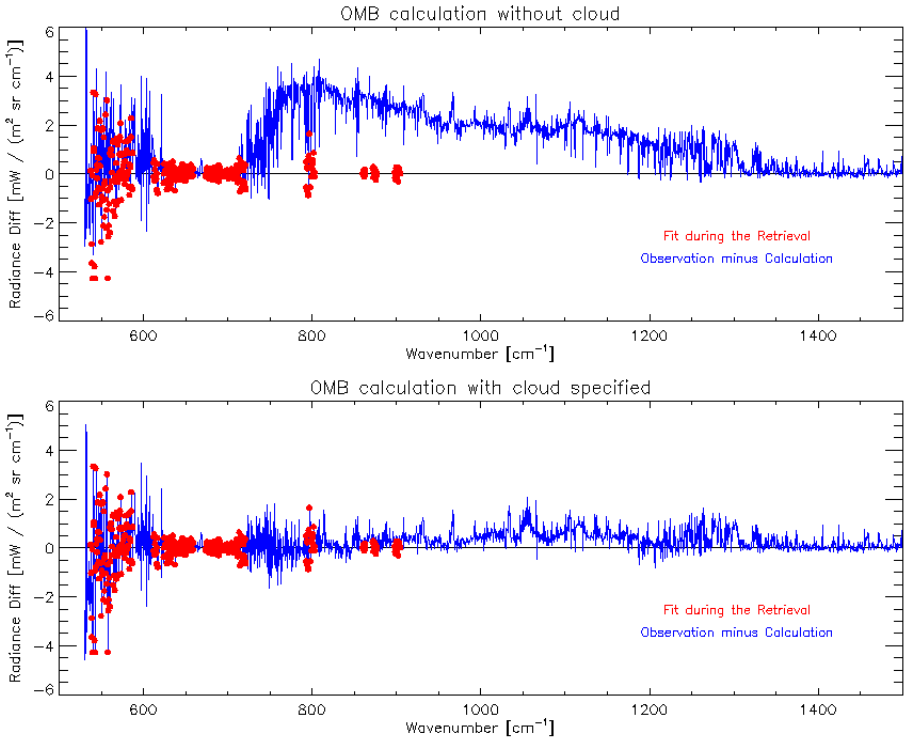

Figure 41: Observed minus computed infrared radiance, where the blue spectrum used a previous retrieval in an OMB calculation whereas the red was the residuals after the retrieval. The top panel shows the result when no cloud information was used in the OMB calculation even though the retrieval retrieved non-zero cloud properties. The bottom panel shows the OMB calculation using those cloud properties as input.

This OMB option provides a lot of power for the user. We have used this option to evaluate biases in MWR observations when we've had collocated radiosonde data, to emulate behavior of an imaginary instrument (e.g., a hyperspectral MWR with channels covering the 118 GHz oxygen and 183 water vapor absorption lines), and to perform sensitivity calculations for changes in cloud properties.

f) Processing other datasets

TROPoe is essentially 1-d data assimilation. Ultimately, it is trying to find the solution that best matches the input observations and the a priori climatology within their uncertainties. There are applications where it may be desirable to run TROPoe without having either IRS or MWR input, such that thermodynamic profiles can be generated with both uncertainties and information content in a consistent manner. One example is how the DOE ARM program blend radiosonde, surface met, and model output in their merged sounding product (Troyan 2012). Another example is to merge RASS-observed virtual temperature profiles with surface met data to create a consistent product. A third example is to post-process the ARM Raman lidar (Turner and Goldsmith 1999), which has two observing channels (the wide field-of-view and narrow field-of-view) to blend them together more seamlessly.

If the user desires to use TROPoe in this manner, the easiest way to do so is to create a fictious MWR dataset with a single channel and use the MWR as the master instrument (i.e., irs_type=-

1), and assume the noise level in this single channel is very large. In our applications, we have created a fictious input MWR file with the brightness temperature at 31.0 GHz of 20 K, and assumed the uncertainty in this channel to be 200 K. The rest of the input data that are being merged can be included via the other keywords (e.g., the ext_ or mod_ blocks). Please consult with the TROPoe developers if you want to use TROPoe for an application like this.

10) Spectral bias correction procedure for the MWR

We have evaluated the spectral bias seen between MWR observations and its forward calculation for a large number of different microwave radiometers. One of the features we have observed regularly is large changes in the spectral bias for adjacent channels (i.e., channels that are close to each other in frequency); this cannot be explained by any atmospheric structure and is indicative of either a bias in the MWR observation or in the forward model. The use of the VIP entry "mwr_tb_bias" allows a first-order correction for either the observation and/or the model.

Identifying that a spectral bias exists is straightforward; you only need to follow the example in section 9a-i. However, to determine the correction that should be applied is much more difficult. In section 9a-ii, we applied the negative of the bias determined in the previous section as the mwr_tb_bias; however, this is a circular correction strategy as the retrieval is being used to determine the spectral bias to use in the retrieval. The best solution is to use an independent observation that you can trust as the "truth" such as a collocated radiosonde, and compute the spectral bias using the OMB method. Several papers have explored how to best determine the spectral bias (e.g., Djalalova et al. 2022; Löhnert and Maier 2012).

11) Built-in Quicklook Plots

For monitoring the retrieval, or easy plotting of TROPoe output, we have created simple quicklook plots that can be configured to run either real-time as the retrieval is being processed or after the retrieval is completed. These quicklook plots are controlled using the plotting block in the VIP file.

- plot_output – a flag that specifies if plots are desired
  - 0 – do not generate quicklook plots (default)
  - 1 – generate a new quicklook plot after every retrieval time
  - 2 – generate plot from pre-existing file without running retrieval.
- plot_path – path to store outputted quicklooks. If not set, then it defaults to the output directory for the netCDF files.
- plot_rootname – prefix for the quicklook plots. If not set, then it defaults to the netCDF file rootname

There are additional VIP options that control the x-axis, y-axis, and colormap limits for all of the plots; these can be found in Appendix 1. If the plot_output option is used, two quicklook plots will be created for each day processed. The first plot contains time-height plots of temperature, water vapor, temperature uncertainty, water vapor uncertainty, potential temperature, equivalent potential temperature, relative humidity and dewpoint. An example of this plot is shown below.

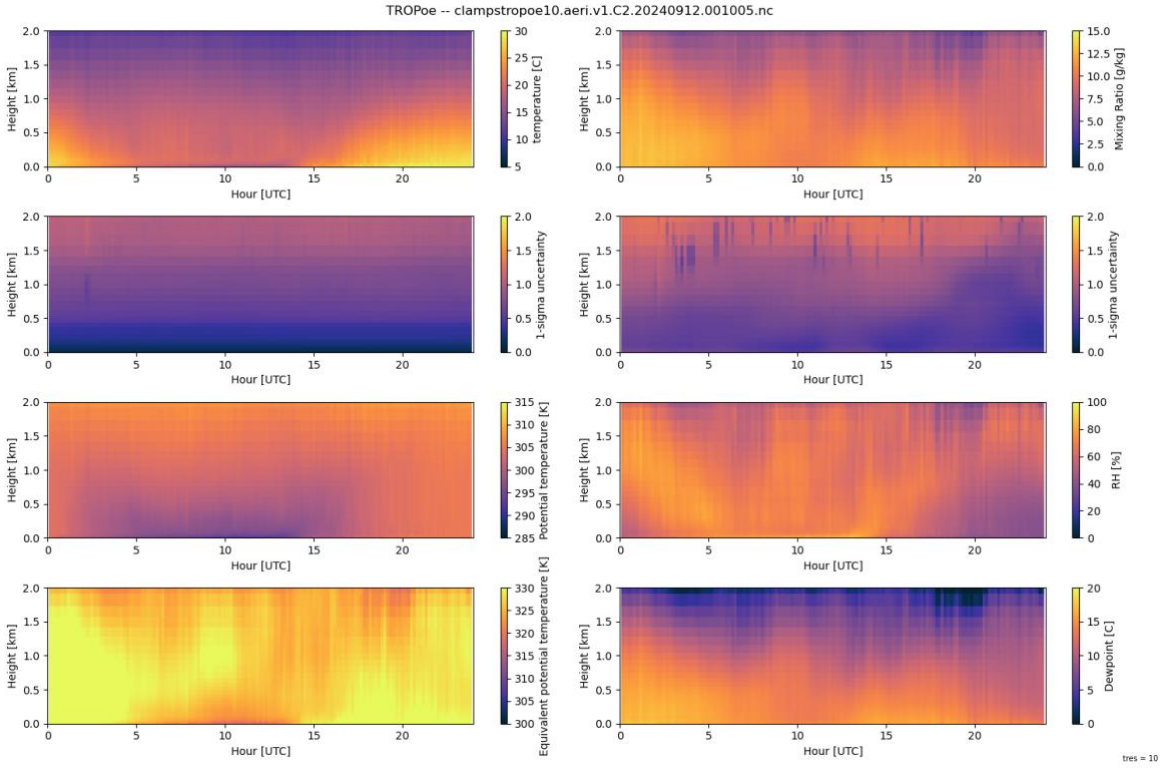

Figure 42: Example of the built-in time-height plots available in TROPoe.

While not shown in the example plot above because this was a clear-sky day, cloud base height is overlaid on the plot when the retrieved liquid water path exceeds a threshold value (the default is 8 g m^{-2}). The second plot that is created are timeseries plots that show RMSr, PBL height, stable PBL height, LCL, MLLCL, CAPE, MLCAPE, CIN and MLCIN.

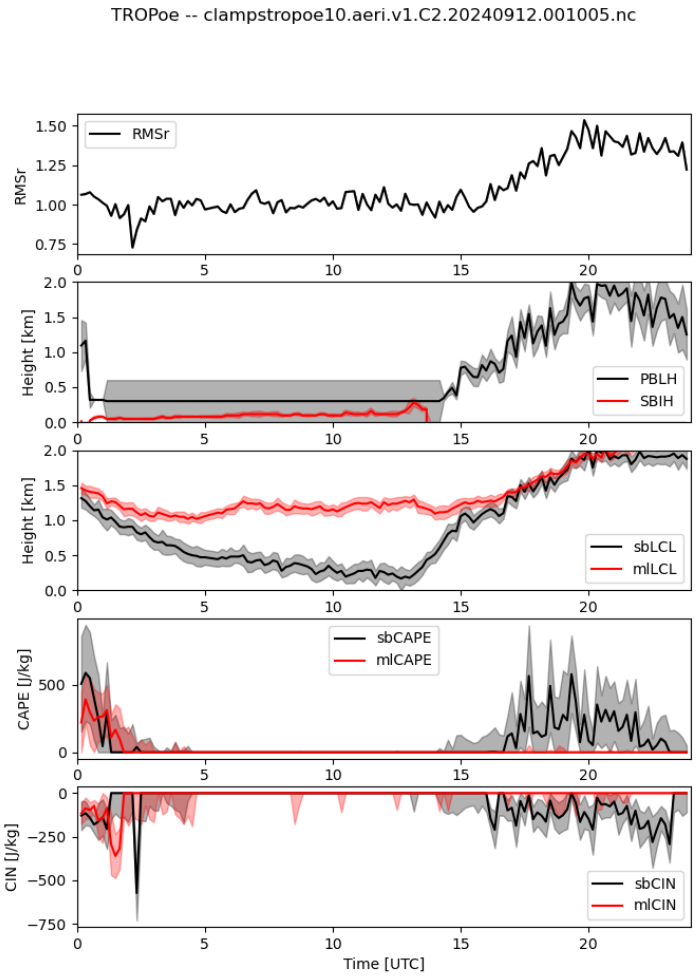

Figure 43: Example of the built-in timeseries plots available in TROPoe. The 1-sigma uncertainty of the variables are shown by the shading.

When plot_option is set to 2, the code will look for an output file that matches the date used in the command to run TROPoe and make quicklook plots if an output file is found. The code will than exit without running the retrieval. This option is useful, if you are not regularly running quicklooks, but need an easy way to view the data for a specific day, or if you want to adjust the limits used when the original quicklook images were made (e.g., to increase the height range of the time-height cross-sections).

12) TROPoe code flowchart

This section provides a high-level overview of the TROPoe software package using flowcharts. The first figure shows the overall algorithm, whereas the next two figures are zooms into particular functions.

The best documentation for any compute code is the actual code itself. We have added many comments throughout all of the functions used by TROPoe. The TROPoe software can be found on GitHub at https://github.com/OAR-atmospheric-observations/TROPoe.

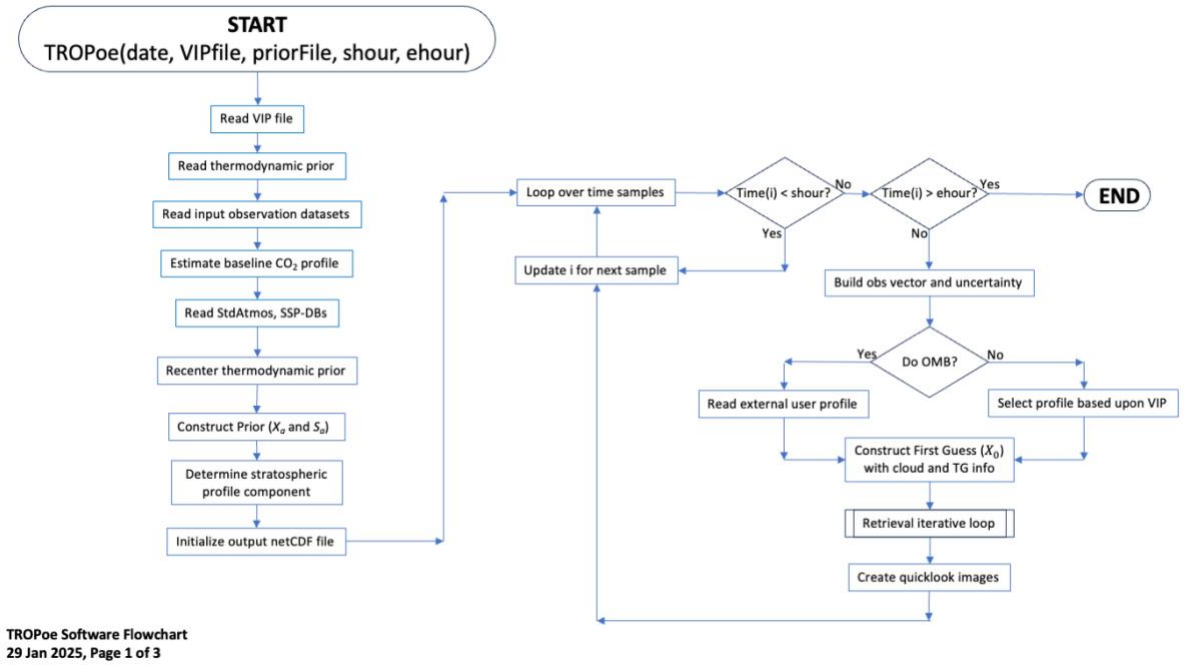

Figure 44 - Overall TROPoe software flow chart. The logic in the "retrieval iterative loop" is shown in the next figure.

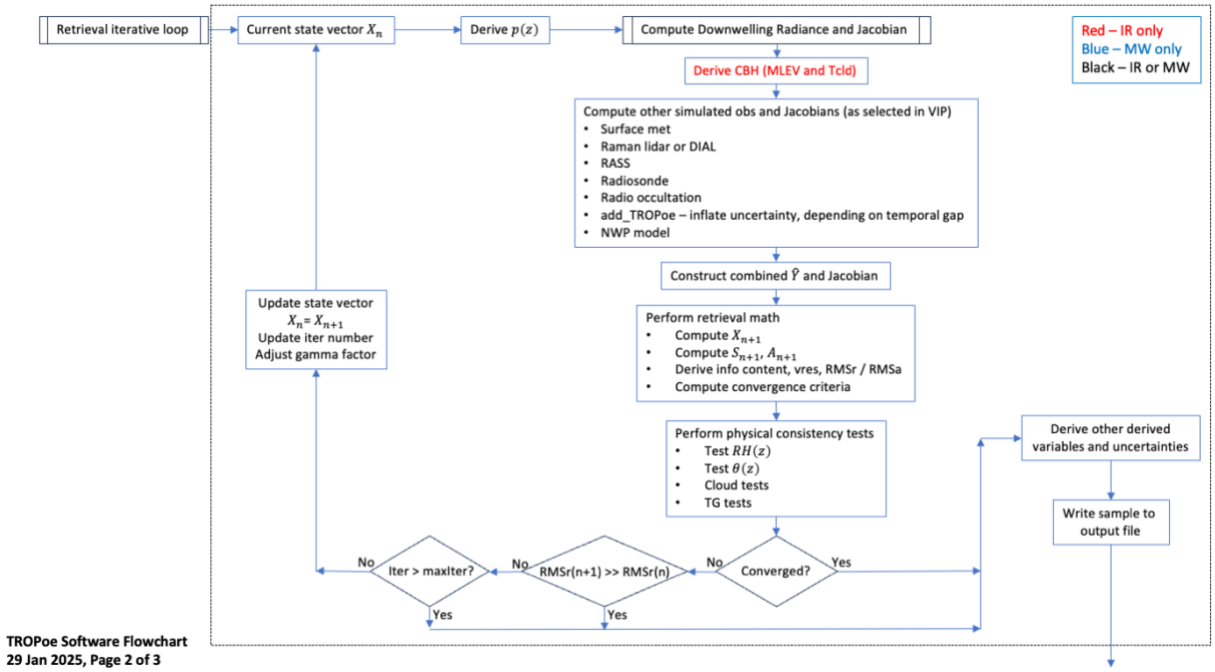

Figure 45 - Flow chart for the "iteration loop". The "compute downwelling radiance and Jacobian" logic is shown in the next figure.

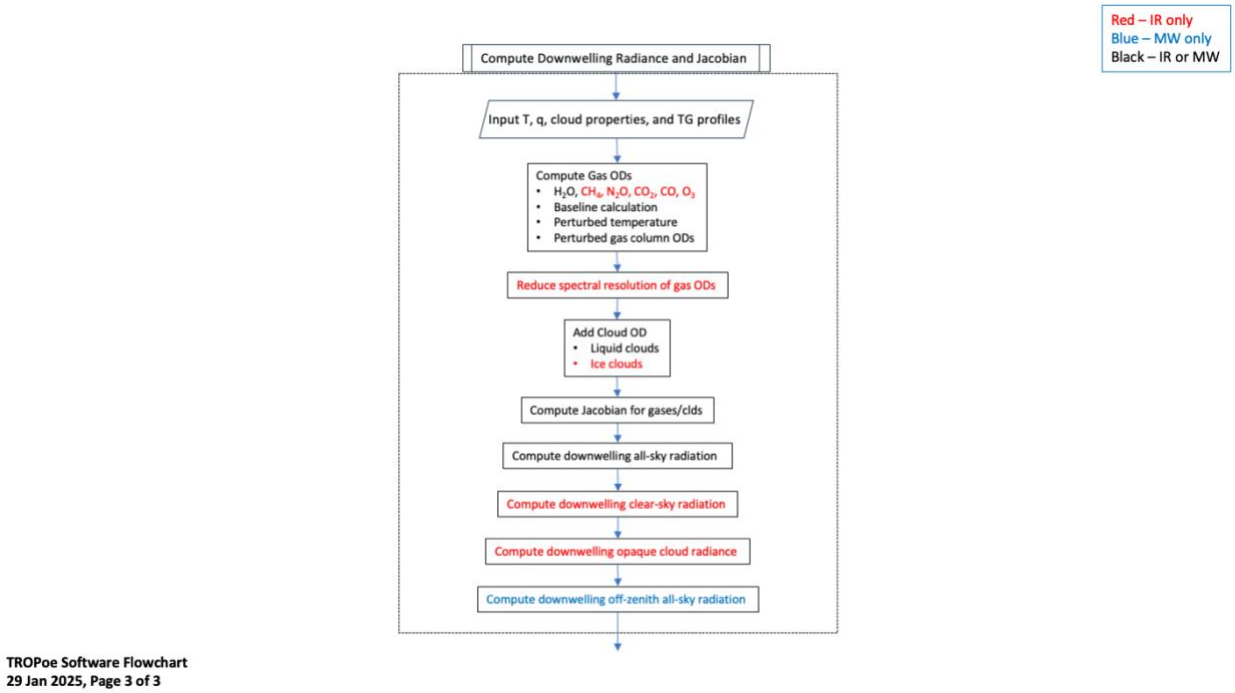

Figure 46 -- Flow chart for the logic that "computes the downwelling radiance and Jacobian"

13) Appendix 1: VIP Entries

These provide the standard VIP entries that are most commonly used. We will interleave comments in *blue italics* for some of these to provide additional context. Entries that are already explained in the above Users Guide, or are considered self-explanatory, have no additional comments. Naturally, any questions regarding any of these parameters should be directed to the authors. Note that the values assigned have been chosen as the defaults to encourage a baseline behavior of the algorithm. If the default value of the key is an integer, then any user-specified value (via the VIP) must also be an integer. Ditto for floating point values.

a) Standard VIP key-value pairs

tres = 0 # Temporal resolution [min], 0 implies maximum (native) temporal resolution

avg_instant = 1 # A flag to specify -1:avg with no sqrt(N), 0:avg with sqrt(N), or 1:instantaneous

We highly recommend that avg_instant always be 1 for IRS retrievals; averaging radiance from clear and cloudy scenes over some time window is very difficult to deconvolve by the retrieval due to the non-linear response of infrared radiance to changing cloud conditions

zgrid = 0.000, 0.010, 0.021, 0.033, 0.046, 0.061, 0.077, 0.095, 0.114, 0.136, 0.159, 0.185, 0.214, 0.245, 0.280, 0.318, 0.359, 0.405, 0.456, 0.512, 0.573, 0.640, 0.714, 0.795, 0.885, 0.983, 1.092, 1.211, 1.342, 1.486, 1.645, 1.819, 2.011, 2.223, 2.455, 2.710, 2.991, 3.300, 3.640, 4.014, 4.426, 4.879, 5.376, 5.924, 6.526, 7.189, 7.918, 8.720, 9.602,10.572,11.639,12.813,14.104,15.525,17.087 # Comma delimited list of heights for the z-grid to use in the retrieval [km]

The zgrid is the height grid that is used for the retrieval. It must start at 0 km, and be monotonically increasing up to at least the tropopause. We recommend a geometric grid like the default grid used here, where each subsequent layer is (for example) 10% thicker than the previous layer.

station_lat = -999.0 # Station latitude [degN]

station_lon = -999.0 # Station longitude [degE]

station_alt = -999.0 # Station altitude [m MSL]

This value must be non-negative. We have it default to a negative value to force you to actually enter station information into the VIP!

station_pres = 1000.0 # Station pressure [mb]; will be only be used if there is no other surface pressure input

station_psfc_min = 800.0 # Default minimum surface pressure [mb]

station_psfc_max = 1040.0 # Default maximum surface pressure [mb]

Please set these station lat/lon/alt fields – it is good practice!

omb_flag = 0 # Flag to perform single forward calculation using input file; useful for o-minus-b (O-B) calculations (0 - off, 1 - on)

omb_path = /data # Path to the input NetCDF file to use for the O-B calculation

omb_file = omb_input.nc # NetCDF file with profiles of temperature [C], rh [%], pressure [mb], and height [km AGL] to use in the forward O-B calculation

omb_required_minht = 10.0 # Minimum height [km AGL] required for the input profiles used in the forward O-B calculation

See the "observation minus background" section for details for these OMB key-values

irs_type = 0 # 0- output, options, and stop, 1 - ARM AERI data, 2 - dmv2cdf AERI data (C1_rnc.cdf and _sum.cdf), 3 - dmv2ncdf (C1.RNC.cdf and .SUM.cdf), 5- ASSIST, -1 - MWR data is to be used as MASTER dataset (no IRS data being read in)
irsch1_path = /data/aerich1 # Path to the IRS ch1 radiance files
irsch2_path = /data/aerich2 # Path to the IRS ch2 radiance files
irssum_path = /data/aerisum # Path to the IRS summary files
irseng_path = /data/aerieng # Path to the IRS engineering files
irs_zenith_scene_mirror_angle = 180 # SceneMirrorAngle [deg] to use for zenith views (default is 180)

The ASSIST uses 0 degrees for its zenith view, and so this must be reset for ASSIST data

irs_min_noise_flag = 1 # If non-zero, then the irs_min_noise_spectrum will be used as a floor; otherwise, will use input IRS noise spectrum from instrument
irs_min_noise_wnum = 500,522,546,575,600,631,747,1439,1770,1884,2217,3000 # Wavenumber array [cm-1] for the minimum noise spectrum
irs_min_noise_spec = 65.309,14.056,3.283,1.333,0.813,0.557,0.304,0.581,0.822,0.025,0.023,0.044 # Noise array [RU] for the minimum noise spectrum
irs_min_675_bt = 263.0 # Minimum brightness temp [K] in the 675-680 cm-1 window -- this is QC screen
irs_max_675_bt = 313.0 # Maximum brightness temp [K] in the 675-680 cm-1 window -- this is QC screen

We have demonstrated that it is easy to overfit IRS data (Adler et al. 2024). Thus, we have developed a minimum noise spectrum that should be used in an IRS-based retrievals. This results in the retrieval using the larger of either the noise spectrum in the IRS summary file or this spectrum. This can be turned off by setting the keyword irs_min_noise_flag = 0

mwr_type = 0 # 0 - none, 1 - Tb fields are individual time series, 2 - Tb field is 2-d array
mwr_path = /data/mwr # Path to the MWR data
mwr_rootname = mwr # Rootname of the MWR data files
mwr_elev_field = elev # Name of the scene mirror elevation field; use 'none' if all data are zenith
mwr_freq_field = freq # Name of the frequency field needed if mwr_type >= 2
mwr_n_tb_fields = 0 # Number of fields to read in
mwr_tb_field_names = tbsky23,tbsky31 # Comma separated list of field names for the Tb fields
mwr_tb_field1_tbmax = 100.0 # Maximum value [K] in the first Tb field, used for QC
mwr_tb_freqs = 23.8,31.4 # Comma separated list of frequency [GHz] of MWR Tb fields
mwr_tb_noise = 0.3,0.3 # Comma separated list of noise levels [K] in the MWR Tb fields
mwr_tb_bias = 0.0,0.0 # Comma separated list of bias [K] in the MWR Tb fields; this value is ADDED to the MWR observations

mwr_tb_replicate = 1 # Number of times to replicate the obs -- this allows us to change the weight of the MWR data in the retrieval (useful for LWP)
mwr_time_delta = 0.083 # The maximum amount of time [hours] that the MWR zenith obs must be to the sampling time to be used

mwrscan_type = 0 # 0 - none, 1 - Tb fields are individual time series, 2 - Tb field is 2-d array
mwrscan_path = None # Path to the MWRscan data
mwrscan_rootname = mwr # Rootname of the MWRscan data files
mwrscan_freq_field = freq # Name of the frequency field needed if mwrscan_type >= 2
mwrscan_elev_field = elev # Name of the scene mirror elevation field; this field must exist
mwrscan_n_tb_fields = 0 # Number of fields to read in
mwrscan_n_elevations = 2 # The number of elevations to use in retrieval (put zenith obs in 'mwr_type')
mwrscan_elevations = 20,160 # The elevation angles to use in deg, where 90 is zenith. The code will look for these obs within the averaging interval
mwrscan_tb_field1_tbmax = 330.0 # Maximum value [K] in the first Tb field, used for QC
mwrscan_tb_field_names = tbsky23,tbsky31 # Comma separated list of field names for the Tb fields
mwrscan_tb_freqs = 23.8,31.4 # Comma separated list of frequency [GHz] of MWR
mwrscan_tb_noise = 0.3,0.3 # Comma separated list of noise levels [K] in the MWR Tb fields
mwrscan_tb_bias = 0.0,0.0 # Comma separated list of bias [K] in the MWR Tb fields; this value is ADDED to the MWR observations
mwrscan_time_delta = 0.25 # The maximum amount of time [hours] that the elevation scan must be to the sampling time to be used.

We highly recommend that the MWRscan angles do not include the 90 degree view (which is assumed to be zenith), as that will already be included in the MWR block. Also, many MWRs that are performing elevation scans do these scans at regular intervals. Thus, the choice of the "tres" keyword should be selected so that there is at least one of these elevation scans in each tres window.

ext_sfc_temp_type = 0 # External surface temperature met data type: 0-none, 1-external met data
ext_sfc_temp_fieldname = temp_mean # Name of surface temperature met field
ext_sfc_temp_units = 1 # Units of surface temperature met field: 1-degC, 2-degK
ext_sfc_temp_npts = 1 # Number of surface temperature met points to use in the retrieval. Minimum=1, maximum=1000. Larger number increases the weight of the observation
ext_sfc_temp_random_error = 0.5 # Random error for the surface temperature measurement [degC]
ext_sfc_temp_rep_error = 0.0 # Representativeness error for the surface temperature measurement [degC], which is added to the random error
ext_sfc_wv_type = 0 # External surface water vapor met data type: 0-none, 1-external met data
ext_sfc_wv_fieldname = rh_mean # Name of surface water vapor met field
ext_sfc_wv_units = 1 # Units of surface water vapor met field: 1-percent_rh, 2-fraction_rh

ext_sfc_wv_npts = 1 # Number of surface water vapor met points to use in the retrieval. Minimum=1, maximum=1000. Larger number increases the weight of the observation

ext_sfc_rh_random_error = 3.0 # Random error for the surface relative humidity measurement [%], which is used with the ext_sfc_random_temp_error to get the uncertainty in WVMR

ext_sfc_wv_mult_error = 1.0 # Multiplier for the error in the surface water vapor mixing ratio measurement. This is applied BEFORE the 'rep_error' value

ext_sfc_wv_rep_error = 0.0 # Representativeness error for the surface water vapor measurement [g/kg], which is added to the assumed random uncertainty of 0.5 degC and 3%RH

ext_sfc_pres_type = 0 # 0 - Use the internal {IRS,MWR} pressure sensor for psfc; 1-external surface met data

ext_sfc_pres_fieldname = atmos_pressure # Name of surface pressure met field

ext_sfc_pres_units = 1 # Units of surface pressure field: 1-kPa, 2-hPa, 3-Pa

ext_sfc_path = /data/met # Path to the external surface met data

ext_sfc_rootname = met # Rootname of the external surface met datastream

ext_sfc_time_format = 0 # Time format for external surface met data: 0-base_time/{time_offset,time} [epoch seconds], 1-same as option 0 but base_time is in miiseconds (ASSIST), 2-time [epoch seconds]

ext_sfc_time_delta = 0.2 # Maximum amount of time from endpoints of external surface met dataset to extrapolate [hours]

ext_sfc_relative_height = 0 # Relative height of the met station to the IRS zenith port [m]; note if met station is below IRS port then the value should be negative

Note the assumed random errors for both the in-situ ambient temperature and RH observations, which are used to assign uncertainties to these observations. These assumed errors are very typical for most meteorological sensors

add_tropoe_T_input_flag = 0 # Flag to add previous good TROPoe temperature retrieval as input to current retrieval (1-yes, 0-no)

add_tropoe_q_input_flag = 0 # Flag to add previous good TROPoe water vapor retrieval as input to current retrieval (1-yes, 0-no)

add_tropoe_input_lwp_thres = 10.0 # Only will consider using previous TROPoe profile if LWP is below this threshold [g/m2] or CBH is above threshold below

add_tropoe_input_cbh_thres = 5.0 # Only will consider using previous TROPoe profile if CBH is above this threshold [km AGL] or LWP is below threshold above

add_tropoe_use_pblh_flag = 1 # If set, then the middle height point in the adder and multiplier profiles becomes maximum of that value or the PBLH [km AGL]

See the section on the "add_tropoe" option

cbh_type = 0 # 0 - output options and stop, 1 - VCEIL, 2 - Gregs ASOS CBH file, 3 - CLAMPS dlfp data, 4 - ARM dlprofwstats data

cbh_path = /data/ceil # Path to the CBH data

cbh_window_in = 20 # Inner temporal window (full-size) centered upon IRS time to look for cloud

cbh_window_out = 180 # Outer temporal window (full-size) centered upon IRS time to look for cloud}
cbh_default_ht = 2.0 # Default CBH height [km AGL], if no CBH data found

The "cbh_window_in" is not used currently, and "cbh_window_out" keyword is used as follow: if the ceilometer identified a cloud base height anytime +/- this time range from the current sample then the minimum value found will be used as the height of the cloud. It is anticipated that the meanings of these two key-value pairs will change after the diagnosed cloud base height logic is improved.

output_rootname = None # String with the rootname of the output file
output_path = /data/tropoe # Path where the output file will be placed
output_clobber = 0 # 0 - do not clobber preexisting output files, 1 - clobber them, 2 - append to the last file of this day

The output_clobber = 2 is a very useful option, especially for real-time processing. It allows the code to only process "new" (or unprocessed) times that occur after the last retrieval in the current output file, and the new retrievals are appended to the current output file. Do not change the VIP file between iterations when using this "append" option!

retrieve_temp = 1 # 0 - do not retrieve temp, 1 - do retrieve temp (default)
retrieve_wvmr = 1 # 0 - do not retrieve wvmr, 1 - do retrieve wvmr (default)
retrieve_lcloud = 1 # 0 - do not retrieve liquid clouds, 1 - retrieve liquid cloud properties
retrieve_icloud = 0 # 0 - do not retrieve ice clouds, 1 - retrieve ice cloud properties

Note that there are similar flags to turning on/off the trace gas retrievals also. However, the trace gas retrievals are still considered experimental, and thus are by default turned off.

qc_rms_value = 10.0 # The RMSa value between ((obs minus calc)/obs_uncert), with values less than this being 'good'
qc_gamma_value = 5.0 # The gamma value, with values less than this being 'good'

These two key-value pairs are used to set the qc_flag: if rmsa > qc_rms_value or gamma > qc_gamma_value, then the qc_flag will indicated "poor quality"

recenter_prior = 1 # 0 - do not recenter, 1 - Recenter WVMR based on sfc wv field and using conserve-RH for temp, 2 - Recenter WVMR based on PWV and conserve-RH for temp, 3 - Recenter WVMR based on sfc wv field and using conserve-covariance for temp, 4 - Recenter WVMR based on PWV and conserve-covariance for temp, 5 - Recenter based on near-sfc radiometric air temperature

The values of recenter_prior of 2, 3, 4, or 5 are experimental. We recommend only using either 0 or 1 for your processing

prior_lwp_mn = 0.0 # Mean LWP [g/m2]
prior_lwp_sd = 200.0 # 1-sigma uncertainty in LWP [g/m2]
prior_lReff_mn = 8.0 # Mean liquid Reff [microns]
prior_lReff_sd = 4.0 # 1-sigma uncertainty in liquid Reff [microns]
prior_itau_mn = 0.0 # Mean ice cloud optical depth (geometric limit)
prior_itau_sd = 50.0 # 1-sigma uncertainty in ice cloud optical depth
prior_iReff_mn = 25.0 # Mean ice cloud Reff [microns]

prior_iReff_sd = 8.0 # 1-sigma uncertainty in ice cloud} Reff [microns]

Used to set the prior mean and standard deviation information for clouds

plot_output = 0 # 0 - do not generate quicklooks of the output; 1 - generate quicklooks of the output after every iteration; 2 - plot from pre-existing file and abort
plot_path = None # Path to store outputted quicklooks. Defaults to output path
plot_rootname = None # Sets a prefix to the quicklooks. Defaults to the output_rootname vip entry
plot_xlim = 0.0, 24.0 # Hours of day
plot_ylim = 0.0, 2.0 # Height AGL in km
plot_temp_lim = -10.0, 20.0 # Temperature limit (C)
plot_wvmr_lim = 0.0, 20.0 # WVMR limit (in g/kg)
plot_tuncert_lim = 0.0, 3.0 # Temperature uncertainty limit (C)
plot_wvuncert_lim = 0.0, 3.0 # WVMR uncertainty limit (g/kg)
plot_theta_lim = 290.0, 320.0 # Potential Temperature limit (K)
plot_rh_lim = 0.0, 100.0 # Relative humidity limit (%)
plot_thetae_lim = 290.0, 320.0 # equivalent potential temperature limit (K)
plot_dewpt_lim = -15.0, 25.0 # Dewpoint limit (C)
plot_comment = None # String to put in the lower left of the quicklook figures
plot_tres_min_gap = None # In minutes. Defaults to 3 x tres
plot_lwp_cbh_threshold = 8.0 # Min LWP to show CBH dots

The TROPoe code has the ability to generate quicklook images, which can be useful for automated processing. It can also generate quicklooks from a previously processed file using plot_output=2 and rerun the code (only the plotting will occur)

Note that many more attributes can be added to the output file by specifying an unlimited number of "globatt_XX = ..." lines in the VIP file, where the XX is unique for each line. This allows you to add metadata describing your site, the instrument and its setup, contacts for the dataset, a DOI for the dataset if you have one, and more.

b) Experimental VIP key-value pairs

There are a large number of other key-value pairs in the VIP file that we consider more experimental. As with any "experimental" options, some are more so than others! Here are the other options that are available, and again, we will offer comments in *blue italics* along the way.

irs_noise_inflation = 1.0 # Value to increase the assumed random error in the IRS data
irs_smooth_noise = 0 # The temporal window [minutes] used to smooth the IRS noise with time

To inflate the entire IRS noise spectrum by a factor, or to spectrally smooth the noise

irs_band_noise_inflation = 1 # 0 -- off (no noise inflation), 1 -- on (will inflate the noise in the spectral band below; requires surface WVMR input from met station)
irs_band_noise_wnums = 775.0,810.0 # A comma separated list of two wavenumbers; the noise spectrum between these will be inflated
irs_band_noise_sfc_npts = 4 # The number of points in the x- (surface water vapor) and y- (noise inflation) axes

irs_band_noise_sfc_wvmr_vals = 0.,7,12,18 # A comma separated list of the surface water vapor mixing ratio values [g/kg] for the x-axis

irs_band_noise_sfc_multiplier = 20.0,20,1,1 # A comma separated list of multipliers [unitless] for the y-axis

These four key-value pairs are used to increase the random error in the 775-810 cm^{-1} region by the multiplier, but the multiplier is a linear function of the surface water vapor mixing ratio. This band was found to improve IRS retrieval performance in moist environments, but has detrimental impacts when the water vapor amount is low (Adler et al. 2024). If there is no surface met, then we HIGHLY RECOMMEND that the 790-810 cm^{-1} band not be used in IRS retrievals!

irs_calib_pres = 0.0, 1.0 # Intercept [mb] and slope [mb/mb to calib (newP = int = slope*obsP) (need comma between them)

Used to perform a first-order linear correction to the surface pressure observation

irs_use_missingDataFlag = 1 # Set this to 1 to use the field 'missingDataFlag' (from the ch1 file) to remove bad IRS data from analysis. If zero, then all IRS data will be processed,

irs_hatch_switch = 1 # 1 - only include hatchOpen=1 when averaging, 0 - include all IRS samples when averaging

irs_ignore_status_hatch = 0 # 0 -- use the hatchOpen flag; 1 -- ignore the hatchOpen flag

irs_ignore_status_missingDataFlag = 0 # 0 -- use the missingDataFlag flag; 1 -- ignore the missingDataFlag flag

Used to handle cases with weird hatch flag settings or when the missingDataFlag isn't set correctly

irs_fv = 0.0 # Apply a foreoptics obscuration correction

irs_fa = 0.0 # Apply an aftoptics obscuration correction

irs_old_ffov_halfangle = 23.0 # Original Half angle [millirad] of the finite field of view of the instrument

irs_new_ffov_halfangle = 0.0 # New half angle [millirad] of the finite field of view of the instrument (values} <= 0 will result in no correction being applied)

irs_spec_cal_factor_ch1 = 1.0 # The multiplicative stretch factor to change the spectral calibration of IRS ch1 data

irs_spec_cal_factor_ch2 = 1.0 # The multiplicative stretch factor to change the spectral calibration of IRS ch2 data

This set of key-value pairs are used to correct/perturb various corrections applied to IRS data

irs_mlev_cbh_wnum_range = 790.0, 810 # The wavenumber range [cm^{-1}] that should be used for the MLEV CBH calculation

irs_tcld_wnum_range = 898.0, 904 # The wavenumber range [cm^{-1}] that should be used for the Tcld CBH calculation

These are used to modify the behavior of the MLEV cloud base height diagnostic

irs_1m_od_thres = 1.0 # IRS channels where the 1m optical depth is above this threshold are not used

Some spectral elements in the infrared spectrum are so opaque due to gaseous absorption that the IRS is unable to make a calibrated measurement in that spectral

element. This key-value pair is used to set the optical depth of a 1-m thick atmospheric layer; if the optical depth is above this threshold, then the forward calculation is set to -999, which will result in that particular spectral element not being used in the retrieval.

ext_wv_prof_type = 0 # External WV profile source: 0-none; 1-sonde; 2-ARM Raman lidar (rlprofmr), 3-NCAR WV DIAL, 4-Model sounding

ext_wv_prof_path = None # Path to the external profile of WV data

ext_wv_prof_minht = 0.0 # Minimum height to use the data from the external WV profiler [km AGL]

ext_wv_prof_maxht = 10.0 # Maximum height to use the data from the external WV profiler [km AGL]

ext_wv_prof_min_error = 0.01 # Minimum random error allowed in the profile at any height [g/kg]

ext_wv_noise_mult_val = 1.0, 1, 1.0 # 3-element comma delimited list with the multipliers to apply the noise profile of the external water vapor profile (must be > 1)

ext_wv_noise_mult_hts = 0.0, 3, 20 # 3-element comma delimited list with the corresponding heights for the noise multipliers [km AGL]

ext_wv_add_rel_error = 0.0 # When using the RLID, I may want to include a relative error contribution to the uncertainty to account for calibration. This is a correlated error component, and thus effects the off-diagonal elements of the observation covariance matrix. Units are [%RH]

ext_wv_ht_offset = 0.0 # Height offset relative to the master instrument [km]; a negative value indicates that this profile starts below the master instrument

ext_wv_time_delta = 1.0 # Maximum amount of time from endpoints of external WV dataset to extrapolate [hours]

These key-value pairs are used to bring in an external water vapor profile as an additional observation in the Y vector. The most-general option is to use the "model sounding" format, which can be used for any gridded dataset (e.g., model forecast output, gridded UAS data, tower data that have multiple heights, remote sensing data, etc). See section 9d

ext_temp_prof_type = 0 # External temperature profile source: 0-none; 1-sonde; 2-ARM Raman lidar (rlproftemp); 4-Model sounding, 5-RASS

ext_temp_prof_path = None # Path to external profile of temp data

ext_temp_prof_minht = 0.0 # Minimum height to use the data from the external temp profiler [km AGL]

ext_temp_prof_maxht = 10.0 # Maximum height to use the data from the external temp profiler [km AGL]

ext_temp_noise_adder_val = 0.0, 0, 0 # 3-element comma delimited list of additive values to apply the noise profile of the external temperature profile (must be >= 0)

ext_temp_noise_adder_hts = 0.0, 3, 20 # 3-element comma delimited list with the corresponding heights for the additive value [km AGL]

ext_temp_ht_offset = 0.0 # Height offset relative to the master instrument [km]; a negative value indicates that this profile starts below the master instrument

ext_temp_time_delta = 1.0 # Maximum amount of time from endpoints of external temp dataset to extrapolate [hours]

These key-value pairs are used to bring in an external temperature profile as an additional observation in the Y vector. The most-general option is to use the "model sounding" format, which can be used for any gridded dataset (e.g., model forecast output, gridded UAS data, tower data that have multiple heights, remote sensing data, etc). See section 9d

mod_wv_prof_type = 0 # NWP model WV profile source: 0-none; 4-Model sounding
mod_wv_prof_path = None # Path to the model profile of WV data
mod_wv_prof_minht = 0.0 # Minimum height to use the data from the model WV profiler [km AGL]
mod_wv_prof_maxht = 10.0 # Maximum height to use the data from the model WV profiler [km AGL]
mod_wv_prof_min_error = 0.01 # Minimum random error allowed in the profile at any height [g/kg]
mod_wv_noise_mult_val = 1.0, 1, 1.0 # 3-element comma delimited list with the multipliers to apply the noise profile of the model water vapor profile (must be > 1)
mod_wv_noise_mult_hts = 0.0, 3, 20 # 3-element comma delimited list with the corresponding heights for the noise multipliers [km AGL]
mod_wv_ht_offset = 0.0 # Height offset relative to the master instrument [km]; a negative value indicates that this profile starts below the master instrument
mod_wv_time_delta = 1.0 # Maximum amount of time from endpoints of model WV dataset to extrapolate [hours]

This is identical logic to the "ext_wv_" block above, and thus provides a way to bring a second external water vapor profile into the observation vector. If you are using NWP model analysis / forecasts as this external dataset, we recommend you use the mod_wv_ keys.

mod_temp_prof_type = 0 # NWP model temperature profile source: 0-none; 4-Model sounding
mod_temp_prof_path = None # Path to the model profile of temp data
mod_temp_prof_minht = 0.0 # Minimum height to use the data from the model temp profiler [km AGL]
mod_temp_prof_maxht = 10.0 # Maximum height to use the data from the model temp profiler [km AGL]
mod_temp_noise_adder_val = 0.0, 0, 0 # 3-element comma delimited list of additive values to apply the noise profile of the model temperature profile (must be >= 0)
mod_temp_noise_adder_hts = 0.0, 3, 20 # 3-element comma delimited list with the corresponding heights for the additive value [km AGL]
mod_temp_ht_offset = 0.0 # Height offset relative to the master instrument [km]; a negative value indicates that this profile starts below the master instrument
mod_temp_time_delta = 1.0 # Maximum amount of time from endpoints of model WV dataset to extrapolate [hours]

This is identical logic to the "ext_temp_" block above, and thus provides a way to bring a second external temperature profile into the observation vector. If you are using NWP model analysis / forecasts as this external dataset, we recommend you use the mod_temp_ keys.

add_tropoe_gamma_threshold = 2 # Only samples with gamma values less than this threshold will be considered for use in future retrieval

add_tropoe_T_noise_adder_val = 3.0, 1, 1 # 3-element comma delimited list of additive values to apply the noise profile of the previous TROPoe input temperature profile (must be >= 0)

add_tropoe_T_noise_adder_hts = 0.0, 1, 20 # 3-element comma delimited list with the corresponding heights for the additive value [km AGL]

add_tropoe_q_noise_mult_val = 5.0, 2, 2 # 3-element comma delimited list with the multipliers to apply the noise profile of the model water vapor profile (must be > 1)

add_tropoe_q_noise_mult_hts = 0.0, 1, 20 # 3-element comma delimited list with the corresponding heights for the noise multipliers [km AGL]

add_tropoe_use_prior_as_max = 1 # If set, then the maximum amount of uncertainty is the uncertainty of the prior

Additional controls the modify the behavior of the add_tropoe option. See sections 9a-iii and 9c-ii for examples.

co2_sfc_type = 0 # External CO2 surface data type: 0-none, 1-DDT QC PGS data

co2_sfc_npts = 1 # Number of surface CO2 in-situ points to use in the retrieval. Minimum=1, maximum=1000. Larger number increases the weight of the observation

co2_sfc_rep_error = 0.0 # Representativeness error for the CO2 surface measurement [ppm], which is added to the uncertainty of the obs in the input file

co2_sfc_path = None # Path to the external surface CO2 data

co2_sfc_relative_height = 0 # Relative height of the CO2 surface measurement to the IRS zenith port [m]; note if in-situ obs is below IRS port then the value should be negative

co2_sfc_time_delta = 1.5 # Maximum amount of time from endpoints of external CO2 in-situ dataset to extrapolate [hours]

*Some sites make in-situ CO2 concentration observations at the surface. These key-value pairs to bring this observation into the **Y** vector*

output_akernal = 1 # 0 - do not output Sop and Akernal; 1 - output normal Sop and normal Akernal; 2 - output normal Sop, normal Akernal, and "no-model" Akernal

The TROPoe output can be voluminous, but this is primarily because the posterior covariance and averaging kernal matrices. This key-value pair allows these fields to be turned off (0), or to output a 2nd posterior and averaging kernal does NOT include the contributions from the mod_ profiles

lbl_home = /home/tropoe/vip/src/lblrtm_v12.17/lblrtm # String with the LBL_HOME path

This selects the version of the LBLRTM to use. To use v12.1, then set this key to /home/tropoe/vip/src/lblrtm_v12.1/lblrtm

lbl_temp_dir = /tmp # Temporary working directory for the retrieval

The LBLRTM creates many temporary files during its execution (and it is run multiple times per iteration, when IRS data are used). If your machine has a very fast disk (or a ram-disk), then set this key to "/tmp2". Then when you run the she-script to execute TROPoe (see section 3), make sure your fast disk is entered as "tmpDir"

lbl_std_atmos = 6 # Standard atmosphere to use in LBLRTM and MonoRTM calcs

There are 6 standard atmospheres associated with the LBLRTM. They are: 1-tropics, 2-mid-latitude summer, 3-mid-latitude winter, 4-sub-arctic summer, 5-sub-arctic winter,

and 6-US standard. While the retrieval has only marginal sensitivity to this setting, select the one that is most appropriate for your application

path_std_atmos = /home/tropoe/vip/src/input/idl_code/std_atmosphere.idl # The path to the IDL save file with the standard atmosphere info in it

This should not be changed

lbl_tape3 = TAPE3.350-1600cm-1.first_7_molecules # The TAPE3 file to use in the lblrtm calculation. Needs to be in the directory lbl_home/hitran/

There is some dependence on the calculation speed of the LBLRTM magnitude spectral range in the tape3, with smaller spectral ranges leading to faster calculations. The other tape3 that is available in the container is TAPE3.10-3500cm-1.first_7_molecules

monortm_version = v5.0 # String with the version information on MonoRTM

monortm_wrapper = /home/tropoe/vip/src/monortm_v5.0/wrapper/monortm_v5 # Turner wrapper to run MonoRTM

monortm_exec = /home/tropoe/vip/src/monortm_v5.0/monortm/monortm_v5.0_linux_intel_sgl # AERs MonoRTM executable

monortm_spec = /home/tropoe/vip/src/monortm_v5.0/monolnfl_v1.0/TAPE3.spectral_lines.dat.0_55.v5.0_veryfast # MonoRTM spectral database

These key-value pairs point to the MonoRTM v5.0, which are critical if MWR data are used in the observation vector. These should not be changed

lblrtm_jac_option = 4 # 1 - LBLRTM Finite Diffs, 2 - 3calc method (deprecated), 3 - deltaOD method (deprecated), 4 - interpol method

Only option 4 is enabled; the others have been deprecated

lblrtm_forward_threshold = 0.0 # The upper LWP threshold [g/m2] to use LBLRTM vs. radxfer in forward calculation

This key is also deprecated and should be left unchanged

lblrtm_jac_interpol_npts_wnum = 10 # The number of points per wnum to use in the compute_jacobian_irs_interpol() function

Used to speed up the infrared jacobian calculation; do not change

monortm_jac_option = 2 # 1 - MonoRTM Finite Diffs, 2 - 3calc method

Option 1 is deprecated; do not change

jac_max_ht = 8.0 # Maximum height to compute the Jacobian [km AGL]

This dictates the maximum height where the jacobian is calculated. It was chosen as there is virtually no information above this altitude for IRS or MWR observations.

max_iterations = 10 # The maximum number of iterations to use

Self-evident

cvgmult = 0.25 # The multiplier used for the convergence criteria (e.g., di2m < cvgmult*dim[Y])

Used to determine convergence in the Rodgers sense

first_guess = 1 # 1 - use prior as FG, 2 - use lapse rate and 60% RH profile as FG, 3 - use previous sample as FG

With the use of the gamma parameter (see Turner and Löhnert 2014), the retrieval is largely insensitive to the first guess (FG). Note that the first guess and the prior, and their impact on the retrieval, is NOT the same!

superadiabatic_maxht = 0.3 # The maximum height a superadiabatic layer at the surface can have [km AGL]

This aows the retrieval to have super adiabatic layers above the surface

spectral_bands = None # An array of spectral bands to use (e.g. 612-618,624-660,674-713,713-722,538-588,793-804,860.1-864.0,872.2-877.5,898.2-905.4)

A comma separated list of spectral bands to use in the retrieval. The example shown is the default values

retrieve_co2 = 0 # 0 - do not retrieve co2, 1 - do retrieve co2 (step model), 2 - do retrieve co2 (exponential model -- disabled)
fix_co2_shape = 0 # (This option only works with retrieve_co2=1): 0 - retrieve a three coefs, 1 - shape coef is f(PBLH) and fixed
retrieve_ch4 = 0 # 0 - do not retrieve ch4, 1 - do retrieve ch4 (step model), 2 - do retrieve ch4 (exponential model -- disabled)
fix_ch4_shape = 0 # (This option only works with retrieve_ch4=1): 0 - retrieve a three coefs, 1 - shape coef is f(PBLH) and fixed
retrieve_n2o = 0 # 0 - do not retrieve n2o, 1 - do retrieve n2o (step model), 2 - do retrieve n2o (exponential model -- disabled)
fix_n2o_shape = 0 # (This option only works with retrieve_n2o=1): 0 - retrieve a three coefs, 1 - shape coef is f(PBLH) and fixed

The retrieve_xx = 1 turn on the retrieval of the various trace gases. The fix_*_shape options should not be changed.

lcloud_ssp = /home/tropoe/vip/src/input/ssp_db_files/ssp_db.mie_wat.gamma_sigma_0p100 # SSP file for liquid cloud properties
icloud_ssp = /home/tropoe/vip/src/input/ssp_db_files/ssp_db.mie_ice.gamma_sigma_0p100 # SSP file for ice cloud properties

These are the paths to lookup tables with precomputed cloud optical properties. These should not be changed

recenter_input = 0.0 # Sfc WVMR or PWV value to use in the recentering process. Set to zero for the value to be determined from other input data (i.e. sfc met)

If there is no surface met, or you desire to override it, set this key to the value of the water vapor mixing ratio you desire (g/kg)

recenter_covar_min_sfactor = 0.6 # Minimum scale factor that will be used to rescale the prior covariance matrix
recenter_covar_max_sfactor = 1.4 # Maximum scale factor that will be used to rescale the prior covariance matrix

Recentering the prior involves multiplying the mean prior water vapor mixing ratio by a scaling factor (SF). Theoretically, the covariance matrix should also be scaled by the same SF. However, experimentation has demonstrated that we don't want to shrink or

expand the prior covariance matrix too much; these key-value pairs provide those limits.

prior_tikhonov_factor = 1.5e-05 # The Tikhonov factor (>= 0) to apply at the prior TQ covariance matrix to improve its conditioning (set to 0 to turn off)

Used to condition the TQ component of the prior covariance matrix (i.e., $\boldsymbol{S}_{a,Tq}$). It has negligible impact

prior_t_ival = 1.0 # The prior inflation factor (>= 1) to apply at the surface for temperature
prior_t_iht = 1.0 # The height [km AGL] where the inflation factor goes to 1 (linear) for temperature
prior_q_ival = 1.0 # The prior inflation factor (>= 1) to apply at the surface for water vapor mixing ratio
prior_q_iht = 1.0 # The height [km AGL] where the inflation factor goes to 1 (linear) for water vapor mixing ratio

These four keys are used to inflate the magnitude of the prior's uncertainty in the temperature and water vapor profiles near the surface

prior_tq_cov_val = 1.0 # A multiplicative value (0 < val <= 1) that is used to decrease the covariance in the prior between temperature and WVMR at a heights

This is used to scale down all of the covariances between temperature and humidity in the prior covariance matrix.

prior_chimney_ht = 0.0 # The height of any 'chimney' [km AGL]; prior data below this height are totay decorrelated

Some IRS instruments are installed in a warm environment, and look through a "chimney" of near ambient temperature before the instrument "sees" the atmosphere. The REFIR-PAD at Dome-C is an example of this. This keyword is used to specify the length of that chimney

prior_co2_mn = -1.0, 5, -5 # Mean co2 concentration [ppm] (see 'retrieve_co2' above)
prior_co2_sd = 2.0, 15, 3 # 1-sigma uncertainty in co2 [ppm]
prior_ch4_mn = 1.793, 0, -5 # Mean ch4 concentration [ppm] (see 'retrieve_ch4' above)
prior_ch4_sd = 0.0538, 0.0015, 3 # 1-sigma uncertainty in ch4 [ppm]
prior_n2o_mn = 0.31, 0, -5 # Mean n2o concentration [ppm] (see 'retrieve_n2o' above)
prior_n2o_sd = 0.0093, 0.0002, 3 # 1-sigma uncertainty in n2o [ppm]

These are used to set the prior mean and standard deviations of the trace gases. These values were determined from multiple years of observations at the ARM SGP site.

min_PBL_height = 0.05 # The minimum height of the planetary boundary layer (used for trace gases) [km AGL]

The PBL height is derived from the potential temperature profile using a parcel method (e.g., Nielsen-Gammon et al. 2008), and is strictly only valid in a convective environment when there is mixing near the surface. However, because PBL height is used in the trace gas retrievals, the PBL height must be non-zero. This keyword is used to set the minimum height of the PBL.

max_PBL_height = 5.0 # The maximum height of the planetary boundary layer (used for trace gases) [km AGL]

This is the maximum height of the PBL allowed

nudge_PBL_height = 0.5 # The temperature offset (nudge) added to the surface temp to find PBL height [C]

The nudge parameter added to the surface temperature in the PBL height calculation

14) Appendix 2: Subset of TROPoe peer-reviewed papers

This is a subset of papers that use TROPoe data, and were chosen to highlight the varied uses that are possible.

Adler, B., J.M. Wilczak, L. Bianco, L. Baiteau, C.J. Cox, G. de Boer, I.V. Djalalova, M.R. Gallagher, J.M. Intrieri, T.P. Meyers, T.A. Myers, J.B. Olson, S. Pezoa, J. Sedlar, E.N. Smith, D.D. Turner, and A.B. White, 2023: Impact of seasonal snow-cover change on the observed and simulated state of the atmospheric boundary layer in a high-altitude mountain valley. *J. Geophys. Res*., **128**, e2023JD038497, doi:10.1029/2023JD038497.

Blumberg, W.G., D.D. Turner, U. Loehnert, and S. Castleberry, 2015: Ground-based temperature and humidity profiling using spectral infrared and microwave observations. Part 2: Actual retrieval performance in clear sky and cloudy conditions. *J. Appl. Meteor. Clim*., **54**, 2305-2319, doi:10.1175/JAMC-D-15-0005.1

Blumberg, W.G., T.J. Wagner, D.D. Turner, and J. Correia Jr., 2017: Quantifying the accuracy and uncertainty of diurnal thermodynamic profiles and convection indices derived from the Atmospheric Emitted Radiance Interferometer. *J. Appl. Meteor. Clim*., **56**, 2747-2766, doi:10.1175/JAMC-D-17-0036.1

Carlin, J. T., E. N. Smith, and K. Giannokopoulos, 2024: Contextualizing polarimetric retrievals of boundary-layer height using state-of-the-art boundary layer profiling. *J. Appl. Meteorol. Climatol*., **63**, 765-780, doi: 10.1175/JAMC-D-23-0231.1

Chipilski, H.G., X. Wang, and D.B. Parsons, 2020: Impact of assimilating PECAN profilers on the prediction of bore-driven nocturnal convection: A multi-scale forecast evaluation for the 6 July 2015 case study. *Month. Wea. Rev*., **148**, 1147-1175, doi:10.1175/MWR-D-19-0171.1.

Chipilski, H.G., X. Wang, D.B. Parsons, A. Johnson, and S.K. Degelia, 2022: The value of assimilating different ground-based profiling networks on the forecasts of bore-generating nocturnal convection. *Mon. Wea. Rev*, **150**, 1273-1292, doi:10.1175/MWR-D-21-0193.1

Degelia, S.K., X. Wang, D.J. Stensrud, and D.D. Turner, 2020: Systematic evaluation of the impact of assimilating a network of ground-based remote sensing profilers for forecasts of nocturnal convection initiation during PECAN. *Month. Wea. Rev*., **148**, 4703-4728, doi:10.1175/MWR-D-20-0118.1.

Degelia, S.K., and X. Wang, 2023: Impacts of methods for estimating the observation error variance for the frequent assimilation of thermodynamic profilers on convective-scale forecasts. *Mon. Wea. Rev*., **151**, 855-875, doi:10.1175/MWR-D-21-0049.1

Djalalova, I.V., D.D. Turner, L. Bianco, J.M Wilczak, J. Duncan, B. Adler, and D. Gottas, 2022: Improving thermodynamic profile retrievals from microwave radiometers by including radio acoustic sounding system (RASS) observations. *Atmos. Meas. Tech*., **15**, 521-537, doi:10.5194/amt-15-521-2022.

Guy, H, D.D. Turner, V.P. Walden, I.M. Brooks, and R.R. Neely III, 2022: Passive ground-based remote sensing of radiation fog. *Atmos. Meas. Tech*., **15**, 5095-5115, doi:10.5194/amt-15-5095-2022

Haghi, K.R., H. Chipilski, A. Johnson, B. Geerts, D.D. Turner, D. Imy, B. Adams-Selin, D.B. Parsons, and X. Wang, 2019: Bore-ing into nocturnal convection. *Bull. Amer. Meteor. Soc*., **100**, 1103-1121, doi:10.1175/BAMS-D-17-0250.1

Hu, J., N. Yussouf, D.D. Turner, T.A. Jones, and X. Wang, 2019: Impact of ground-based remote sensing boundary layer observations on short-term probabilistic forecasts of a tornadic supercell event. *Wea. Forecasting*, **34**, 1453-1476, doi:10.1175/WAF-D-18-0200.1.

Newsom, R.K., D.D. Turner, R. Lehtinen, C. Muenkel, and R. Roininen, 2020: Evaluation of a compact differential absorption lidar for routine water vapor profiling of the atmospheric boundary layer. *J. Atmos. Oceanic Technol*., **37**, 47-65, doi:10.1175/JTECH-D-18-0102.1

Smith, E.N., B. R. Greene, T. B. Bell, W. G. Blumberg, R. Wakefield, D. Rief, Q. Niu, Q. Wang, and D. D. Turner, 2021: Evaluation and application of multi-instrument boundary-layer thermodynamic retrievals. *Bound. Lay. Meteorol*., **181**, 95-123, doi: 10.1007/s10546-021-00640-2

Wagner, T.J., D.D. Turner, T. Heus, and W.G. Blumberg, 2022: Observing profiles of derived kinematic field quantities using a network of profiling sites. *J. Atmos. Oceanic Tech*., **39**, 335-351, doi:10.1175/JTECH-D-21-0061.1.

Wagner, T. J., A. C. Czarnetzki, M. Christiansen, R. Bradley Pierce, C. O. Stanier, A. F. Dickens, and E. W. Eloranta, 2022: Observations of the development and vertical structure of the lake-breeze circulation during the 2017 Lake Michigan Ozone Study. *J. Atmos. Sci*., **79**, 1005-1020. doi.org:10.1175/JAS-D-20-0297.1.

Wakefield, R.A., D.D. Turner, and J.B. Basara, 2021: Evaluation of a land-atmosphere coupling metric computed from a ground-based infrared interferometer. *J. Hydrometeor*., **22**, 2073-2087, doi:10.1175/JHM-D-20-0303.1.

Wakefield, R.A., D.D. Turner, T. Rosenberger, T. Heus, T. Wagner, J. Santanello, and J. Basara, 2023: A methodology for estimating the energy and moisture budget of the convective boundary layer using continuous ground-based infrared spectrometer observations. *J. Appl. Meteor. Clim*., **62**, 901-914, doi:10.1175/JAMC-D-22-0163.1.

15) Appendix 3: Version change log

Date Created	Version tagged	Main new contributions
20260225	tropoe:v1.0	Added the shell scripts that run both TROPoe and MIXCRA to the code distribution. Fixed MIXCRA bugs (sfc_emissivity, apply_t_cloud_constraints) and plotting bugs. Added all VIP information to MIXCRA output. Added default MIXCRA surface emissivity spectrum, and changed MIXCRA's default IRS spectral bands. Updated some VIP metadata to be consistent with User Guide. Fixed ASSIST blackbody cavity factor error. Added User Guide to the distribution. Changed the way we call Popen as process.
20251117	tropoe:v0.20	Added MIXCRA. Fixed OMB logic so it would process a samples in an input TROPoe file.
20250518	tropoe:v0.19	Bug fixes for both diagnostic CBH methods. Fixed bug in read_external_timeseries logic to handle missing data. Turned off diagnostic fields when in OMB mode. Updated code to process both REFIR-PAD datasets
20250220	tropoe:v0.18.2	Bug fix to previous 0.18.2
20250126	tropoe:v0.18.2	Added derived CBH using MLEV and Tcld methods, and added sinc-squared to convolve_to_irs so that the REFIR-PAD could be properly modeled
20250107	tropoe:v0.18.1	Added ability to change required minht for OMB input profile (for Jeana Mascio) -- this is unofficial release
20241227	tropoe:v0.18	Bug fixed in_tropoe logic, and updated so append works. Changed units to make ARM happy. Refactored read_external_timeseries logic. Updated OMB logic to also use previous TROPoe output as input
20241108	tropoe:v0.17	Added the in_tropoe option (Adler et al. 2024), OMB calculation option, reenabled the chimney effect, irs_1m_od_logic, and new VIP specified limits on the amount the apriori covariance matrix can be scaled
20241016	tropoe:20141016	Only change is that the LBLRTM v12.17 was added to the container. Everything else is unchanged
20240807	tropoe:v0.16	Constrain the amount of scaling of the prior covariance matrix when recentering, and updated fix_nonphysical_wv
20240722	tropoe:v0.15	Automatically inflate IRs noise in 780-810 cm-1 region if no met data, allow O-B to be computed if VIP.max_iterations=0, fix units error in plotting script, added test for TAPE3 wnum limits
20240522	tropoe:v0.14	Min noise floor for {ext,mod}_wv_prof_min_error, added REFIR-PAD capability, irs_scene_mirror_angle allowed to be -999
20240223	tropoe:v0.13	Fixed metadata issue in output file, improved QC for masked input and negative IR radiances
20231120	tropoe:v0.12	Multiple bug fixes: opres with VIP.clobber, esat NaNs, recentering dimension, location structure
20231105	tropoe:v0.11+	QCflag bug fix, met-single-sample fix, append-reproducibility bug fixed
20231016	tropoe:v0.10	Spring priors instead of annual priors, DWD ceilometer input, ability to read Cologne MWR sfc data, ASSIST sum sfc data, new cDFS / vres fields that do not include model input, automated plotting capability, better processing of VIP keywords that are arrays, some other bug fixes

20230927	tropoe:v0.9.3	Sorted model profile by height, bug fix on input surface pressure unit check
20230918	tropoe:v0.9.1	Added E-PROFILE instruments, ability to add height-offset to the external_profiles
20230909	tropoe:v0.9	Very simple bug fix of the TG profiles

16) Appendix 4: Header of the TROPoe output netCDF file

This is an example header from a typical TROPoe output file. This example was created by v0.20.

```
netcdf tropoe.zenith_and_elevation.20220616.000005.nc {
dimensions:
   time = UNLIMITED ; // (142 currently)
   height = 55 ;
   obs_dim = 36 ;
   gas_dim = 3 ;
   dfs_dim = 16 ;
   arb_dim1 = 123 ;
   arb_dim2 = 123 ;
variables:
   int base_time ;
      base_time:long_name = "Epoch time" ;
      base_time:units = "seconds since 1970-01-01 00:00:00 0:00" ;
   double time_offset(time) ;
      time_offset:long_name = "Time offset from base_time" ;
      time_offset:units = "Seconds" ;
   double time(time) ;
      time:long_name = "Time" ;
      time:units = "seconds since 2022-06-16 00:00:00 0:00" ;
      time:calendar = "standard" ;
   double hour(time) ;
      hour:long_name = "Time" ;
      hour:units = "hours since 2022-06-16 00:00:00 0:00" ;
   short qc_flag(time) ;
      qc_flag:long_name = "Manual QC flag" ;
      qc_flag:variable_type = "Quality control" ;
      qc_flag:comment1 = "unitless" ;
      qc_flag:comment2 = "Value of 0 implies quality of the retrieved fields is ok; non-zero values indicate that the sample has
suspect quality" ;
      qc_flag:comment3 = "See the global attribute Retrieval_option_flags to determine which variables were being retrieved for
this data file" ;
      qc_flag:value_0 = "Quality of retrieval is ok" ;
      qc_flag:value_2 = "Implies retrieval did not converge" ;
      qc_flag:value_3 = "Implies retrieval converged but RMS between the observed_vector and forward_calc is too large" ;
      qc_flag:value_4 = "Implies the gamma value of the retrieval was too large" ;
      qc_flag:RMSa_threshold_used_for_QC = "10.0 [unitless]" ;
      qc_flag:gamma_threshold_used_for_QC = "5.0 [unitless]" ;
   float height(height) ;
      height:long_name = "Height above ground level" ;
      height:units = "km" ;
   float temperature(time, height) ;
      temperature:long_name = "Temperature" ;
      temperature:units = "degC" ;
      temperature:variable_type = "Retrieved" ;
   float waterVapor(time, height) ;
      waterVapor:long_name = "Water vapor mixing ratio" ;
      waterVapor:units = "g/kg" ;
      waterVapor:variable_type = "Retrieved" ;
   float lwp(time) ;
      lwp:long_name = "Liquid water path" ;
      lwp:units = "g/m2" ;
      lwp:variable_type = "Retrieved" ;
   float lReff(time) ;
```

```
    lReff:long_name = "Liquid water effective radius" ;
    lReff:units = "microns" ;
    lReff:variable_type = "Retrieved" ;
  float iTau(time) ;
    iTau:long_name = "Ice cloud optical depth (geometric limit)" ;
    iTau:variable_type = "Retrieved but disabled -- will default to the a priori" ;
    iTau:comment1 = "unitless" ;
  float iReff(time) ;
    iReff:long_name = "Ice effective radius" ;
    iReff:units = "microns" ;
    iReff:variable_type = "Retrieved but disabled -- will default to the a priori" ;
  float co2(time, gas_dim) ;
    co2:long_name = "Carbon dioxide concentration" ;
    co2:units = "ppm" ;
    co2:variable_type = "Retrieved but disabled -- will default to the a priori" ;
    co2:comment = "Parameterized profile information; see users guide" ;
  float ch4(time, gas_dim) ;
    ch4:long_name = "Methane concentration" ;
    ch4:units = "ppm" ;
    ch4:variable_type = "Retrieved but disabled -- will default to the a priori" ;
    ch4:comment = "Parameterized profile information; see users guide" ;
  float n2o(time, gas_dim) ;
    n2o:long_name = "Nitrous oxide concentration" ;
    n2o:units = "ppm" ;
    n2o:variable_type = "Retrieved but disabled -- will default to the a priori" ;
    n2o:comment = "Parameterized profile information; see users guide" ;
  float sigma_temperature(time, height) ;
    sigma_temperature:long_name = "1-sigma uncertainty in temperature" ;
    sigma_temperature:units = "degC" ;
    sigma_temperature:variable_type = "Uncertainty of associated variable" ;
  float sigma_waterVapor(time, height) ;
    sigma_waterVapor:long_name = "1-sigma uncertainty in water vapor mixing vapor" ;
    sigma_waterVapor:units = "g/kg" ;
    sigma_waterVapor:variable_type = "Uncertainty of associated variable" ;
  float sigma_lwp(time) ;
    sigma_lwp:long_name = "1-sigma uncertainty in liquid water path" ;
    sigma_lwp:units = "g/m2" ;
    sigma_lwp:variable_type = "Uncertainty of associated variable" ;
  float sigma_lReff(time) ;
    sigma_lReff:long_name = "1-sigma uncertainty in liquid water effective radius" ;
    sigma_lReff:units = "microns" ;
    sigma_lReff:variable_type = "Uncertainty of associated variable" ;
  float sigma_iTau(time) ;
    sigma_iTau:long_name = "1-sigma uncertainty in ice cloud optical depth (geometric limit)" ;
    sigma_iTau:variable_type = "Uncertainty of associated variable" ;
    sigma_iTau:comment1 = "unitless" ;
  float sigma_iReff(time) ;
    sigma_iReff:long_name = "1-sigma uncertainty in ice effective radius" ;
    sigma_iReff:units = "microns" ;
    sigma_iReff:variable_type = "Uncertainty of associated variable" ;
  float sigma_co2(time, gas_dim) ;
    sigma_co2:long_name = "1-sigma uncertainty in carbon dioxide concentration" ;
    sigma_co2:units = "ppm" ;
    sigma_co2:variable_type = "Uncertainty of associated variable" ;
    sigma_co2:comment = "Parameterized profile information; see users guide" ;
  float sigma_ch4(time, gas_dim) ;
    sigma_ch4:long_name = "1-sigma uncertainty in methane concentration" ;
    sigma_ch4:units = "ppm" ;
```

sigma_ch4:variable_type = "Uncertainty of associated variable" ;
sigma_ch4:comment = "Parameterized profile information; see users guide" ;
float sigma_n2o(time, gas_dim) ;
sigma_n2o:long_name = "1-sigma uncertaintiy in nitrous oxide concentration" ;
sigma_n2o:units = "ppm" ;
sigma_n2o:variable_type = "Uncertainty of associated variable" ;
sigma_n2o:comment = "Parameterized profile information; see users guide" ;
short converged_flag(time) ;
converged_flag:long_name = "Convergence flag" ;
converged_flag:variable_type = "Quality control" ;
converged_flag:comment1 = "unitless" ;
converged_flag:value_0 = "0 indicates no convergence" ;
converged_flag:value_1 = "1 indicates convergence in Rodgers sense (i.e., di2m << dimY)" ;
converged_flag:value_2 = "2 indicates convergence (best rms after rms increased drastically" ;
converged_flag:value_3 = "3 indicates convergence (best rms after max_iter)" ;
converged_flag:value_9 = "9 indicates found NaN in Xnp1" ;
float gamma(time) ;
gamma:long_name = "Gamma parameter" ;
gamma:variable_type = "Diagnostic variable" ;
gamma:comment1 = "unitless" ;
gamma:comment2 = "See Turner and Loehnert JAMC 2014 for details" ;
short n_iter(time) ;
n_iter:long_name = "Number of iterations performed" ;
n_iter:variable_type = "Diagnostic variable" ;
n_iter:comment1 = "unitless" ;
float rmsr(time) ;
rmsr:long_name = "Root mean square error between IRS and MWR obs in the observation vector and the forward calculation" ;
rmsr:variable_type = "Quality control" ;
rmsr:comment1 = "unitless" ;
rmsr:comment2 = "Computed as sqrt(sum_over_i[((Y_i - F(Xn_i)) / sigma_Y_i)^2] / sizeY)" ;
rmsr:comment3 = "Only IRS radiance observations in the observation vector are used" ;
float rmsa(time) ;
rmsa:long_name = "Root mean square error between observation vector and the forward calculation" ;
rmsa:variable_type = "Quality control" ;
rmsa:comment1 = "unitless" ;
rmsa:comment2 = "Computed as sqrt(sum_over_i[((Y_i - F(Xn_i)) / sigma_Y_i)^2] / sizeY)" ;
rmsa:comment3 = "Entire observation vector used in this calculation" ;
float rmsp(time) ;
rmsp:long_name = "Root mean square error between prior T/q profile and the retrieved T/q profile" ;
rmsp:variable_type = "Diagnostic variable" ;
rmsp:comment1 = "unitless" ;
rmsp:comment2 = "Computed as sqrt(mean[((Xa - Xn) / sigma_Xa)^2])" ;
float chi2(time) ;
chi2:long_name = "Chi-square statistic of Y vs. F(Xn)" ;
chi2:variable_type = "Diagnostic variable" ;
chi2:comment1 = "unitless" ;
chi2:comment2 = "Computed as sqrt(sum_over_i[((Y_i - F(Xn_i)) / Y_i)^2] / sizeY)" ;
float convergence_criteria(time) ;
convergence_criteria:long_name = "Convergence criteria di^2" ;
convergence_criteria:variable_type = "Diagnostic variable" ;
convergence_criteria:comment1 = "unitless" ;
float dfs(time, dfs_dim) ;
dfs:long_name = "Degrees of freedom of signal" ;
dfs:variable_type = "Diagnostic variable" ;
dfs:comment1 = "unitless" ;
dfs:comment2 = "total DFS, then DFS for each of temperature, waterVapor, LWP, L_Reff, I_tau, I_Reff, carbonDioxide, methane, nitrousOxide" ;

float dfs_no_model(time, dfs_dim) ;
dfs_no_model:long_name = "Degrees of freedom of signal excluding model data" ;
dfs_no_model:variable_type = "Diagnostic variable" ;
dfs_no_model:comment1 = "unitless" ;
dfs_no_model:comment2 = "total DFS, then DFS for each of temperature, waterVapor, LWP, L_Reff, I_tau, I_Reff, carbonDioxide, methane, nitrousOxide" ;
dfs_no_model:comment3 = "If no model data is used in the obs vector this field will be the same as dfs" ;
float sic(time) ;
sic:long_name = "Shannon information content" ;
sic:variable_type = "Diagnostic variable" ;
sic:comment1 = "unitless" ;
float vres_temperature(time, height) ;
vres_temperature:long_name = "Vertical resolution of the temperature profile" ;
vres_temperature:units = "km" ;
vres_temperature:variable_type = "Diagnostic variable" ;
float vres_temperature_no_model(time, height) ;
vres_temperature_no_model:long_name = "Vertical resolution of the temperature profile exluding model data" ;
vres_temperature_no_model:units = "km" ;
vres_temperature_no_model:variable_type = "Diagnostic variable" ;
vres_temperature_no_model:comment1 = "If no model data is used in the obs vector this field will be the same as vres_temperature" ;
float vres_waterVapor(time, height) ;
vres_waterVapor:long_name = "Vertical resolution of the water vapor profile" ;
vres_waterVapor:units = "km" ;
vres_waterVapor:variable_type = "Diagnostic variable" ;
float vres_waterVapor_no_model(time, height) ;
vres_waterVapor_no_model:long_name = "Vertical resolution of the water vapor profile exluding model data" ;
vres_waterVapor_no_model:units = "km" ;
vres_waterVapor_no_model:variable_type = "Diagnostic variable" ;
vres_waterVapor_no_model:comment1 = "If no model data is used in the obs vector this field will be the same as vres_waterVapor" ;
float cdfs_temperature(time, height) ;
cdfs_temperature:long_name = "Vertical profile of the cumulative degrees of freedom of signal for temperature" ;
cdfs_temperature:variable_type = "Diagnostic variable" ;
cdfs_temperature:comment1 = "unitless" ;
float cdfs_temperature_no_model(time, height) ;
cdfs_temperature_no_model:long_name = "Vertical profile of the cumulative degrees of freedom of signal for temperature excluding model data" ;
cdfs_temperature_no_model:variable_type = "Diagnostic variable" ;
cdfs_temperature_no_model:comment1 = "unitless" ;
cdfs_temperature_no_model:comment2 = " If no model data is used in the obs vector this field will be the same as cdfs_temperature" ;
float cdfs_waterVapor(time, height) ;
cdfs_waterVapor:long_name = "Vertical profile of the cumulative degrees of freedom of signal for water vapor" ;
cdfs_waterVapor:variable_type = "Diagnostic variable" ;
cdfs_waterVapor:comment1 = "unitless" ;
float cdfs_waterVapor_no_model(time, height) ;
cdfs_waterVapor_no_model:long_name = "Vertical profile of the cumulative degrees of freedom of signal for water vapor excluding model data" ;
cdfs_waterVapor_no_model:variable_type = "Diagnostic variable" ;
cdfs_waterVapor_no_model:comment1 = "unitless" ;
cdfs_waterVapor_no_model:comment2 = "If no model data is used in the obs vector this field will be the same as cdfs_waterVapor" ;
float cbh(time) ;
cbh:long_name = "Cloud base height above ground level" ;
cbh:units = "km" ;
cbh:variable_type = "Observation used within retrieval" ;
short cbh_flag(time) ;

cbh_flag:long_name = "Flag indicating the source of the cbh" ;
cbh_flag:variable_type = "Diagnostic variable" ;
cbh_flag:comment1 = "unitless" ;
cbh_flag:comment2 = "See users guide for explanation of the values" ;
cbh_flag:value0 = "Value 0 implies Clear Sky" ;
cbh_flag:value1 = "Value 1 implies Inner Window from ceilometer" ;
cbh_flag:value2 = "Value 2 implies Outer Window from ceilometer" ;
cbh_flag:value3 = "Value 3 implies Default CBH" ;
float cbh_tcld(time) ;
cbh_tcld:long_name = "Cloud base height above ground level using Tcld method" ;
cbh_tcld:units = "km" ;
cbh_tcld:variable_type = "Derived from retrieved variables" ;
cbh_tcld:spectral_range = "Spectral window used is 898.00 to 904.00 cm-1" ;
float cbh_mlev(time) ;
cbh_mlev:long_name = "Cloud base height above ground level using MLEV method" ;
cbh_mlev:units = "km" ;
cbh_mlev:variable_type = "Derived from retrieved variables" ;
cbh_mlev:spectral_range = "Spectral window used is 790.00 to 810.00 cm-1" ;
float cld_emis(time) ;
cld_emis:long_name = "Cloud emissivity derived using the MLEV method" ;
cld_emis:comment1 = "unitless" ;
cld_emis:variable_type = "Derived from retrieved variables" ;
cld_emis:spectral_range = "Spectral window used is 790.00 to 810.00 cm-1" ;
float pressure(time, height) ;
pressure:long_name = "Derived pressure" ;
pressure:units = "mb" ;
pressure:variable_type = "Derived from retrieved variables" ;
pressure:comment = "derived from surface pressure observations and the hyposmetric calculation using the thermodynamic profiles" ;
float theta(time, height) ;
theta:long_name = "Potential temperature" ;
theta:units = "degK" ;
theta:variable_type = "Derived from retrieved variables" ;
theta:comment = "This field is derived from the retrieved fields" ;
float thetae(time, height) ;
thetae:long_name = "Equivalent potential temperature" ;
thetae:units = "degK" ;
thetae:variable_type = "Derived from retrieved variables" ;
thetae:comment = "This field is derived from the retrieved fields" ;
float rh(time, height) ;
rh:long_name = "Relative humidity" ;
rh:units = "%" ;
rh:variable_type = "Derived from retrieved variables" ;
rh:comment = "This field is derived from the retrieved field" ;
float dewpt(time, height) ;
dewpt:long_name = "Dew point temperature" ;
dewpt:units = "degC" ;
dewpt:variable_type = "Derived from retrieved variables" ;
dewpt:comment = "This field is derived from the retrieved fields" ;
float co2_profile(time, height) ;
co2_profile:long_name = "CO2 profile" ;
co2_profile:units = "ppm" ;
co2_profile:variable_type = "Derived from retrieved variables" ;
co2_profile:comment = "This field is derived from the retrieved fields" ;
float ch4_profile(time, height) ;
ch4_profile:long_name = "CH4 profile" ;
ch4_profile:units = "ppm" ;
ch4_profile:variable_type = "Derived from retrieved variables" ;

ch4_profile:comment = "This field is derived from the retrieved fields" ;
float n2o_profile(time, height) ;
n2o_profile:long_name = "N2O profile" ;
n2o_profile:units = "ppm" ;
n2o_profile:variable_type = "Derived from retrieved variables" ;
n2o_profile:comment = "This field is derived from the retrieved fields" ;
float pwv(time) ;
pwv:long_name = "Precipitable water vapor" ;
pwv:units = "cm" ;
pwv:variable_type = "Derived from retrieved variables" ;
pwv:comment1 = "This field is derived from the retrieved fields" ;
pwv:comment2 = "A value of -999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float pblh(time) ;
pblh:long_name = "Planetary boundary layer height" ;
pblh:units = "km" ;
pblh:variable_type = "Derived from retrieved variables" ;
pblh:comment1 = "This field is derived from the retrieved fields" ;
pblh:comment2 = "A value of -999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float sbih(time) ;
sbih:long_name = "Surface-based inversion height" ;
sbih:units = "km" ;
sbih:variable_type = "Derived from retrieved variables" ;
sbih:comment1 = "This field is derived from the retrieved fields" ;
sbih:comment2 = "A value of -999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float sbim(time) ;
sbim:long_name = "Surface-based inversion magnitude" ;
sbim:units = "degC" ;
sbim:variable_type = "Derived from retrieved variables" ;
sbim:comment1 = "This field is derived from the retrieved fields" ;
sbim:comment2 = "A value of -999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float sbLCL(time) ;
sbLCL:long_name = "Lifted condesation level for a surface-based parcel" ;
sbLCL:units = "km" ;
sbLCL:variable_type = "Derived from retrieved variables" ;
sbLCL:comment1 = "This field is derived from the retrieved fields" ;
sbLCL:comment2 = "A value of -999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float sbCAPE(time) ;
sbCAPE:long_name = "Convective available potential energy for a surface-based parcel" ;
sbCAPE:units = "J/kg" ;
sbCAPE:variable_type = "Derived from retrieved variables" ;
sbCAPE:comment1 = "This field is derived from the retrieved fields" ;
sbCAPE:comment2 = "A value of -9999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float sbCIN(time) ;
sbCIN:long_name = "Convective inhibition for a surface-based parcel" ;
sbCIN:units = "J/kg" ;
sbCIN:variable_type = "Derived from retrieved variables" ;
sbCIN:comment1 = "This field is derived from the retrieved fields" ;
sbCIN:comment2 = "A value of -9999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float mlLCL(time) ;
mlLCL:long_name = "Lifted condesation level for a mixed-layer parcel" ;
mlLCL:units = "km" ;

mlLCL:variable_type = "Derived from retrieved variables" ;
mlLCL:comment1 = "This field is derived from the retrieved fields" ;
mlLCL:comment2 = "A value of -999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float mlCAPE(time) ;
mlCAPE:long_name = "Convective available potential energy for a mixed-layer parcel" ;
mlCAPE:units = "J/kg" ;
mlCAPE:variable_type = "Derived from retrieved variables" ;
mlCAPE:comment1 = "This field is derived from the retrieved fields" ;
mlCAPE:comment2 = "A value of -9999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float mlCIN(time) ;
mlCIN:long_name = "Convective inhibition for a mixed-layer parcel" ;
mlCIN:units = "J/kg" ;
mlCIN:variable_type = "Derived from retrieved variables" ;
mlCIN:comment1 = "This field is derived from the retrieved fields" ;
mlCIN:comment2 = "A value of -9999 indicates that this field could not be computed (typically because the value was aphysical)" ;
float sigma_pwv(time) ;
sigma_pwv:long_name = "1-sigma uncertainties in precipitable water vapor" ;
sigma_pwv:units = "cm" ;
sigma_pwv:variable_type = "Uncertainty of associated variable" ;
sigma_pwv:comment1 = "This field is derived from the retrieved fields" ;
sigma_pwv:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_pwv:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_pblh(time) ;
sigma_pblh:long_name = "1-sigma uncertainties in the PBL height" ;
sigma_pblh:units = "km" ;
sigma_pblh:variable_type = "Uncertainty of associated variable" ;
sigma_pblh:comment1 = "This field is derived from the retrieved fields" ;
sigma_pblh:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_pblh:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_sbih(time) ;
sigma_sbih:long_name = "1-sigma uncertainties in the surface-based inversion height" ;
sigma_sbih:units = "km" ;
sigma_sbih:variable_type = "Uncertainty of associated variable" ;
sigma_sbih:comment1 = "This field is derived from the retrieved fields" ;
sigma_sbih:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_sbih:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_sbim(time) ;
sigma_sbim:long_name = "1-sigma uncertainties in the surface-based inversion magnitude" ;
sigma_sbim:units = "degC" ;
sigma_sbim:variable_type = "Uncertainty of associated variable" ;
sigma_sbim:comment1 = "This field is derived from the retrieved fields" ;
sigma_sbim:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_sbim:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_sbLCL(time) ;
sigma_sbLCL:long_name = "1-sigma uncertainties in the LCL for a surface-based parcel" ;
sigma_sbLCL:units = "km" ;
sigma_sbLCL:variable_type = "Uncertainty of associated variable" ;

sigma_sbLCL:comment1 = "This field is derived from the retrieved fields" ;
sigma_sbLCL:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_sbLCL:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_sbCAPE(time) ;
sigma_sbCAPE:long_name = "1-sigma uncertainties in surface-based CAPE" ;
sigma_sbCAPE:units = "J/kg" ;
sigma_sbCAPE:variable_type = "Uncertainty of associated variable" ;
sigma_sbCAPE:comment1 = "This field is derived from the retrieved fields" ;
sigma_sbCAPE:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_sbCAPE:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_sbCIN(time) ;
sigma_sbCIN:long_name = "1-sigma uncertainties in surface-based CIN" ;
sigma_sbCIN:units = "J/kg" ;
sigma_sbCIN:variable_type = "Uncertainty of associated variable" ;
sigma_sbCIN:comment1 = "This field is derived from the retrieved fields" ;
sigma_sbCIN:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_sbCIN:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_mlLCL(time) ;
sigma_mlLCL:long_name = "1-sigma uncertainties in the LCL for a mixed-layer parcel" ;
sigma_mlLCL:units = "km" ;
sigma_mlLCL:variable_type = "Uncertainty of associated variable" ;
sigma_mlLCL:comment1 = "This field is derived from the retrieved fields" ;
sigma_mlLCL:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_mlLCL:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_mlCAPE(time) ;
sigma_mlCAPE:long_name = "1-sigma uncertainties in mixed-layer CAPE" ;
sigma_mlCAPE:units = "J/kg" ;
sigma_mlCAPE:variable_type = "Uncertainty of associated variable" ;
sigma_mlCAPE:comment1 = "This field is derived from the retrieved fields" ;
sigma_mlCAPE:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_mlCAPE:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
float sigma_mlCIN(time) ;
sigma_mlCIN:long_name = "1-sigma uncertainties in mixed-layer CIN" ;
sigma_mlCIN:units = "J/kg" ;
sigma_mlCIN:variable_type = "Uncertainty of associated variable" ;
sigma_mlCIN:comment1 = "This field is derived from the retrieved fields" ;
sigma_mlCIN:comment2 = "The uncertainties were determined using a monte carlo sampling of the posterior covariance matrix" ;
sigma_mlCIN:comment3 = "A value of -999 indicates that the uncertainty in this inded could not be computed (typically because the values were all unphysical)" ;
short obs_flag(obs_dim) ;
obs_flag:long_name = "Flag indicating type of observation for each vector element" ;
obs_flag:variable_type = "Observation used within retrieval" ;
obs_flag:comment1 = "unitless" ;
obs_flag:value_01 = "IRS spectral radiance in wavenumber -- i.e., cm^(-1)" ;
obs_flag:value_02 = "MWR spectral brightness temperature [degK] from a zenith-pointing microwave radiometer" ;
obs_flag:value_05 = "Surface met temperature in degC from External surface met station" ;
obs_flag:value_05_comment1 = "Surface met station is 0 m above height=0 level" ;

obs_flag:value_06 = "Surface met water vapor in g/kg from External surface met station" ;
obs_flag:value_06_comment1 = "Surface met station is 0 m above height=0 level" ;
obs_flag:value_10 = "MWR spectral brightness temperature [degK] from an elevation scanning microwave radiometer" ;
obs_flag:value_10_comment1 = "Dimension is coded to be (frequency[GHz]*100)+(elevation_angle[deg]/1000)" ;
double obs_dimension(obs_dim) ;
obs_dimension:long_name = "Dimension of the observation vector" ;
obs_dimension:comment1 = "mixed units -- see obs_flag field above" ;
obs_dimension:variable_type = "Observation used within retrieval" ;
float obs_vector(time, obs_dim) ;
obs_vector:long_name = "Observation vector Y" ;
obs_vector:comment1 = "mixed units -- see obs_flag field above" ;
obs_vector:variable_type = "Observation used within retrieval" ;
float obs_vector_uncertainty(time, obs_dim) ;
obs_vector_uncertainty:long_name = "1-sigma uncertainty in the observation vector (sigY)" ;
obs_vector_uncertainty:comment1 = "mixed units -- see obs_flag field above" ;
obs_vector_uncertainty:variable_type = "Uncertainty of associated variable" ;
float forward_calc(time, obs_dim) ;
forward_calc:long_name = "Forward calculation from state vector (i.e., F(Xn))" ;
forward_calc:comment1 = "mixed units -- see obs_flag field above" ;
forward_calc:variable_type = "Diagnostic variable" ;
short arb1(arb_dim1) ;
arb1:long_name = "Arbitrary dimension" ;
arb1:comment1 = "mixed units" ;
arb1:comment2 = "contains (1) temperature profile, (2) water vapor profile (3) liquid water path, (4) liquid water Reff, (5) ice cloud optical depth, (6) ice cloud Reff, (7) carbon dioxide (8) methane, (9) nitrous oxide" ;
arb1:variable_type = "Diagnostic variable" ;
short arb2(arb_dim2) ;
arb2:long_name = "Arbitrary dimension" ;
arb2:comment1 = "mixed units" ;
arb2:comment2 = "contains (1) temperature profile, (2) water vapor profile (3) liquid water path , (4) liquid water Reff, (5) ice cloud optical depth, (6) ice cloud Reff, (7) carbon dioxide (8) methane, (9) nitrous oxide" ;
arb2:variable_type = "Diagnostic variable" ;
double Xa(arb_dim1) ;
Xa:long_name = "Prior mean state" ;
Xa:comment1 = "mixed units -- see field arb above" ;
Xa:variable_type = "Diagnostic variable" ;
double Sa(arb_dim1, arb_dim2) ;
Sa:long_name = "Prior covariance" ;
Sa:comment1 = "mixed units -- see field arb above" ;
Sa:variable_type = "Diagnostic variable" ;
float Xop(time, arb_dim1) ;
Xop:long_name = "Optimal solution" ;
Xop:comment1 = "mixed units -- see field arb above" ;
Xop:variable_type = "Diagnostic variable" ;
float Sop(time, arb_dim1, arb_dim2) ;
Sop:long_name = "Covariance matrix of the solution" ;
Sop:comment1 = "mixed units -- see field arb above" ;
Sop:variable_type = "Diagnostic variable" ;
float Akernal(time, arb_dim1, arb_dim2) ;
Akernal:long_name = "Averaging kernal" ;
Akernal:comment1 = "mixed units -- see field arb above" ;
Akernal:variable_type = "Diagnostic variable" ;
float lat ;
lat:long_name = "Latitude" ;
lat:units = "degrees_north" ;
float lon ;
lon:long_name = "Longitude" ;
lon:units = "degrees_east" ;

float alt ;
alt:long_name = "station height above mean sea level" ;
alt:units = "m" ;

// global attributes:
:algorithm_code = "TROPoe Retrieval Code (formerly AERIoe)" ;
:algorithm_authors = "Dave Turner, NOAA Global Systems Laboratory (dave.turner@noaa.gov), Josh Gebauer, NOAA National Severe Storms Laboratory / CIWRO (joshua.gebauer@noaa.gov), Tyler Bell, NOAA National Severe Storms Laboratory / CIWRO (tyler.bell@noaa.gov)" ;
:algorithm_comment1 = "TROPoe is a physical-iterative algorithm that retrieves thermodynamic profiles from a wide range of ground-based remote sensors. It was primarily designed to use either infrared spectrometers or microwave radiometers as the primary instrument, and include observations from other sources to improve the quality of the retrieved profiles" ;
:algorithm_comment2 = "Original code was written in IDL and is described by the \"AERIoe\" papers listed below" ;
:algorithm_comment3 = "Code was ported to python, and packaged into a container with the needed radiative transfer models and other required inputs" ;
:algorithm_disclaimer = "TROPoe was developed by NOAA and is provided on an as-is basis, with no warranty" ;
:algorithm_code_version = "0.19-38-gf84b6e3" ;
:algorithm_package_version = "TROPoe-0.20" ;
:algorithm_reference1 = "DD Turner and U Loehnert, 2014: Information Content and Uncertanties in Thermodynamic Profiles and Liquid Cloud Properties Retrieved from the Ground-Based Atmospheric Emitted Radiance Interferometer (AERI), J Appl Met Clim, vol 53, pp 752-771, doi:10.1175/JAMC-D-13-0126.1" ;
:algorithm_reference2 = "DD Turner and WG Blumberg, 2019: Improvements to the AERIoe thermodynamic profile retrieval algorithm. IEEE Selected Topics Appl. Earth Obs. Remote Sens., 12, 1339-1354, doi:10.1109/JSTARS.2018.2874968" ;
:algorithm_reference3 = "DD Turner and U Loehnert, 2021: Ground-based temperature and humidity profiling: Combining active and passive remote sensors, Atmos. Meas. Tech., vol 14, pp 3033-3048, doi:10.5194/amt-14-3033-2021" ;
:forward_model_reference1 = "The forward radiative transfer models are from Atmospheric and Environmental Research Inc (AER); an overview is provided by Clough et al., 2005: Atmospheric radiative transfer modeling: A summary of the AER codes, JQSRT, vol 91, pp 233-244, doi:10.1016/j.jqsrt.2004.05.058" ;
:forward_model_reference2 = "The infrared model is LBLRTM; papers describing this model include doi:10.1029/2018JD029508, doi:10.1175/amsmonographs-d-15-0041.1, and doi:10.1098/rsta.2011.0295" ;
:forward_model_reference3 = "The microwave model is MonoRTM; papers describing this model include doi:10.1109/TGRS.2010.2091416 and doi:10.1109/TGRS.2008.2002435" ;
:datafile_created_on_date = "2025-12-31 00:12:51" ;
:datafile_created_on_machine = "x86_64" ;
:site = "JOYCE Site at Juelich, Germany" ;
:instrument = "HATPRO version5" ;
:mwr_format = "Uni Cologne formatted HATPRO data" ;
:cloud_info = "No ceilometer used; default cloud base height used instead" ;
:Comment_on_recentering1 = "The WVMR profile prior recentered using a surface WVMR value of 6.52 g/kg" ;
:Comment_on_recentering2 = "The WVMR mean profile was scaled by a factor of 0.83" ;
:Comment_on_recentering3 = "The WVMR covariance was scaled by a factor of 0.91" ;
:Comment_on_recentering4 = "The temperature prior was recentered using the converve-RH method" ;
:Prior_dataset_comment = "Mid-latitude prior computed from ARM SGP Radiosondes" ;
:Prior_dataset_filename = "prior.MIDLAT.nc" ;
:Prior_dataset_number_profiles = 5868LL ;
:Prior_dataset_T_inflation_factor = "1.0 at the surface to 1.0 at 1.0 km AGL" ;
:Prior_dataset_Q_inflation_factor = "1.0 at the surface to 1.0 at 1.0 km AGL" ;
:Prior_dataset_TQ_correlation_reduction_factor = 1. ;
:Retrieval_option_flags = "1, 1, 1, 0, 0, 0, 0" ;
:Retrieval_option_flags_variables = "doTemp, doWVMR, doLiqCloud, doIceCloud, doCO2, doCH4, doN2O" ;
:vip_tres = "10 minutes. Note that the sample time corresponds to the center of the averaging interval. A value of 0 implies that no averaging was performed" ;
:Retrieval_start_hour = 0. ;
:Conventions = "CF-1.10" ;
:VIP_tres = 10 ;
:VIP_avg_instant = 1 ;
:VIP_zgrid = "0.000, 0.010, 0.021, 0.033, 0.046, 0.061, 0.077, 0.095, 0.114, 0.136, 0.159, 0.185, 0.214, 0.245, 0.280, 0.318, 0.359, 0.405, 0.456, 0.512, 0.573, 0.640, 0.714, 0.795, 0.885, 0.983, 1.092, 1.211, 1.342, 1.486, 1.645, 1.819, 2.011, 2.223, 2.455,

2.710, 2.991, 3.300, 3.640, 4.014, 4.426, 4.879, 5.376, 5.924, 6.526, 7.189, 7.918, 8.720, 9.602,10.572,11.639,12.813,14.104,15.525,17.087" ;
:VIP_station_lat = 50.909 ;
:VIP_station_lon = 6.414 ;
:VIP_station_alt = 84. ;
:VIP_station_pres = 1000. ;
:VIP_station_psfc_min = 800. ;
:VIP_station_psfc_max = 1040. ;
:VIP_omb_flag = 0 ;
:VIP_omb_path = "/data" ;
:VIP_omb_file = "omb_input.nc" ;
:VIP_omb_required_minht = 10. ;
:VIP_irs_type = -1 ;
:VIP_irsch1_path = "/data/aerich1" ;
:VIP_irsch2_path = "/data/aerich2" ;
:VIP_irssum_path = "/data/aerisum" ;
:VIP_irseng_path = "/data/aerieng" ;
:VIP_irs_zenith_scene_mirror_angle = 180 ;
:VIP_irs_min_noise_flag = 1 ;
:VIP_irs_min_noise_wnum = "500,522,546,575,600,631,747,1439,1770,1884,2217,3000" ;
:VIP_irs_min_noise_spec = "65.309,14.056,3.283,1.333,0.813,0.557,0.304,0.581,0.822,0.025,0.023,0.044" ;
:VIP_irs_noise_inflation = 1. ;
:VIP_irs_smooth_noise = 0 ;
:VIP_irs_band_noise_inflation = 1 ;
:VIP_irs_band_noise_wnums = "775.0,810.0" ;
:VIP_irs_band_noise_sfc_npts = 4 ;
:VIP_irs_band_noise_sfc_wvmr_vals = "0.,7,12,18" ;
:VIP_irs_band_noise_sfc_multiplier = "20.0,20,1,1" ;
:VIP_irs_calib_pres = 0., 1. ;
:VIP_irs_use_missingDataFlag = 1 ;
:VIP_irs_hatch_switch = 1 ;
:VIP_irs_ignore_status_hatch = 0 ;
:VIP_irs_ignore_status_missingDataFlag = 0 ;
:VIP_irs_fv = 0. ;
:VIP_irs_fa = 0. ;
:VIP_irs_old_ffov_halfangle = 23. ;
:VIP_irs_new_ffov_halfangle = 0. ;
:VIP_irs_min_675_bt = 263. ;
:VIP_irs_max_675_bt = 313. ;
:VIP_irs_spec_cal_factor_ch1 = 1. ;
:VIP_irs_spec_cal_factor_ch2 = 1. ;
:VIP_irs_mlev_cbh_wnum_range = 790., 810. ;
:VIP_irs_tcld_wnum_range = 898., 904. ;
:VIP_irs_1m_od_thres = 1. ;
:VIP_mwr_type = 3 ;
:VIP_mwr_path = "/data/input_data/mwr.hatpro_juelich" ;
:VIP_mwr_rootname = "sups_joy_mwr00_l1_tb_zo_p00" ;
:VIP_mwr_elev_field = "ele" ;
:VIP_mwr_freq_field = "freq_sb" ;
:VIP_mwr_n_tb_fields = 14 ;
:VIP_mwr_tb_field_names = "tb" ;
:VIP_mwr_tb_field1_tbmax = 100. ;
:VIP_mwr_tb_freqs = "22.24, 23.04, 23.84, 25.44, 26.24, 27.84, 31.4, 51.26, 52.28, 53.86, 54.94, 56.66, 57.3, 58.0" ;
:VIP_mwr_tb_noise = "0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.7, 0.7, 0.5, 0.3, 0.2, 0.2, 0.2" ;
:VIP_mwr_tb_bias = "0.04, 0.04, 0.45, -0.57, -0.02, 0.20, -0.20, 1.94, 2.42, -0.79, 0.21, 0.05, 0.02, -0.04" ;
:VIP_mwr_tb_replicate = 1 ;
:VIP_mwr_time_delta = 0.083 ;
:VIP_mwrscan_type = 3 ;

```
:VIP_mwrscan_path = "/data/input_data/mwr.hatpro_juelich" ;
:VIP_mwrscan_rootname = "sups_joy_mwrBL00_l1_tb_p00" ;
:VIP_mwrscan_freq_field = "freq_sb" ;
:VIP_mwrscan_elev_field = "ele" ;
:VIP_mwrscan_n_tb_fields = 3 ;
:VIP_mwrscan_n_elevations = 3 ;
:VIP_mwrscan_elevations = "30.0, 19.2, 10.2" ;
:VIP_mwrscan_tb_field1_tbmax = 330. ;
:VIP_mwrscan_tb_field_names = "tb" ;
:VIP_mwrscan_tb_freqs = "56.66, 57.3, 58.0" ;
:VIP_mwrscan_tb_noise = "0.2,  0.2,  0.2" ;
:VIP_mwrscan_tb_bias = "0.05,  0.02, -0.04" ;
:VIP_mwrscan_time_delta = 0.25 ;
:VIP_ext_sfc_temp_type = 1 ;
:VIP_ext_sfc_temp_fieldname = "ta" ;
:VIP_ext_sfc_temp_units = 2 ;
:VIP_ext_sfc_temp_npts = 1 ;
:VIP_ext_sfc_temp_random_error = 0.5 ;
:VIP_ext_sfc_temp_rep_error = 0. ;
:VIP_ext_sfc_wv_type = 1 ;
:VIP_ext_sfc_wv_fieldname = "hur" ;
:VIP_ext_sfc_wv_units = 2 ;
:VIP_ext_sfc_wv_npts = 1 ;
:VIP_ext_sfc_rh_random_error = 3. ;
:VIP_ext_sfc_wv_mult_error = 1. ;
:VIP_ext_sfc_wv_rep_error = 0. ;
:VIP_ext_sfc_pres_type = 1 ;
:VIP_ext_sfc_pres_fieldname = "pa" ;
:VIP_ext_sfc_pres_units = 3 ;
:VIP_ext_sfc_path = "/data/input_data/mwr.hatpro_juelich" ;
:VIP_ext_sfc_rootname = "sups_joy_mwr00_l1_tb_zo_p00" ;
:VIP_ext_sfc_time_format = 2 ;
:VIP_ext_sfc_time_delta = 0.2 ;
:VIP_ext_sfc_relative_height = 0 ;
:VIP_ext_wv_prof_type = 0 ;
:VIP_ext_wv_prof_path = "None" ;
:VIP_ext_wv_prof_minht = 0. ;
:VIP_ext_wv_prof_maxht = 10. ;
:VIP_ext_wv_prof_min_error = 0.01 ;
:VIP_ext_wv_noise_mult_val = 1., 1., 1. ;
:VIP_ext_wv_noise_mult_hts = 0., 3., 20. ;
:VIP_ext_wv_add_rel_error = 0. ;
:VIP_ext_wv_ht_offset = 0. ;
:VIP_ext_wv_time_delta = 1. ;
:VIP_ext_temp_prof_type = 0 ;
:VIP_ext_temp_prof_path = "None" ;
:VIP_ext_temp_prof_minht = 0. ;
:VIP_ext_temp_prof_maxht = 10. ;
:VIP_ext_temp_noise_adder_val = 0., 0., 0. ;
:VIP_ext_temp_noise_adder_hts = 0., 3., 20. ;
:VIP_ext_temp_ht_offset = 0. ;
:VIP_ext_temp_time_delta = 1. ;
:VIP_mod_wv_prof_type = 0 ;
:VIP_mod_wv_prof_path = "None" ;
:VIP_mod_wv_prof_minht = 0. ;
:VIP_mod_wv_prof_maxht = 10. ;
:VIP_mod_wv_prof_min_error = 0.01 ;
:VIP_mod_wv_noise_mult_val = 1., 1., 1. ;
```

:VIP_mod_wv_noise_mult_hts = 0., 3., 20. ;
:VIP_mod_wv_ht_offset = 0. ;
:VIP_mod_wv_time_delta = 1. ;
:VIP_mod_temp_prof_type = 0 ;
:VIP_mod_temp_prof_path = "None" ;
:VIP_mod_temp_prof_minht = 0. ;
:VIP_mod_temp_prof_maxht = 10. ;
:VIP_mod_temp_noise_adder_val = 0., 0., 0. ;
:VIP_mod_temp_noise_adder_hts = 0., 3., 20. ;
:VIP_mod_temp_ht_offset = 0. ;
:VIP_mod_temp_time_delta = 1. ;
:VIP_add_tropoe_T_input_flag = 0 ;
:VIP_add_tropoe_q_input_flag = 0 ;
:VIP_add_tropoe_input_lwp_thres = 10. ;
:VIP_add_tropoe_input_cbh_thres = 5. ;
:VIP_add_tropoe_gamma_threshold = 2 ;
:VIP_add_tropoe_T_noise_adder_val = 3., 1., 1. ;
:VIP_add_tropoe_T_noise_adder_hts = 0., 1., 20. ;
:VIP_add_tropoe_q_noise_mult_val = 5., 2., 2. ;
:VIP_add_tropoe_q_noise_mult_hts = 0., 1., 20. ;
:VIP_add_tropoe_use_prior_as_max = 1 ;
:VIP_add_tropoe_use_pblh_flag = 1 ;
:VIP_co2_sfc_type = 0 ;
:VIP_co2_sfc_npts = 1 ;
:VIP_co2_sfc_rep_error = 0. ;
:VIP_co2_sfc_path = "None" ;
:VIP_co2_sfc_relative_height = 0 ;
:VIP_co2_sfc_time_delta = 1.5 ;
:VIP_cbh_type = 2 ;
:VIP_cbh_path = "None" ;
:VIP_cbh_window_in = 20 ;
:VIP_cbh_window_out = 180 ;
:VIP_cbh_default_ht = 1. ;
:VIP_output_akernal = 1 ;
:VIP_lbl_home = "/home/tropoe/vip/src/lblrtm_v12.17/lblrtm" ;
:VIP_lbl_std_atmos = 6 ;
:VIP_path_std_atmos = "/home/tropoe/vip/src/input/idl_code/std_atmosphere.idl" ;
:VIP_lbl_tape3 = "TAPE3.350-1600cm-1.first_7_molecules" ;
:VIP_monortm_version = "v5.0" ;
:VIP_monortm_wrapper = "/home/tropoe/vip/src/monortm_v5.0/wrapper/monortm_v5" ;
:VIP_monortm_exec = "/home/tropoe/vip/src/monortm_v5.0/monortm/monortm_v5.0_linux_intel_sgl" ;
:VIP_monortm_spec =
"/home/tropoe/vip/src/monortm_v5.0/monolnfl_v1.0/TAPE3.spectral_lines.dat.0_55.v5.0_veryfast" ;
:VIP_lblrtm_jac_option = 4 ;
:VIP_lblrtm_forward_threshold = 0. ;
:VIP_lblrtm_jac_interpol_npts_wnum = 10 ;
:VIP_monortm_jac_option = 2 ;
:VIP_jac_max_ht = 8. ;
:VIP_max_iterations = 10 ;
:VIP_cvgmult = 0.25 ;
:VIP_first_guess = 1 ;
:VIP_superadiabatic_maxht = 0.3 ;
:VIP_spectral_bands = "612.0-618.0,624.0-660.0,674.0-713.0,713.0-722.0,538.0-588.0,793.0-804.0,860.1-864.0,872.2-
877.5,898.2-905.4" ;
:VIP_retrieve_temp = 1 ;
:VIP_retrieve_wvmr = 1 ;
:VIP_retrieve_co2 = 0 ;
:VIP_fix_co2_shape = 0 ;

```
    :VIP_retrieve_ch4 = 0 ;
    :VIP_fix_ch4_shape = 0 ;
    :VIP_retrieve_n2o = 0 ;
    :VIP_fix_n2o_shape = 0 ;
    :VIP_retrieve_lcloud = 1 ;
    :VIP_retrieve_icloud = 0 ;
    :VIP_lcloud_ssp = "/home/tropoe/vip/src/input/ssp_db_files/ssp_db.mie_wat.gamma_sigma_0p100" ;
    :VIP_icloud_ssp = "/home/tropoe/vip/src/input/ssp_db_files/ssp_db.mie_ice.gamma_sigma_0p100" ;
    :VIP_qc_rms_value = 10. ;
    :VIP_qc_gamma_value = 5. ;
    :VIP_recenter_prior = 1 ;
    :VIP_recenter_input = 0. ;
    :VIP_recenter_covar_min_sfactor = 0.6 ;
    :VIP_recenter_covar_max_sfactor = 1.4 ;
    :VIP_prior_tikhonov_factor = 1.5e-05 ;
    :VIP_prior_t_ival = 1. ;
    :VIP_prior_t_iht = 1. ;
    :VIP_prior_q_ival = 1. ;
    :VIP_prior_q_iht = 1. ;
    :VIP_prior_tq_cov_val = 1. ;
    :VIP_prior_chimney_ht = 0. ;
    :VIP_prior_co2_mn = 418.527018404839, 5., -5. ;
    :VIP_prior_co2_sd = 2., 15., 3. ;
    :VIP_prior_ch4_mn = 1.793, 0., -5. ;
    :VIP_prior_ch4_sd = 0.0538, 0.0015, 3. ;
    :VIP_prior_n2o_mn = 0.31, 0., -5. ;
    :VIP_prior_n2o_sd = 0.0093, 0.0002, 3. ;
    :VIP_prior_lwp_mn = 0. ;
    :VIP_prior_lwp_sd = 200. ;
    :VIP_prior_lReff_mn = 8. ;
    :VIP_prior_lReff_sd = 4. ;
    :VIP_prior_itau_mn = 0. ;
    :VIP_prior_itau_sd = 50. ;
    :VIP_prior_iReff_mn = 25. ;
    :VIP_prior_iReff_sd = 8. ;
    :VIP_min_PBL_height = 0.3 ;
    :VIP_max_PBL_height = 5. ;
    :VIP_nudge_PBL_height = 0.5 ;
    :VIP_plot_output = 0 ;
    :VIP_plot_path = "/data/usecase02.mwr_only.zenith_and_elevation" ;
    :VIP_plot_rootname = "tropoe.zenith_and_elevation" ;
    :VIP_plot_xlim = 0., 24. ;
    :VIP_plot_ylim = 0., 2. ;
    :VIP_plot_temp_lim = -10., 20. ;
    :VIP_plot_wvmr_lim = 0., 20. ;
    :VIP_plot_tuncert_lim = 0., 3. ;
    :VIP_plot_wvuncert_lim = 0., 3. ;
    :VIP_plot_theta_lim = 290., 320. ;
    :VIP_plot_rh_lim = 0., 100. ;
    :VIP_plot_thetae_lim = 290., 320. ;
    :VIP_plot_dewpt_lim = -15., 25. ;
    :VIP_plot_comment = "" ;
    :VIP_plot_tres_min_gap = 30 ;
    :VIP_plot_lwp_cbh_threshold = 8. ;
    :Total_clock_execution_time_in_s = "7853.4793" ;
}
```

17) References

Adler, B., D.D. Turner, L. Bianco, I.V. Djalalova, T. Myers, and J.M. Wilczak, 2024: Improving solution availability and temporal consistency of an optimal-estimation physical retrieval for ground-based thermodynamic profiling. *Atmos. Meas. Technol*., **17**, 6603-6624, doi:10.5194/amt-17-6603-2024.

Adler, B., and N. Kalthoff, 2014: Multi-scale transport processes observed in the boundary layer over a mountainous island. *Bound. Lay. Meteor.*, **153**, 515-537, doi:10.1007/s10546-014-9957-8.

Benjamin, S.G., and coauthors, 2016: A North American hourly assimilation and model forecast cycle: The rapid refresh. *Monthly Wea. Rev.,* **144**, 1669-1694, doi:10.1175/MWR-D-15-0242.1

Blumberg, W.G., D.D. Tuner, U Löhnert, and S. Castleberry, 2015: Ground based temperature and humidity profiling using spectral infrared and microwave observations. Part II: Actual retrieval performance in clear-sky and cloudy conditions. *J. Appl. Meteor. Climatol.*, **54**, 2305 – 2319.

Cadeddu, M.P., J.C. Liljegren, and D.D. Turner, 2013: The Atmospheric Radiation Measurement (ARM) program network of microwave radiometers: Instruments, data, and retrievals. *Atmos. Meas. Tech.*, **6**, 2359-2372, doi:10.5194/amt-6-2359-2013.

Cimini, D., F. Nasir, E.R. Westwater, V.H. Payne, D.D. Turner, E.J. Mlawer, M.L. Exner, and M. Cadeddu, 2009: Comparison of ground-based millimeter-wave observations and simulations in the Arctic winter. *IEEE Trans. Geosci. Remote Sens*., **47**, 3098-3106, doi:10.1109/TGRS.2009.2020743

Clough, S.A., and M.J. Iacono, 1995: Line-by-line calculation of atmospheric fluxes and cooling rates. 2: Applications to carbon dioxide, ozone, methane, nitrous oxide, and halocarbons. J Geophys. Res., 100, 16519-16535.

Clough, S.A., M.W. Shephard, E.J. Mlawer, J.S. Delamere, M.J. Iacono, K. Cady-Pereira, S. Boukabara, and P.D. Brown, 2005: Atmospheric radiative transfer modeling: A summary of the AER codes. *J. Quant. Spectroc. Radiative Trans.*, **91**, 233-244, doi:10.1016/j.jqsrt.2004.05.058.

Crewell, S., and U. Löhnert, 2007: Accuracy of boundary layer temperature profiles retrieved with multi-frequency, multi-angle microwave radiometry, *IEEE Trans. Geosci. Remote Sens.*, **45**(7), 2195–2201.

Di Natale, G., L. Palchetti, G. Bianchini, and M. Del Guasta, 2017: Simultaneous retrieval of water vapour, temperature and cirrus clouds properties from measurements of far infrar3ed spectral radiance over the Antarctic Plateau. *Atmos. Meas. Tech*., 10, 825-837, doi:10.5194/amt-10-825-2017.

Djalalova, I.V., D.D. Turner, L. Bianco, J.M Wilczak, J. Duncan, B. Adler, and D. Gottas, 2022: Improving thermodynamic profile retrievals from microwave radiometers by including radio acoustic sounding system (RASS) observations. *Atmos. Meas. Tech*., **15**, 521-537, doi:10.5194/amt-15-521-2022

Feltz, W.F., W.L. Smith, R.O. Knuteson, H.E. Revercomb, H.M. Woolf, and H.B. Howe, 1998: Meteorological applications of temperature and water vapor retrievals from the ground-based Atmospheric Emitted Radiance Interferometer (AERI). *J. Appl. Meteor.*, **37**, 857-875.

Feltz, W. F., W.L. Smith, H.B. Howe, R.O. Knuteson, H. Woolf, and H.E. Revercomb, 2003: Near-continuous profiling of temperature, moisture, and atmospheric stability using the Atmospheric Emitted Radiance Interferometer (AERI). *J. Appl. Meteor*, **42**, 584-597.

Guy, H., D.D. Turner, V.P. Walden, I.M. Brooks, and R.R. Neely III, 2022: Passive ground-based remote sensing of radiation fog. *Atmos. Meas. Tech*., **15**, 5095-5115, doi:10.5194/amt-15-5095-2022.

Guy, H., I.M. Brooks, D.D. Turner, C.J. Cox, P.M. Rowe, M.D. Shupe, V.P. Walden, and R.R. Neely III, 2023: Observations of fog-aerosol interactions over central Greenland. *J. Geophys. Res*., **128**, e2023JD038718, doi:10.1029/2023JD038718

Huang, H.-L., W.L. Smith, J. Li, P. Antonei, X. Wu, R.O. Knuteson, B. Huang, and B.J. Osborne, 2004: Minimum local emissivity variance retrieval of cloud altitude and effective spectral emissivity – simulation and initial verification. *J. App. Meteor*., **43**, 795-809.

Knuteson, R. O., and coauthors, 2004a: Atmospheric Emitted Radiance Interferometer. Part I: Instrument design. *J. Atmos. Oceanic Technol.*, **21**, 1763-1776.

Knuteson, R. O., and coauthors, 2004b: Atmospheric Emitted Radiance Interferometer. Part II: Instrument performance. *J. Atmos. Oceanic Technol.*, **21**, 1777-1789.

Löhnert, U., D.D. Turner, and S. Crewell, 2009: Ground-based temperature and humidity profiling using spectral infrared and microwave observations. Part 1: Simulated retrieval performance in clear sky conditions. *J. Appl. Meteor. Clim.*, **48**, 1017-1032, doi:10.1175/2008JAMC2060.1

Löhnert, U., and O. Maier, 2012: Operational profiling of temperature using ground-based microwave radiometry at Payerne: Prospects and challenges. *Atmos. Meas. Techniq.*, **5**, 1121-1134, doi:10.5194/amt-5-1121-2012.

Löhnert, U., J.H. Schween, C. Acquistapace, K. Ebell, M. Maahn, M. Barrera-Verdejo, A. Hirsikko, B. Bohn, A, Knaps, E. O'Connor, C. Simmer, A. Wahner, and S. Crewell, 2015: JOYCE: Jülich Observatory for Cloud Evolution. *Bull. Amer. Meteor. Soc*, 96, 1157-1174, doi:10.1175/BAMS-D-14-00105.1.

Maahn, M., D.D. Turner, U. Loehnert, D.J. Posselt, K. Ebe, G.G. Mace, and J.M. Comstock, 2020: Optimal estimation retrievals and their uncertainties: What every atmospheric scientist should know. Bull. Amer. Meteor. Soc., 101, E1512-E1523, doi:10.1175/BAMS-D-19-0027.1

Marke, T., K. Ebe, U. Loehnert, and D.D. Turner, 2016: Statistical retrieval of thin liquid cloud microphysical properties using ground-based infrared and microwave observations. *J. Geophys. Res*., **121**, 14558-14573, doi:10.1002/2016JD025667.

Michaud-Belleau, V., M. Gaudreau, J. Lacoursiere, E. Boisvert, L. Ravelomanantsoa, D.D. Turner, and L. Rochette, 2025: The Atmospheric Sounder Spectrometer by Infrared Spectral Technology (ASSIST): Instrument design and signal processing. Atmos. Meas. Tech., 18, 3585-3609, doi:10.5194/amt-18-3585-2025.

Mlawer, E.J., J. Mascio, D.D. Turner, V.E. Payne, C.J. Flynn, and R. Pincus, 2024: A more transparent infrared window. *J. Geophys. Res*., **129**, e2024JD041366, doi:10.1029/2024JD041366.

Nielson-Gammon, J.W., C.L. Powe, M.J. Mahoney, W.M. Angevine, C. Senff, A. White, C. Berkowitz, C. Doran, and K. Knupp, 2008: Multisensor estimation of mixing heights over a coastal city. J. Appl. Meteor. Clim., 47, 27-43, doi:10.1175/2007JAMC1503.1

Payne, V.H., E.J. Mlawer, K.E. Cady-Pereira, and J.-L. Moncet, 2011: Water vapor continuum absorption in the microwave. *IEEE Trans. Geosci. Remote Sens.*, **49**, 2194-2208, doi:10.1109/TGRS.2010.2091416.

Revercomb, H.E., H. Buijs, H.B. Howe, D.D. LaPorte, W.L. Smith, and L.A. Sromovsky, 1988: Radiometric calibration of IR Fourier transform spectrometers: Solution to a problem with the high-resolution interferometer sounder. *Appl. Opt.*, 27, 3210-3218.

Rodgers, C.D., 2000: *Inverse Methods for Atmospheric Sounding: Theory and Practice*. Series on Atmospheric, Oceanic, and Planetary Physics, Vol. 2, World Scientific, 238 pp.

Rose, T., S. Crewell, U. Löhnert, and C. Simmer, 2005: A network suitable microwave radiometer for operational monitoring of the cloudy atmosphere. *Atmos. Res.*, **75**, 183-200, doi:10.1016/j.atmosres.2004.12.005.

Rosenkranz, P.W., 2017: Line-by-line microwave radiative transfer (nonscattering). Remote Sens. Code Library. [Online]. Available: http://cetemps.aquila.infn.it/mwrnet/lblmrt_ns.html, doi: 10.21982/ M81013.

Sisterson, D.L, R.A. Peppler, T.S. Cress, P.J. Lamb, and D.D. Turner, 2016: The ARM Southern Great Plains (SGP) site. *The Atmospheric Radiation Measurement Program: The First 20 Years*, Meteor. Monograph. Amer. Meteor. Soc. **57**, 6.1-6.14, DOI:10.1175/AMSMONOGRAPHS-D-16-0004.1.

Smith, W.L., W.F. Feltz, R.O. Knuteson, H.E. Revercomb, H.M. Woolf, and H.B. Howe, 1999: The retrieval of planetary boundary layer structure using ground-based infrared spectral radiance measurements. *J. Atmos. Oceanic Technol.*, **16**, 323-333.

Solheim, F., J.R. Godwin, E.R. Westwater, Y. Han, S.J. Keihm, K. Marsh, and R. Ware, 1998: Radiometric profiling of temperature, water vapor, and cloud liquid water using various inversion methods. *Radio Sci.*, **33**, 393-404.

Schween, J.H., S. Crewell, and U. Löhnert, 2011: Horizontal-humidity gradient from one single-scanning microwave radiometer. *IEEE Geosci. Remote Sens. Lett.*, 8, 336-340, doi:10.1109/LGRS.2010.2072981.

Troyan, D., 2012: Merged sounding value-added product. DOE ARM technical report DOE/SC-ARM/TR-087. Available from https://www.arm.gov/publications/tech_reports/doe-sc-arm-tr-087.pdf.

Turner, D.D., and J.E.M. Goldsmith, 1999: Twenty-four-hour Raman lidar water vapor measurements during the Atmospheric Radiation Measurement Program's 1996 and 1997 water vapor intensive observation period. *J. Atmos. Oceanic Tech.*, **16**, 1062-1076.

Turner, D.D., 2003: Microphysical properties of single and mixed-phase Arctic clouds derived from ground-based AERI observations. Ph.D. Dissertation, University of Wisconsin-Madison, 167 pp. Available from https://search.library.wisc.edu/catalog/999951770902121.

Turner, D.D., 2005: Arctic mixed-phase cloud properties from AERI-lidar observations: Algorithm and results from SHEBA. J. Appl. Meteor., 44, 427-444, doi:10.1175/JAM2208.1.

Turner, D.D., R.O. Knuteson, H.E. Revercomb, C. Lo, and R.G. Dedecker, 2006: Noise reduction of Atmospheric Emitted Radiance Interferometer (AERI) observations using principal component analysis. *J. Atmos. Oceanic Technol.*, **23**, 1223-1238.

Turner, D.D., 2007: Improved ground-based liquid water path retrievals using a combined infrared and microwave approach. *J. Geophys. Res.*, **112**, D15204, doi:10.1029/2007JD008530.

Turner, D.D., and E.J. Mlawer, 2010: The Radiative Heating in Underexplored Bands Campaigns (RHUBC). *Bull. Amer. Meteor. Soc.*, **91**, doi:10.1175/2010BAMS2904.1.

Turner, D. D., and U. Löhnert 2014: Information content and uncertainties in thermodynamic profiles and liquid cloud properties retrieved from the ground-based Atmospheric Emitted Radiance Interferometer (AERI). *J. Appl. Meteor. Climatol.*, **53**, 752-771, doi:10.1175/JAMC-D-13-0126.1.

Turner, D.D., and R.G. Ellingson, Eds., 2016: *The Atmospheric Radiation Measurement (ARM) Program: The First 20 Years*. Meteor. Monograph, **57**, American Meteorological Society.

Turner, D.D., E.J. Mlawer, and H.E. Revercomb, 2016a: Water vapor observations in the ARM program. *The Atmospheric Radiation Measurement Program: The First 20 Years*. Meteor. Monograph, 57, Amer. Meteor. Soc., 13.1-13.18, doi:10.1175/AMSMONOGRAPHS-D-15-0025.1

Turner, D.D., S. Kneifel, and M.P. Cadeddu, 2016b: An improved liquid water absorption model at microwave frequencies for supercooled liquid water clouds. J. *Atmos. Oceanic Technol.*, **33**, 33-44, doi:10.1175/JTECH-D-15-0074.1.

Turner, D.D., and W.G. Blumberg, 2019: Improvements to the AERIoe thermodynamic profile retrieval algorithm. *IEEE Selected Topics Appl. Earth Obs. Remote Sens.*, 12, 1339-1354, doi:10.1109/JSTARS.2018.2874968.

Turner, D.D., and U. Löhnert, 2021: Ground-based temperature and humidity profiling: Combining active and passive remote sensors. *Atmos. Meas. Technol*., 14, 3033-3048, doi:10.5194/amt-14-3033-2021

Turner, D.D., M.P. Cadeddu, J. Simonson, and T.J. Wagner, 2025: Propagating information content: An example with advection. *Atmos. Meas. Tech*., **18**, 3533-3546, doi:10.5194/amt-18-3533-2025.

Turner, D.D., B. Adler, L. Bianco, J.M. Wilczak, V. Michaud-Belleau, and L. Rochette, 2025: Intercomparison of seven collocated ground-based interferometers and retrieved thermodynamic profiles. Atmos. Meas. Technol., submitted

Von Klitzing, L., D.D. Turner, D. Lange, and V. Wulfmeyer, 2026: Improved method for temporally interpolating radiosonde profiles in the convective boundary layer. *Atmos. Meas. Tech.*, accepted.

Wagner, T. J., W. F. Feltz, and S. A. Ackerman, 2008: The temporal evolution of convective indices in storm-producing environments. *Wea. Forecasting*, **23**, 786 – 794.

Wagner, T.J., P.M. Klein, and D.D. Turner, 2019: A new generation of ground-based mobile platforms for active and passive profiling of the boundary layer. *Bull. Amer. Meteor. Soc.*, **100**, 137-153, doi:10.1175/BAMS-D-17-0165.1

Wulfmeyer, V., R.M. Hardesty, D.D. Turner, A. Behrendt, M. Cadeddu, P. Di Girolamo, P. Schluessel, J. van Baelen, and F. Zus, 2015: A review of the remote sensing of lower-tropospheric thermodynamic profiles and its indispensable role for the understanding and

simulation of water and energy cycles. *Rev. Geophys*., **53**, 819-895, doi:10.1002/2014RG000476